

尼恩：百亿级数据存储架构起源

在40岁老架构师 尼恩的**读者交流群**(50+)中，经常性的指导小伙伴们改造简历。

经过尼恩的改造之后，很多小伙伴拿到了一线互联网企业如得物、阿里、滴滴、极兔、有赞、希音、百度、网易、美团的面试机会，拿到了大厂机会。

这些机会的来源，主要是尼恩给小伙伴 改造了简历，植入了亮点项目、黄金项目。

尼恩的 亮点项目、黄金项目 需要持续迭代。

下一个亮点项目、黄金项目是：百亿级数据存储架构。

同时，小伙伴在面试时，经常遇到这个面试难题。比如，前几天一个小伙伴面试字节，就遇到了这道题

字节面试：百亿级数据存储，怎么设计？只是分库分表吗？

于是，尼恩组织小伙伴开始研究和 设计 《百亿级数据存储架构》，帮助大家打造一个新的黄金项目，实现大厂的梦想。

这个文档将成为大家 面试的杀手锏，此文当最新PDF版本，可以找40岁老架构师尼恩获取。

ElasticSearch

学习圣经

尼恩 编著

未来职业,如何突围:三栖架构师

FASAC三栖合一架构师

Future Super Architect Community

- **第一栖: Java架构**
- **第二栖: Go架构**
- **第三栖: 大数据架构**

一次穿透 ElasticSearch 工业级开发

疯狂创客圈Java高并发社区

(技术自由圈 内部资料)

当然,上面的面试题以及参考答案,也会收入咱们的《[尼恩Java面试宝典PDF](#)》V171版本,供后面的小伙伴参考,提升大家的3高架构、设计、开发水平。

最新《尼恩架构笔记》《尼恩高并发三部曲》《尼恩Java面试宝典》的PDF,请关注本公众号【技术自由圈】获取,回复:领电子书

本文作者:

- 第一作者 Moen (负责写初稿)
- 第二作者 尼恩 (40岁老架构师,负责提升此文的技术高度,让大家有一种俯视技术的感觉)

本文目录

尼恩：百亿级数据存储架构起源

本文目录

百亿级数据存储架构，只有分库分表吗？

从0到1，百亿级数据存储架构，怎么设计？

百亿级数据存储架构，怎么维护多副本之间的数据一致？

从0到1，百亿级数据存储架构，怎么设计？

ElasticSearch 从入门到工业级使用

1.1 什么是全文检索

1.2 ES简介

1.3 ES基本概念

2 安装和DSL的使用

2.1 安装ES

2.2 安装Kibana

2.3通过脚本一键启动ES

2.4使用 DSL 操作ES

2.4.1DSL 定义基本介绍

DSL (Domain Specific Language)，一种特定领域的查询语言，用于构建复杂的查询和聚合操作。

2.4.2DSL 定义语法说明

- (1) 关键字(Keywords)
- (2) 标识符(Identifiers)
- (3) 表达式(Expressions)
- (4) 运算符(Operators)
- (5) 函数调用(Function Calls)
- (6) 控制流(Control Flow)
- (7) 注释(Comments)

2.4.3DSL常见语法

- (1) 查询 (Query)
- (2) 过滤 (Filter)
- (3) 聚合 (Aggregation)

2.4.4DSL示例

1.单独查询

- (1) 基本查询 (Match Query)
- (2) 多字段查询 (Multi-match Query)
- (3) 范围查询 (Range Query)
- (4) 布尔查询 (Bool Query)
- (5) 聚合查询 (Aggregation Query)
- (6) 排序 (Sorting)
- (7) 分页 (Pagination)

2.组合查询

- (1) 组合多个must查询
- (2) 组合must和should查询
- (3) 组合must和must_not查询
- (4) 组合多个should查询
- (5) 嵌套组合查询

3 ES 的分词器

3.1 倒排索引

3.2 默认分词器

3.3 中文分词

3.4 IK分词器

3.5 自定义扩展字典

4 高级的DSL 查询

4.1 管理索引

4.2 数据查询

4.3 查询模式

4.4 使用别名

5 从0开始，ES工业级Java 应用开发

5.1 High Level Client基本用法

5.2 聚合查询

5.2.1 DSL聚合查询

5.2.2 ES客户端实现聚合查询

5.3 ES ES工业级Java 应用开发

6 ES集群架构

6.1 ES集群的5大角色

6.2 ES集群节点角色配置

6.3 ES节点配置组合

6.4 高可用ES的部署架构

小型的ES集群 (<10) 的节点架构

中型的ES集群 (10-50) 的节点架构

超大型的ES集群的节点架构 (150个节点+)

6.5 小型ES集群的安装

6.5.1 Image镜像

6.5.2 创建目录结构

6.5.3 安装ES集群

6.6 使用kibana访问集群

7 数据类型和映射

7.1 映射的创建

7.2 动态映射Dynamic Mapping

7.3 显示的设置mapping

7.4 ES中常见数据类型

8 底层知识：正排索引和倒排索引底层原理

8.1 什么是正排索引

8.2 什么是倒排索引

8.3 ES如何做到快速索引

8.3.1 Posting List

8.3.2 Term Dictionary

8.3.3 Term Index

8.3.4 Trie树 (前缀树, 字典树)

8.3.5 为什么ES检索比Mysql快

附录：DSL常用

一、前言

二、DSL常用查看索引语句

2.1 查看所有索引的信息 (indices?v)

2.2 查看特定索引的信息 (_stats)

2.3 查看索引的映射 (_mapping)

2.4 查看索引的设置 (_settings)

2.5 查看索引的别名 (aliases)

2.6 查看索引的文档数量 (_count)

2.7 查看索引的存储大小 (_stats/store)

2.8 查看索引的字段名 (_search)

2.9 查看索引的分片信息 (_cat/shards)

2.10 查看索引的健康状态 (_cluster/health)

2.11 查看索引的索引速率 (_cat/thread_pool)

2.12 查看索引的搜索速率 (_cat/thread_pool/search)

2.13 查看索引的索引速率 (_cat/thread_pool/index)

2.14 查看索引的删除速率 (_cat/thread_pool/delete)

2.15 查看索引的刷新速率 (_cat/thread_pool/refresh)

2.16 查看索引的合并速率 (_cat/thread_pool/merge)

2.17 查看索引的查询速率 (_cat/thread_pool/get)

2.18 查看索引的更新速率 (_cat/thread_pool/update)

三、DSL常用查询语句

- 3.1 查询所有 (match_all)
 - 3.2 关键词全文查询 (match)
 - 3.3 精确匹配查询 (term)
 - 3.4 多值精确匹配查询 (terms)
 - 3.5 通配符查询 (wildcard)
 - 3.6 多值通配符查询 (多wildcard)
 - 3.7 字段前缀查询 (prefix)
 - 3.8 多值字段前缀查询 (多prefix)
 - 3.9 模糊匹配查询 (fuzzy)
 - 3.10 正则表达式查询 (regexp)
 - 3.11 根据当天时间查询 (range + filter + now)
 - 3.12 根据自定义时间段查询 (range + filter + datetime)
 - 3.13 满足条件查询 (must)
 - 3.14 排除条件查询 (must_not)
 - 3.15 包含key查询 (must + exists)
 - 3.16 不包含key查询 (must_not + exists)
 - 3.17 排序查询 (sort)
 - 3.17.1 降序
 - 3.17.2 升序
 - 3.18 指定返回字段查询 (_source)
 - 3.19 指定条数查询 (size)
 - 3.20 分页查询 (from)
 - 3.21 关联多个index查询
 - 3.22 布尔查询 (bool)
 - 3.22.1 bool+must查询
 - 3.22.2 bool+must_not查询
 - 3.22.3 bool+should查询
 - 3.22.4 bool+filter查询
 - 3.22.5 bool多种组合 (bool+must+must_not+should+filter)
 - 3.23 match、term、terms、should、must_not、bool、filter区别
- 四、DSL常用统计语句
- 4.1 aggs terms
 - 4.2 aggs cardinality
 - 4.3 aggs value_count
 - 4.4 aggs terms+script
 - 4.5 aggs多重组合
 - 4.6 aggs语句参数解析
- 五、所遇到问题的解决方案
- 5.1 解决统计显示所有数量而不是10000问题
 - 5.2 解决must与should同时使用must不生效问题

说在最后：有问题找老架构取经

百亿级数据存储架构，只有分库分表吗？

很多的小伙伴来咨询尼恩，百亿级数据存储怎么架构，说他们的面试中，都遇到的。

他们回到回答了分库分表，比如，当一个表（比如t_order）达到500万条或2GB时，需要考虑水平分表。

然后面试官，不满意。

很多的小伙伴来咨询尼恩，为什么？

这里，尼恩用20年的技术功力，给大家做一个彻底性、系统化梳理，帮助大家吊打面试。

从0到1, 百亿级数据存储架构, 怎么设计?

咱们的生产需求上, 百亿级数据存储架构, 一般来说, 需要具备以下四个能力:

- 高并发的在线ACID事务 (分库分表)
- 高并发的在线搜索 (倒排表副本)
- 海量数据的离线处理 (高可用+全量副本)
- 冗余表双写能力 (不同业务维度的副本)

其中, 上面的冗余表双写能力, 也就是 高并发的 多业务维度 在线ACID 事务处理能力

比如在海量订单场景,

- 用户维度的在线ACID 事务订单处理能力, 需要进行用户维度的分库分表。
- 商家维度的在线ACID 事务订单处理能力, 需要进行商家维度的分库分表。

如果不需要 不同业务维度的 在线ACID 事务订单处理能力, 那么冗余表双写能力 这个是可选项。

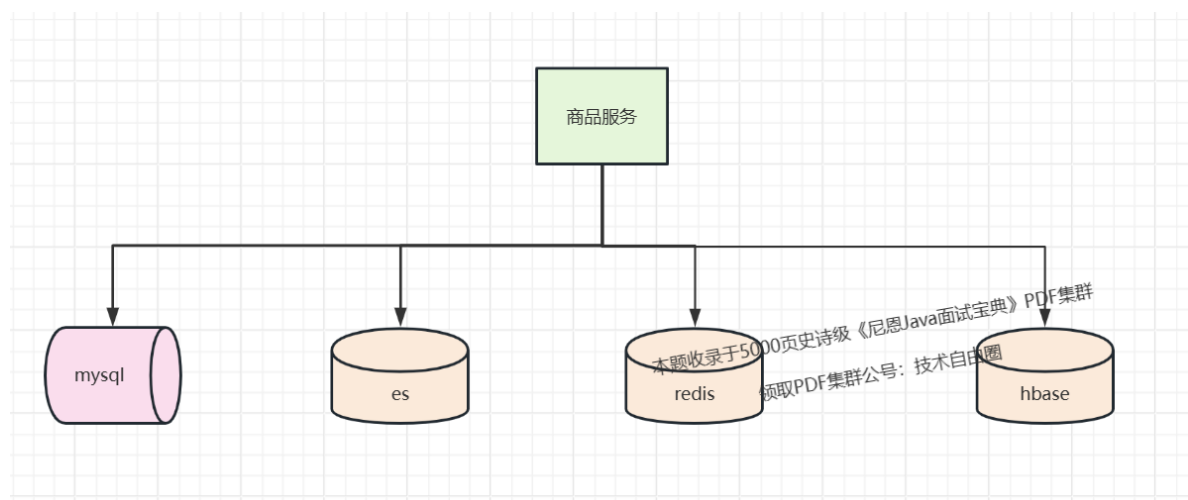
这是引入这么多的副本, 有好处, 也有坏处:

- 好处是满足各种各样的处理要求
- 坏处是我们要维护多个副本之间的数据一致。

百亿级数据存储架构, 怎么维护多副本之间的数据一致?

便于商品的聚合搜索, 高速搜索, 采用两大优化方案:

- 把商品数据冗余存储在Elasticsearch中, 实现高速搜索
- 把商品数据冗余存储在redis 中, 实现高速缓存



很多的时候, 要求保持很高的数据一致性。

比如:

- 要求 mysql 与 es 做到秒级别的数据同步。
- 要求 mysql 与 redis 做到秒级别的数据同步。

- 要求 mysql 与 hbase 做到秒级别的数据同步。

接下来，以 mysql 与 es 的数据一致，作为业务场景进行分析，其他的场景比如mysql 与 redis 的数据一致性方案，都是差不多的。

只要大家能把下面的 5大数据一致性方案，滔滔不绝的说出来，面试官一定会爱到“不能自己、口水直流”。

从0到1, 百亿级数据存储架构，怎么设计？

从0到1, 百亿级数据存储架构，40岁老架构尼恩团队，计划用一系列的文章帮大家实现：

- 高并发搜索篇：从0到1，从入门到 Elasticsearch 工业级使用
- 100亿级任务调度篇：从0到1，从入门到 XXLJOB 工业级使用
- 100亿级海量存储篇：从0到1，从入门到 HABSE 工业级使用
- 100亿级离线计算篇：从0到1，从入门到 Flink 工业级使用

已经发布的文章包括：

[100亿级任务调度篇：从0到1, 从入门到 XXLJOB 工业级使用](#)

ElasticSearch 从入门到工业级使用

1.1 什么是全文检索

将非结构化数据中的一部分信息提取出来，重新组织，使其变得有一定结构，然后对此有一定结构的数据进行搜索，从而达到搜索相对较快的目的。这部分从非结构化数据中提取出的然后重新组织的信息，我们称之为索引。

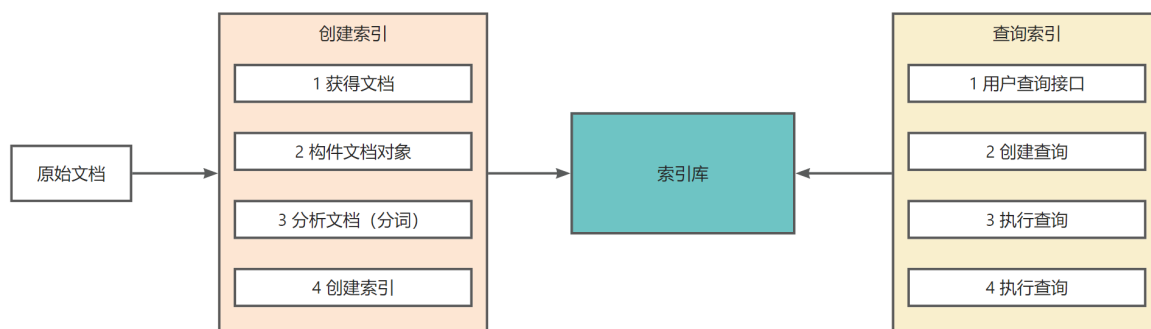
例如：字典。字典的拼音表和部首检字表就相当于字典的索引，对每一个字的解释是非结构化的，如果字典没有音节表和部首检字表，在茫茫辞海中找一个字只能顺序扫描。

然而字的某些信息可以提取出来进行结构化处理，比如读音，就比较结构化，分声母和韵母，分别只有几种可以一一列举，于是将读音拿出来按一定的顺序排列，每一项读音都指向此字的详细解释的页数。

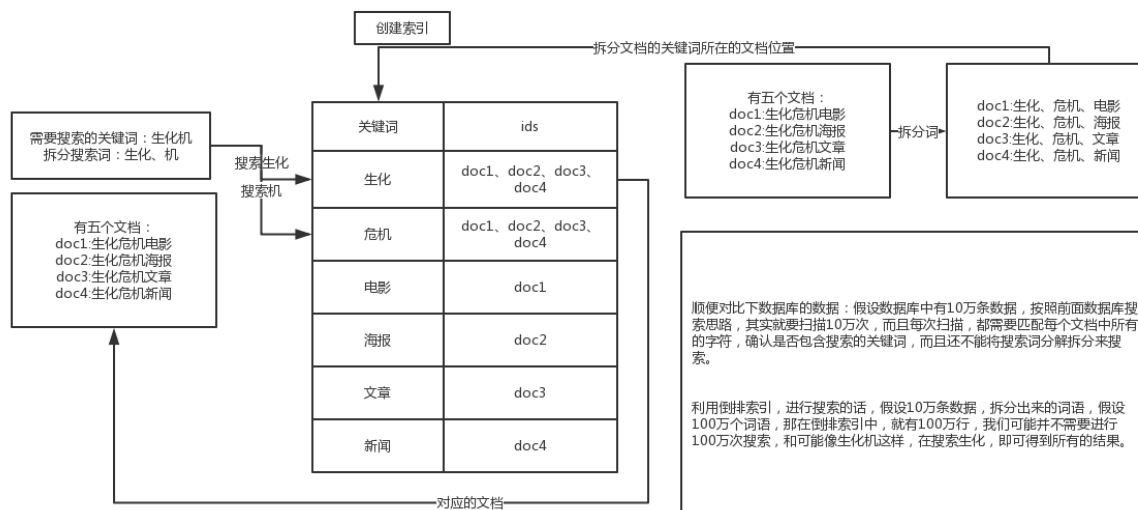
我们搜索时按结构化的拼音搜到读音，然后按其指向的页数，便可找到我们的非结构化数据——也即对字的解释。

这种先建立索引，再对索引进行搜索的过程就叫全文检索(Full-text Search)。

虽然创建索引的过程也是非常耗时的，但是索引一旦创建就可以多次使用，全文检索主要处理的是查询，所以耗时间创建索引是值得的。



比如使用全文检索，所搜索“生化机”



(有可能是手抖打错了，本来是生化危机)，但是期望需要出来右侧的 4条 记录

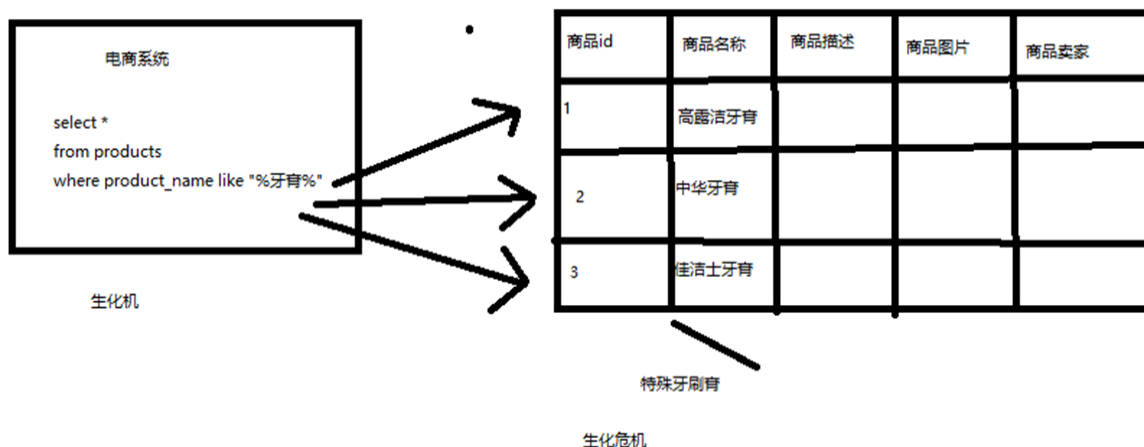
有 4条 数据将每条数据进行词条拆分。

如“生化危机电影”拆成：生化、危机、电影 关键词（拆分结果与策略算法有关）每个关键词将对应包含此关键词的数据 ID搜索的时候，直接匹配这些关键词，就能拿到包含关键词的数据这个过程就叫做全文检索。

而词条拆分和词条对应的 ID 这个就是倒排索引的的基本原理

对比数据库的缺陷

mysql如果没有索引的情况下，共有100万条,按照之前的思路,其实就要扫描100万次，而且每次扫描,都需要匹配那个文本所有的字符，确认是否包含搜索的关键词，而且还不能将搜索词拆解开来进行搜索



利用倒排索引

进行搜索的话，假设100万条数据,拆分出来的词语,假设有1000万个词语，那么在倒排索引中,就有1000

万行,我们可能并不需要搜索1000万次。很可能说,在搜索到第一次的时候,我们就可以找到这个搜索词对应的数据。也可能是第100次,或者第1000次

全文检索使用场景

- 维基百科, 类似百度百科, 牙膏, 牙膏的维基百科, 全文检索, 高亮, 搜索推荐
- The Guardian (国外新闻网站), 类似搜狐新闻, 用户行为日志(点击, 浏览, 收藏, 评论) + 社交网络数据(对某某新闻的相关看法), 数据分析, 给到每篇新闻文章的作者, 让他知道他的文章的公众反馈(好, 坏, 热门, 垃圾, 鄙视, 崇拜)
- Stack Overflow (国外的程序异常讨论论坛), IT问题, 程序的报错, 提交上去, 有人会跟你讨论和回答, 全文检索, 搜索相关问题和答案, 程序报错了, 就会将报错信息粘贴到里面去, 搜索有没有对应的答案
- GitHub (开源代码管理), 搜索上千亿行代码 (5) 电商网站, 检索商品
- 日志数据分析, logstash采集日志, ES进行复杂的数据分析 (ELK技术, elasticsearch+logstash+kibana)
- 商品价格监控网站, 用户设定某商品的价格阈值, 当低于该阈值的时候, 发送通知消息给用户, 比如说订阅牙膏的监控, 如果高露洁牙膏的家庭套装低于50块钱, 就通知我, 我就去买
- BI系统, 商业智能, Business Intelligence。比如说有个大型商场集团, BI, 分析一下某某区域最近3年的用户消费金额的趋势以及用户群体的组成构成, 产出相关的数张报表, **区, 最近3年, 每年消费金额呈现100%的增长, 而且用户群体85%是高级白领, 开一个新商场。ES执行数据分析和挖掘, Kibana进行数据可视化

1.2 ES简介

Elasticsearch, 简称为es, es是一个开源的高扩展的分布式全文检索引擎, 它可以近乎实时的存储、检索数据; 本身扩展性很好, 可以扩展到上百台服务器, 处理PB级别的数据。

es也使用Java开发并使用Lucene作为其核心来实现所有索引和搜索的功能, 但是它的目的是通过简单的RESTful API来隐藏Lucene的复杂性, 从而让全文搜索变得简单。

Elasticsearch是面向文档(document oriented)的, 这意味着它可以存储整个对象或文档(document)。然而它不仅仅是存储, 还会索引(index)每个文档的内容使之可以被搜索。

在Elasticsearch中, 你可以对文档(而非成行成列的数据)进行索引、搜索、排序、过滤。

Elasticsearch比传统关系型数据库如下:

```
Relational DB -> Databases -> Tables -> Rows -> Columns
Elasticsearch -> Indices -> Types -> Documents -> Fields
```

Elasticsearch提供多种语言支持, 其中Java的客户端为 Java REST Client。

而它又分成两种: 高级和低级的。高级包含更多的功能, 如果把高级比作MyBatis的话, 那么低级就相当于JDBC, 是基于Netty和Server通讯相关。

高级的 Client类似Mybatis是对于Low Level的封装。



1.3 ES基本概念

1. 索引库

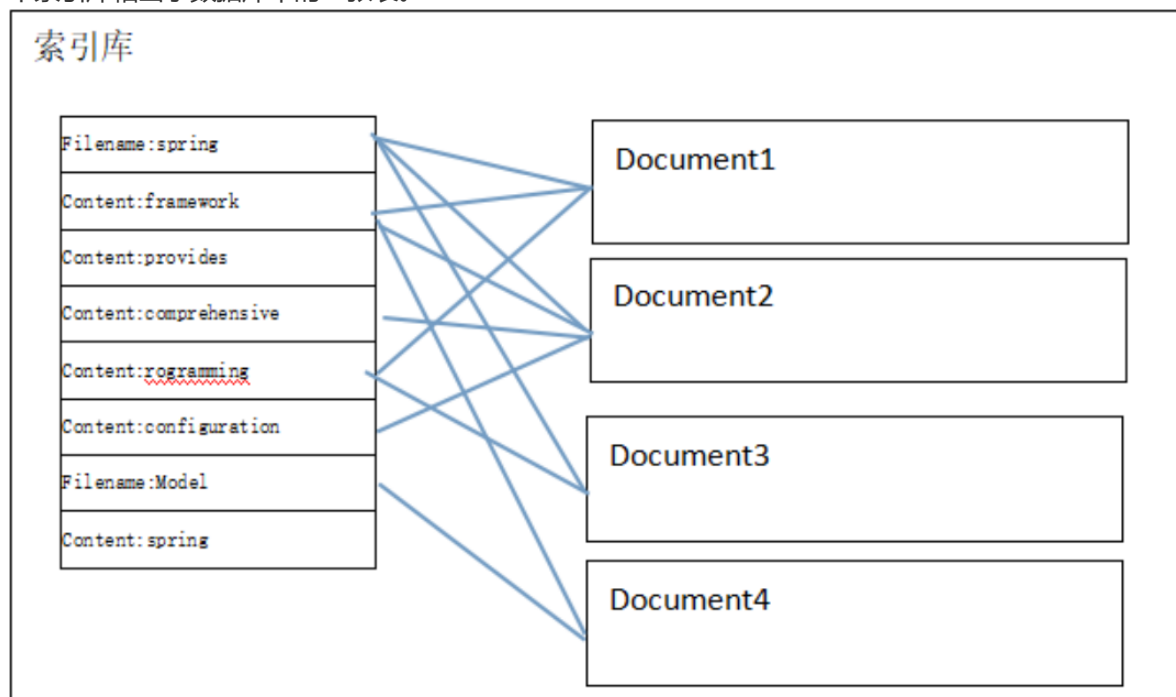
ElasticSearch将它的数据存储在一个或多个索引（index）中。

用SQL领域的术语来类比，索引就像数据库，可以向索引写入文档或者从索引中读取文档，并通过ElasticSearch内部使用Lucene将数据写入索引或从索引中检索数据。

Elastic Search使用倒排索引（Inverted Index）来做快速的全文搜索，这点与数据库不同，一般数据库的索引，用B+Tree来实现。

Relational DB	Databases	Tables	表结构	Rows	Columns
ElasticSearch	Indices	Types	映射mapping	Documents	Fields 字段

索引库就是存储索引的保存在磁盘上的一系列的文件。里面存储了建立好的索引信息以及文档对象。一个索引库相当于数据库中的一张表。



2. document对象

获取原始内容的目的是为了索引，在索引前需要将原始内容创建成文档（Document），文档中包括一个一个的域（Field），域中存储内容。

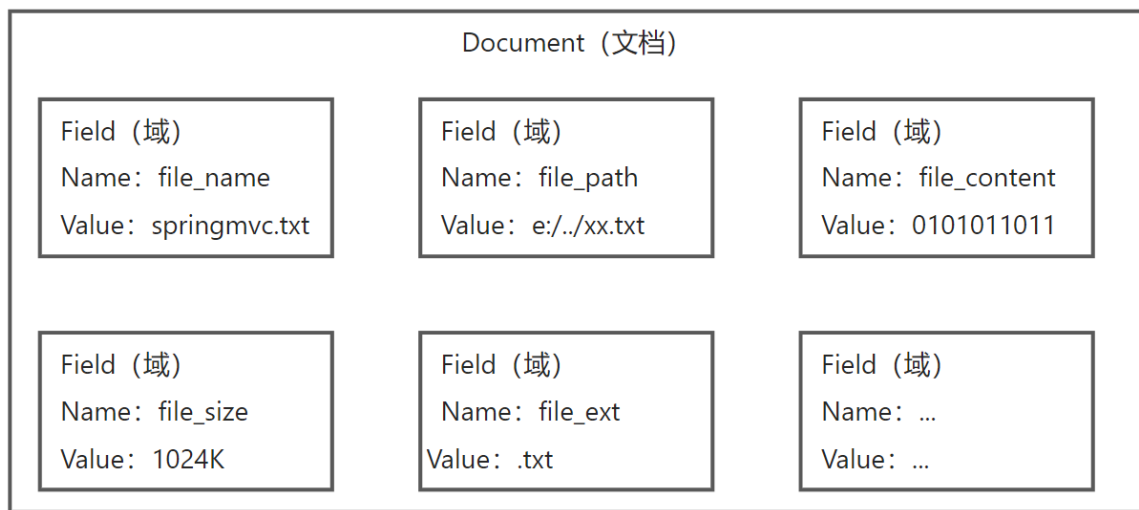
每个文档都有一个唯一的编号，就是文档id。

document对象相当于表中的一条记录。

文档（document）是ElasticSearch中的主要实体。

对所有使用ElasticSearch的案例来说，他们最终都可以归结为对文档的搜索。

文档由字段构成。



3. field对象

如果我们把document看做是数据库中一条记录的话，field相当于是记录中的字段。field是索引库中存储数据的最小单位。field的数据类型大致可以分为数值类型和文本类型，一般需要查询的字段都是文本类型的，field的还有如下属性：

- 是否分词：是否对域的内容进行分词处理。前提是我们要对域的内容进行查询。
- 是否索引：将Field分析后的词或整个Field值进行索引，只有索引方可搜索到。比如：商品名称、商品简介分析后进行索引，订单号、身份证号不用分词但也要索引，这些将来都要作为查询条件。
- 是否存储：将Field值存储在文档中，存储在文档中的Field才可以从Document中获取。比如：商品名称、订单号，凡是将来要从Document中获取的Field都要存储。

4. term对象

从文档对象中拆分出来的每个单词叫做一个Term，不同的域中拆分出来的相同的单词是不同的term。

term中包含两部分一部分是文档的域名，另一部分是单词的内容。

term是创建索引的关键词对象。

8. 类型 (type)

每个文档都有与之对应的类型 (type) 定义。

这允许用户在一个索引中存储多种文档类型，并为不同文档提供类型提供不同的映射。

type的版本迭代

- 5.x及以前版本一个index有一个或者多个type
- 6.X版本一个index只有一个index
- 7.X版本移除了type，type相关的所有内容全部变成Deprecated，为了兼容升级和过渡，所有的7.X版本es数据写入后type字段都默认被置为_doc
- 8.X版本完全废弃type

9. 映射 (mapping)

mapping是处理数据的方式和规则方面做一些限制，如某个字段的数据类型、默认值、分析器、是否被索引等等，这些都是映射里面可以设置的，其它就是处理es里面数据的一些使用规则设置也叫做映射，按着最优规则处理数据对性能提高很大，因此才需要建立映射，并且需要思考如何建立映射才能对性能更好。

10. 分片 (shards)

代表索引分片，es可以把一个完整的索引分成多个分片，这样的好处是可以把一个大的索引拆分成多个，分布到不同的节点上。构成分布式搜索。分片的数量只能在索引创建前指定，并且索引创建后不能更改。

5.X默认不能通过配置文件定义分片

ES默认5:1 5个主分片，每个分片，1个副本分片

11. 副本 (replicas)

代表索引副本，es可以设置多个索引的副本，副本的作用：

- 提高系统的容错性，当某个节点某个分片损坏或丢失时可以从副本中恢复。
- 是提高es的查询效率，es会自动对搜索请求进行负载均衡。

12. 集群 (cluster)

代表一个集群，集群中有多个节点 (node)，其中有一个为主节点，这个主节点是可以通过选举产生的，主从节点是对于集群内部来说的。es的一个概念就是去中心化，字面上理解就是无中心节点，这是对于集群外部来说的，因为从外部来看es集群，在逻辑上是个整体，你与任何一个节点的通信和与整个es集群通信是等价的。

2 安装和DSL的使用

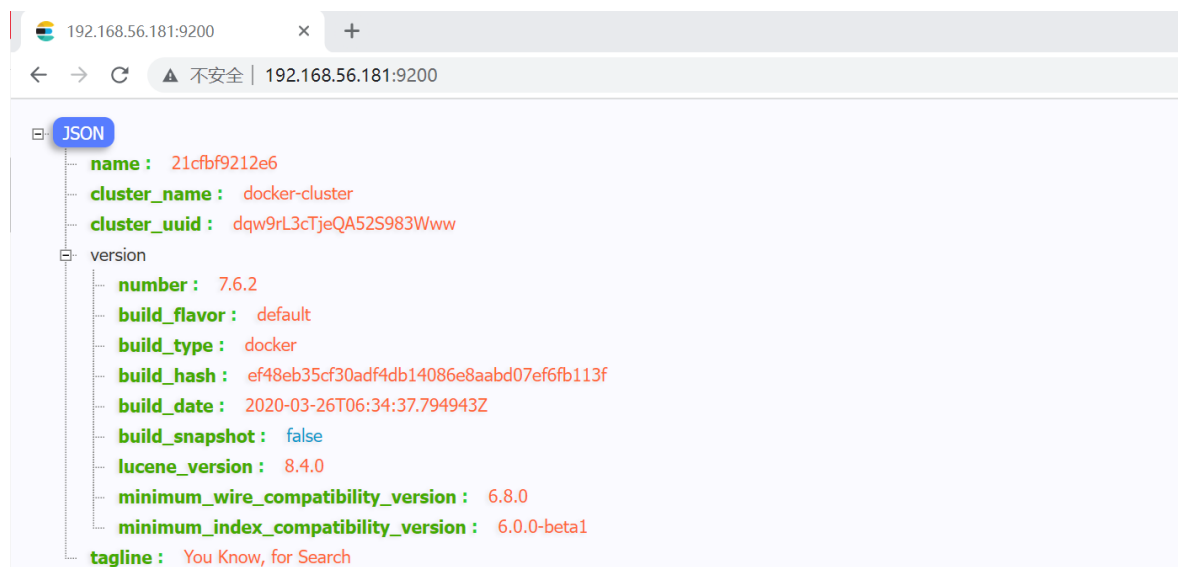
2.1 安装ES

使用docker安装单点Elasticsearch，步骤如下：

```
docker network create elastic
docker pull docker.elastic.co/elasticsearch/elasticsearch:7.15.2
docker run -di --name es --net elastic -p 192.168.56.181:9200:9200 -p
192.168.56.181:9300:9300 -e "discovery.type=single-node"
docker.elastic.co/elasticsearch/elasticsearch:7.15.2
```

9200端口(Web管理平台端口) 9300(服务默认端口)

浏览器输入地址访问：<http://192.168.56.181:9200/>



3. 系统参数配置

es发现重启启动失败了，这时什么原因呢？

这与我们刚才修改的配置有关，因为elasticsearch在启动的时候会进行一些检查，比如最多打开的文件的个数以及虚拟内存区域数量等等，如果你放开了此配置，意味着需要打开更多的文件以及虚拟内存，所以我们还需要系统调优

修改vi /etc/security/limits.conf，追加内容 (nofile是单个进程允许打开的最大文件个数 soft nofile 是软限制 hard nofile是硬限制)

```
* soft nofile 65536
* hard nofile 65536
```

修改vi /etc/sysctl.conf，追加内容 (限制一个进程可以拥有的VMA(虚拟内存区域)的数量)

```
vm.max_map_count=655360
```

执行下面命令 修改内核参数马上生效

```
sysctl -p
```

重新启动虚拟机，再次启动容器，发现已经可以启动并远程访问

```
reboot
```

2.2 安装Kibana

Kibana是一个针对Elasticsearch的开源分析及可视化平台，用来搜索、查看交互存储在Elasticsearch索引中的数据。使用Kibana，可以通过各种图表进行高级数据分析及展示。

Kibana让海量数据更容易理解。它操作简单，基于浏览器的用户界面可以快速创建仪表盘 (dashboard) 实时显示Elasticsearch查询动态。

设置Kibana非常简单。

无需编码或者额外的基础架构，几分钟内就可以完成Kibana安装并启动Elasticsearch索引监测。

Query DSL是一个Java开源框架用于构建类型安全的SQL查询语句。

采用API代替传统的拼接字符串来构造查询语句。

目前QueryDSL支持的平台包括JPA, JDO, SQL, Java Collections, RDF, Lucene, Hibernate Search。elasticsearch提供了一整套基于JSON的DSL语言来定义查询。

```
docker pull docker.elastic.co/kibana/kibana:7.15.2
docker run -di --name kb --net elastic -p 192.168.56.181:5601:5601 -e
"ELASTICSEARCH_HOSTS=http://192.168.56.181:9200"
docker.elastic.co/kibana/kibana:7.15.2
```

1. 拉取kibana镜像

```
docker pull docker.elastic.co/kibana/kibana:7.15.2
```

2. 安装kibana容器

```
docker run -di --name kb --net elastic -p 192.168.56.181:5601:5601 -e
"ELASTICSEARCH_HOSTS=http://192.168.56.181:9200"
docker.elastic.co/kibana/kibana:7.15.2
```

3. 修改配置文件

```
docker exec -it kb /bin/bash
vi config/kibana.yml
```

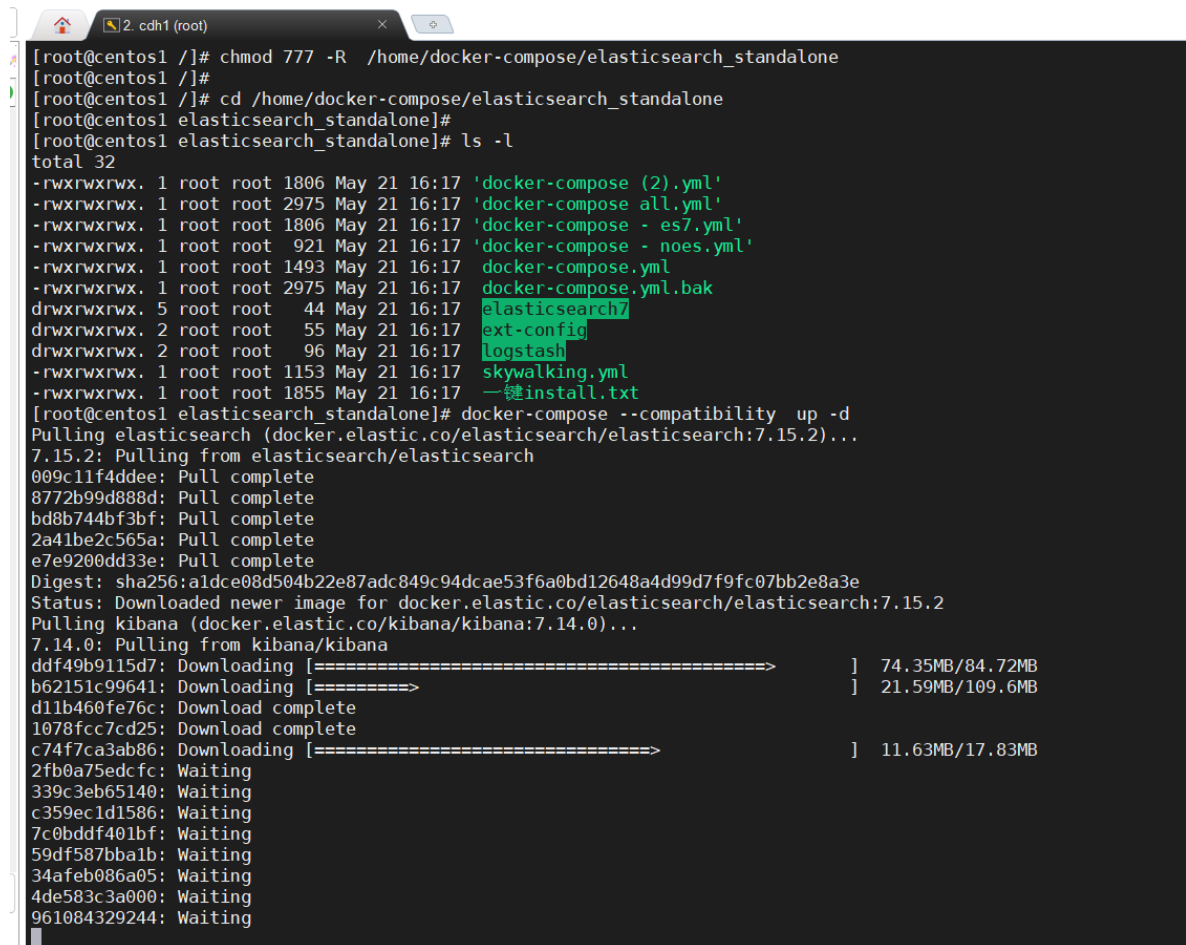
好像不改也可以，因为上面docker启动有了ES地址

```
#修改elasticsearch.hosts: [ "http://elasticsearch:9200" ], 如下:
elasticsearch.hosts: [ "http://192.168.56.181:9200" ]
```

2.3通过脚本一键启动ES

通过尼恩提供的脚本，和配置文件，可以一键启动ES

这个非常的容易，非常的轻量级。具体请参见视频。



```
[root@centos1 ~]# chmod 777 -R /home/docker-compose/elasticsearch_standalone
[root@centos1 ~]#
[root@centos1 ~]# cd /home/docker-compose/elasticsearch_standalone
[root@centos1 elasticsearch_standalone]#
[root@centos1 elasticsearch_standalone]# ls -l
total 32
-rwxrwxrwx. 1 root root 1806 May 21 16:17 'docker-compose (2).yml'
-rwxrwxrwx. 1 root root 2975 May 21 16:17 'docker-compose all.yml'
-rwxrwxrwx. 1 root root 1806 May 21 16:17 'docker-compose - es7.yml'
-rwxrwxrwx. 1 root root 921 May 21 16:17 'docker-compose - noes.yml'
-rwxrwxrwx. 1 root root 1493 May 21 16:17 docker-compose.yml
-rwxrwxrwx. 1 root root 2975 May 21 16:17 docker-compose.yml.bak
drwxrwxrwx. 5 root root 44 May 21 16:17 elasticsearch7
drwxrwxrwx. 2 root root 55 May 21 16:17 ext-config
drwxrwxrwx. 2 root root 96 May 21 16:17 logstash
-rwxrwxrwx. 1 root root 1153 May 21 16:17 skywalking.yml
-rwxrwxrwx. 1 root root 1855 May 21 16:17 一键install.txt
[root@centos1 elasticsearch_standalone]# docker-compose --compatibility up -d
Pulling elasticsearch (docker.elastic.co/elasticsearch/elasticsearch:7.15.2)...
7.15.2: Pulling from elasticsearch/elasticsearch
009c11f4ddee: Pull complete
8772b99d888d: Pull complete
bd8b744bf3bf: Pull complete
2a41be2c565a: Pull complete
e7e9200dd33e: Pull complete
Digest: sha256:a1dce08d504b22e87adc849c94dcae53f6a0bd12648a4d99d7f9fc07bb2e8a3e
Status: Downloaded newer image for docker.elastic.co/elasticsearch/elasticsearch:7.15.2
Pulling kibana (docker.elastic.co/kibana/kibana:7.14.0)...
7.14.0: Pulling from kibana/kibana
ddf49b9115d7: Downloading [=====>] 74.35MB/84.72MB
b62151c99641: Downloading [=====>] 21.59MB/109.6MB
d11b460fe76c: Download complete
1078fcc7cd25: Download complete
c74f7ca3ab86: Downloading [=====>] 11.63MB/17.83MB
2fb0a75edcfc: Waiting
339c3eb65140: Waiting
c359ec1d1586: Waiting
7c0bddf401bf: Waiting
59df587bba1b: Waiting
34afeb086a05: Waiting
4de583c3a000: Waiting
961084329244: Waiting
```

启动完之后的效果

```
查看 分割 多执行 隧道 软件包 设置 帮助 x 服务器
cdh1 (root)
elasticsearch | [2024-05-21T16:29:03.082][INFO] [o.e.c.m.MetadataMappingService] [node-1] [.kibana-7.14.0-001/PW-uzY10SF05KPiyydnRfg] update_mapping [d
elasticsearch | [2024-05-21T16:29:03.200][INFO] [o.e.x.i.a.TransportPutLifecycleAction] [node-1] adding index lifecycle policy [kibana-event-log-policy]
elasticsearch | [2024-05-21T16:29:03.270][INFO] [o.e.c.m.MetadataIndexTemplateService] [node-1] adding template [.kibana-event-log-7.14.0-template] for
ex patterns [.kibana-event-log-7.14.0-*]
elasticsearch | [2024-05-21T16:29:03.355][INFO] [o.e.c.m.MetadataCreateIndexService] [node-1] [.kibana-event-log-7.14.0-000001] creating index, cause [a
, templates [.kibana-event-log-7.14.0-template], shards [1]/[1]]
elasticsearch | [2024-05-21T16:29:03.358][INFO] [o.e.c.r.a.AllocationService] [node-1] updating number_of_replicas to [0] for indices [.kibana-event-log
14.0-000001]
elasticsearch | [2024-05-21T16:29:03.489][INFO] [o.e.x.i.IndexLifecycleTransition] [node-1] moving index [.kibana-event-log-7.14.0-000001] from [null] t
{"phase":"new", "action":"complete", "name":"complete"} in policy [kibana-event-log-policy]
elasticsearch | [2024-05-21T16:29:03.530][INFO] [o.e.c.r.a.AllocationService] [node-1] Cluster health status changed from [YELLOW] to [GREEN] (reason: [
rds started [.kibana-event-log-7.14.0-000001][0]]).
elasticsearch | [2024-05-21T16:29:03.577][INFO] [o.e.x.i.IndexLifecycleTransition] [node-1] moving index [.kibana-event-log-7.14.0-000001] from [{"phase
new", "action":"complete", "name":"complete"}] to [{"phase":"hot", "action":"unfollow", "name":"branch-check-unfollow-prerequisites"}] in policy [kibana-event-
-policy]
kibana | {"type":"log", "@timestamp":"2024-05-21T08:29:03+00:00", "tags":["info", "plugins", "reporting"], "pid":1213, "message":"Browser executable: /
/share/kibana/x-pack/plugins/reporting/chromium/headless_shell-linux_x64/headless_shell"}
kibana | {"type":"log", "@timestamp":"2024-05-21T08:29:03+00:00", "tags":["warning", "plugins", "reporting"], "pid":1213, "message":"Enabling the Chrom
sandbox provides an additional layer of protection."}
elasticsearch | [2024-05-21T16:29:03.651][INFO] [o.e.x.i.IndexLifecycleTransition] [node-1] moving index [.kibana-event-log-7.14.0-000001] from [{"phase
hot", "action":"unfollow", "name":"branch-check-unfollow-prerequisites"}] to [{"phase":"hot", "action":"rollover", "name":"check-rollover-ready"}] in policy [k
na-event-log-policy]
kibana | {"type":"log", "@timestamp":"2024-05-21T08:29:03+00:00", "tags":["info", "status"], "pid":1213, "message":"Kibana is now degraded (was unavail
le)"}
kibana | {"type":"log", "@timestamp":"2024-05-21T08:29:04+00:00", "tags":["info", "plugins", "securitySolution"], "pid":1213, "message":"Dependent plug
setup complete - Starting ManifestTask"}
elasticsearch | [2024-05-21T16:29:08.468][INFO] [o.e.c.m.MetadataCreateIndexService] [node-1] [.ds-ilm-history-5-2024.05.21-000001] creating index, caus
initialize data stream], templates [ilm-history], shards [1]/[0]]
elasticsearch | [2024-05-21T16:29:08.484][INFO] [o.e.c.m.MetadataCreateDataStreamService] [node-1] adding data stream [ilm-history-5] with write index [.
ilm-history-5-2024.05.21-000001], backing indices [], and aliases []
elasticsearch | [2024-05-21T16:29:08.603][INFO] [o.e.x.i.IndexLifecycleTransition] [node-1] moving index [.ds-ilm-history-5-2024.05.21-000001] from [null]
to [{"phase":"new", "action":"complete", "name":"complete"}] in policy [ilm-history-ilm-policy]
elasticsearch | [2024-05-21T16:29:08.655][INFO] [o.e.x.i.IndexLifecycleTransition] [node-1] moving index [.ds-ilm-history-5-2024.05.21-000001] from [{"p
e":"new", "action":"complete", "name":"complete"}] to [{"phase":"hot", "action":"unfollow", "name":"branch-check-unfollow-prerequisites"}] in policy [ilm-histo
ilm-policy]
elasticsearch | [2024-05-21T16:29:08.702][INFO] [o.e.c.r.a.AllocationService] [node-1] Cluster health status changed from [YELLOW] to [GREEN] (reason: [
rds started [.ds-ilm-history-5-2024.05.21-000001][0]]).
elasticsearch | [2024-05-21T16:29:08.744][INFO] [o.e.x.i.IndexLifecycleTransition] [node-1] moving index [.ds-ilm-history-5-2024.05.21-000001] from [{"p
e":"hot", "action":"unfollow", "name":"branch-check-unfollow-prerequisites"}] to [{"phase":"hot", "action":"rollover", "name":"check-rollover-ready"}] in polic
ilm-history-ilm-policy]
kibana | {"type":"log", "@timestamp":"2024-05-21T08:29:09+00:00", "tags":["info", "status"], "pid":1213, "message":"Kibana is now available (was degra
)"}

```

接下来，可以访问es

<http://cdh1:9200>

```
← ↻ ⚠ 不安全 | cdh1:9200
1 {
2   "name": "node-1",
3   "cluster_name": "elasticsearch",
4   "cluster_uuid": "GIIdmWUzSQ1-0ch9Ysux8Ww",
5   "version": {
6     "number": "7.15.2",
7     "build_flavor": "default",
8     "build_type": "docker",
9     "build_hash": "93d5a7f6192e8a1a12e154a2b81bf6fa7309da0c",
10    "build_date": "2021-11-04T14:04:42.515624022Z",
11    "build_snapshot": false,
12    "lucene_version": "8.9.0",
13    "minimum_wire_compatibility_version": "6.8.0",
14    "minimum_index_compatibility_version": "6.0.0-beta1"
15  },
16   "tagline": "You Know, for Search"
17 }
```

接下来，可以访问 Kibana

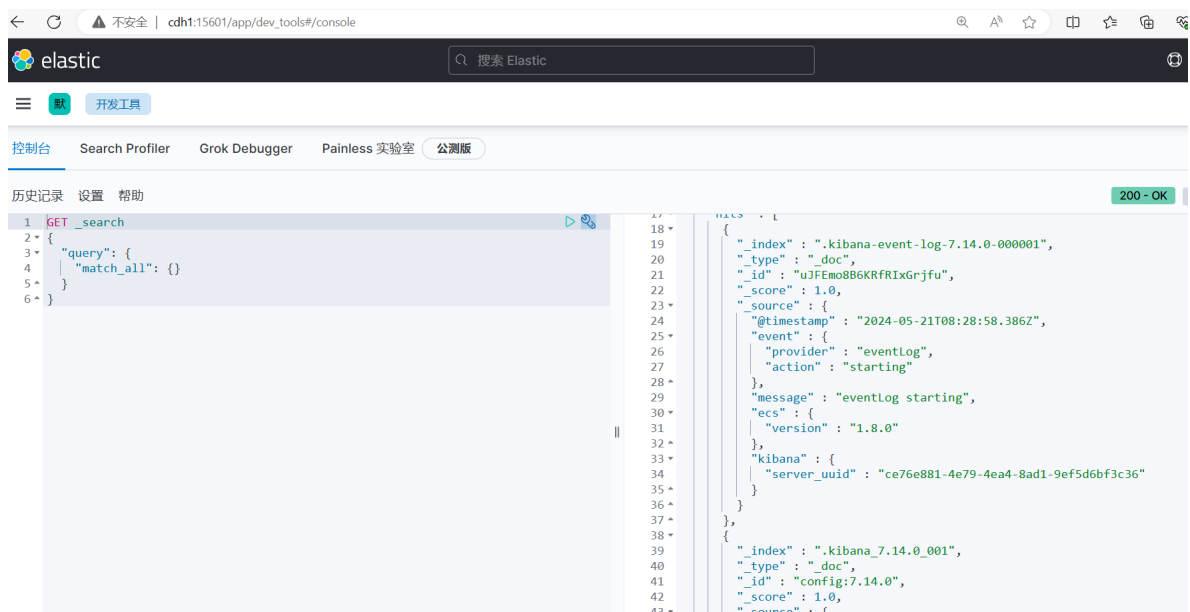
默认的地址 <http://cdh1:5601>

尼恩的一键环境地址 <http://cdh1:5601>



2.4使用 DSL 操作ES

在 Kibana的开发工具界面，可以执行 DSL 去进行ES的查询。



es开发，常常需要用到DSL语法去定义好 es的查询语句。

就像 mysql开发，需要提前定义好 sql语句，并进行sql 的执行和测试一样。

2.4.1 DSL 定义基本介绍

DSL (Domain Specific Language) , 一种特定领域的查询语言, 用于构建复杂的查询和聚合操作。

在Elasticsearch中, 可用DSL语法来定义查询和过滤条件, 以及执行聚合操作。

DSL语法 具有JSON格式, 因此它非常易于阅读和编写。

2.4.2 DSL 定义语法说明

(1) 关键字(Keywords)

- DSL通常会定义一组关键字, 这些关键字具有特殊含义, 并在DSL中起到关键作用。关键字通常不能用作标识符或变量名。
- 示例: 在一个简单的数学表达式DSL中, 可能会定义关键字如"add"、"subtract"等来表示加法和减法操作。

(2) 标识符(Identifiers)

- 标识符是用来表示变量名、函数名或其他用户定义的名称。它们需要遵循特定的命名规则, 如大小写敏感、不包含特殊字符等。
- 示例: 在一个配置文件DSL中, 可以使用标识符来表示不同的配置项, 如"username"、"password"等。

(3) 表达式(Expressions)

- 表达式是DSL中最基本的构建块, 用于计算或产生某个值。表达式可以包括变量、常量、运算符和函数调用。
- 示例: 在一个数学表达式DSL中, 可以将"2 + 3"作为一个表达式, 计算结果为5。

(4) 运算符(Operators)

- 运算符用于执行各种操作, 例如算术运算、逻辑运算、比较运算等。DSL中的运算符根据所涉及的领域和需求而定。
- 示例: 在一个布尔表达式DSL中, 可以定义逻辑运算符如"and"、"or"用于连接多个条件。

(5) 函数调用(Function Calls)

- DSL可以支持函数调用, 允许用户使用预定义或自定义的函数来完成特定的任务。函数调用通常由函数名称和传递给函数的参数组成。
- 示例: 在一个日期处理DSL中, 可以定义函数"formatDate(date, format)", 其中"date"是日期值, "format"是日期格式字符串。

(6) 控制流(Control Flow)

- 控制流语句用于控制程序的执行流程, 例如条件语句(if-else)和循环语句(while、for)等。DSL可以支持特定的控制流语句来满足领域特定需求。
- 示例: 在一个工作流程DSL中, 可以使用条件语句来判断某个条件是否满足并执行相应的操作。

(7) 注释(Comments)

- 注释用于向DSL代码添加说明性文本, 以便开发人员理解和维护代码。注释通常不会被编译或执行, 仅用于阅读目的。
- 示例: 在DSL中, 可以使用双斜杠(//)或特定的注释标记来添加注释, 如: "// 这是一个示例注释"。

2.4.3 DSL常见语法

(1) 查询 (Query)

用于指定搜索条件，并返回与之匹配的文档结果集。以下是一些常见的查询类型：

- Match查询：用于根据指定字段的内容进行全文匹配。
- Term查询：用于精确匹配某个字段的值，不进行分词。
- Range查询：用于匹配某个范围内的值。
- Bool查询：用于组合多个查询条件。

(2) 过滤 (Filter)

用于筛选满足指定条件的文档结果集。过滤器通常用于在查询结果上应用额外的限制。以下是一些常见的过滤器类型：

- Term过滤器：用于精确匹配某个字段的值。
- Range过滤器：用于筛选符合某个范围条件的值。
- Bool过滤器：用于组合多个过滤条件。

(3) 聚合 (Aggregation)

用于对文档结果集进行分组和统计操作，从而生成各种汇总数据。聚合可以计算平均值、总和、最大值、最小值等。以下是一些常见的聚合类型：

- Terms聚合：根据指定字段的值进行分组，并返回每个分组的文档统计信息。
- Date Histogram聚合：按照时间间隔对文档进行分组，并返回每个时间间隔的文档统计信息。
- Range聚合：将文档分组到不同的范围，并返回每个范围的文档统计信息。

2.4.4DSL示例

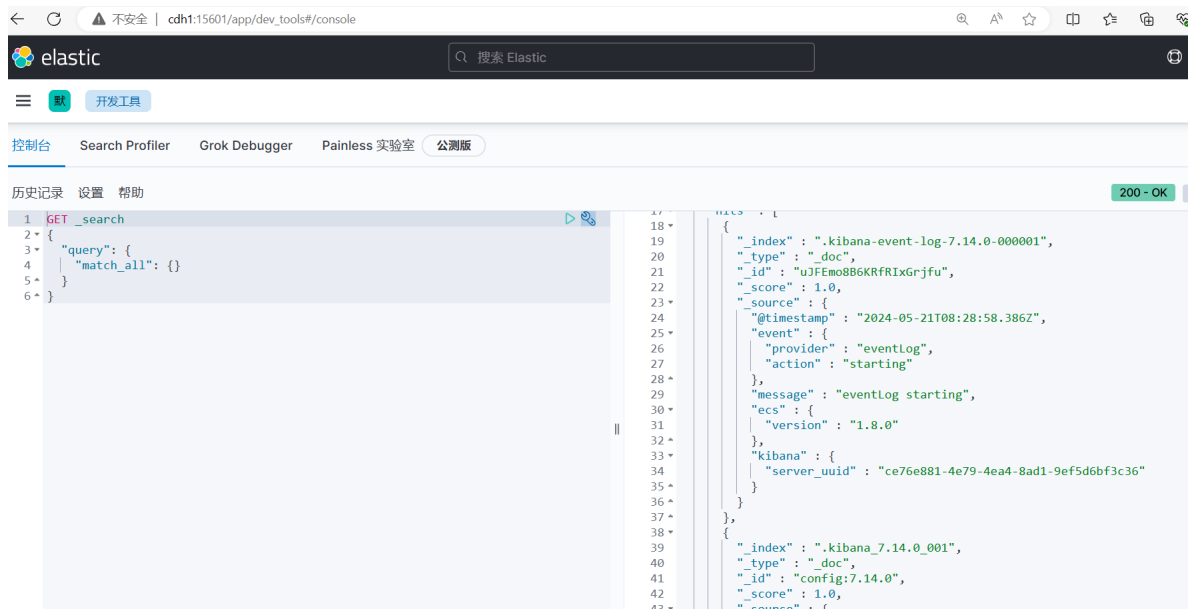
1.单独查询

我用的ES版本为7.15，以下所有示例均基于这个版本，不同版本在写法上略有差异，使用时要注意喔。

(1) 基本查询 (Match Query)

- 查询指定字段中匹配关键词或短语的文档。
- 示例：查询"title"字段包含"Elasticsearch"的文档。

```
GET /index/_search
{
  "query": {
    "match": {
      "title": "Elasticsearch"
    }
  }
}
```



(2) 多字段查询 (Multi-match Query)

- 在多个字段中匹配指定的关键词或短语。
- 示例：在"title"和"content"字段中匹配"Elasticsearch"的文档。

```
GET /index/_search
{
  "query": {
    "multi_match": {
      "query": "Elasticsearch",
      "fields": ["title", "content"]
    }
  }
}
```

(3) 范围查询 (Range Query)

- 根据范围条件匹配字段中的值。
- 示例：查询价格在50到100之间的文档。

```
GET /index/_search
{
  "query": {
    "range": {
      "price": {
        "gte": 50,
        "lte": 100
      }
    }
  }
}
```

(4) 布尔查询 (Bool Query)

- 将多个查询组合为逻辑上的AND、OR或NOT关系。

- 示例：查询标题包含"Elasticsearch"且价格大于等于50的文档。

```
GET /index/_search
{
  "query": {
    "bool": {
      "must": [
        { "match": { "title": "Elasticsearch" } },
        { "range": { "price": { "gte": 50 } } }
      ]
    }
  }
}
```

(5) 聚合查询 (Aggregation Query)

- 计算和统计数据集中的汇总信息。
- 示例：计算字段"sales"的总和作为结果返回。

```
GET /index/_search
{
  "aggs": {
    "total_sales": {
      "sum": { "field": "sales" }
    }
  }
}
```

(6) 排序 (Sorting)

- 对搜索结果按照指定字段进行排序。
- 示例：按照"timestamp"字段的降序对结果进行排序。

```
GET /index/_search
{
  "sort": [
    { "timestamp": { "order": "desc" } }
  ]
}
```

(7) 分页 (Pagination)

- 从搜索结果中获取特定范围的文档。
- 示例：返回从0开始的10个文档作为结果。

```
GET /index/_search
{
  "from": 0,
  "size": 10,
  "query": {
    "match_all": {}
  }
}
```

2.组合查询

因为大部分日常使用，都是组合查询，所以在此也罗列一些组合查询的示例。

(1) 组合多个must查询

- 示例：要求标题包含"Elasticsearch"且内容包含"数据分析"的文档。

```
GET /index/_search
{
  "query": {
    "bool": {
      "must": [
        { "match": { "title": "Elasticsearch" } },
        { "match": { "content": "数据分析" } }
      ]
    }
  }
}
```

(2) 组合must和should查询

- 示例：要求标题包含"Elasticsearch"且（价格大于等于50或评分高于4）的文档。

```
GET /index/_search
{
  "query": {
    "bool": {
      "must": [
        { "match": { "title": "Elasticsearch" } }
      ],
      "should": [
        { "range": { "price": { "gte": 50 } } },
        { "range": { "rating": { "gt": 4 } } }
      ]
    }
  }
}
```

(3) 组合must和must_not查询

- 示例：要求标题包含"Elasticsearch"且不包含"教程"的文档。

```
GET /index/_search
```

```
{
  "query": {
    "bool": {
      "must": [
        { "match": { "title": "Elasticsearch" } }
      ],
      "must_not": [
        { "match": { "title": "教程" } }
      ]
    }
  }
}
```

(4) 组合多个should查询

- 示例：要求标题包含"Elasticsearch"或内容包含"数据分析"或作者是"John"的文档。

```
GET /index/_search
{
  "query": {
    "bool": {
      "should": [
        { "match": { "title": "Elasticsearch" } },
        { "match": { "content": "数据分析" } },
        { "match": { "author": "John" } }
      ]
    }
  }
}
```

(5) 嵌套组合查询

- 示例：要求标题包含"Elasticsearch"且（价格大于等于50或评分高于4）且（内容包含"搜索引擎"或标签是"技术"）的文档。

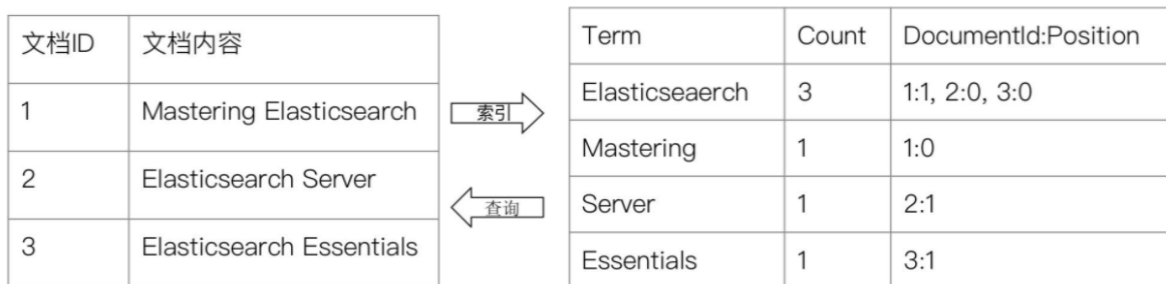
```
GET /index/_search
{
  "query": {
    "bool": {
      "must": [
        { "match": { "title": "Elasticsearch" } }
      ],
      "should": [
        { "range": { "price": { "gte": 50 } } },
        { "range": { "rating": { "gt": 4 } } }
      ],
      "minimum_should_match": 1,
      "filter": [
        { "match": { "content": "搜索引擎" } },
        { "terms": { "tags": ["技术"] } }
      ]
    }
  }
}
```

- 将多个查询组合使用bool查询的must、should或must_not子句。
- 示例：查询标题包含"Elasticsearch"且（价格大于等于50或评分高于4）的文档。

```
GET /index/_search
{
  "query": {
    "bool": {
      "must": [
        { "match": { "title": "Elasticsearch" } },
        {
          "bool": {
            "should": [
              { "range": { "price": { "gte": 50 } } },
              { "range": { "rating": { "gt": 4 } } }
            ]
          }
        }
      ]
    }
  }
}
```

3 ES 的分词器

3.1 倒排索引



如上图

- 左边的是正排索引，通过文档的id如查找文档的内容
- 右边的是倒排索引，通过单词统计次数以及文档的位置，

如Elasticsearch出现的次数为3，在id=1，id=2，id=3都出现过，且位置分别为1，0，0

3.2 默认分词器

默认分词器对于英文分词的效果如下

```
POST /_analyze
{
  "text": "You can use Elasticsearch to store, search, and manage data",
  "analyzer": "standard"
}
```

结果如下:

```
2 {
3   "tokens" : [
4     {
5       "token" : "you",
6       "start_offset" : 0,
7       "end_offset" : 3,
8       "type" : "<ALPHANUM>",
9       "position" : 0
10    },
11    {
12     "token" : "can",
13     "start_offset" : 4,
14     "end_offset" : 7,
15     "type" : "<ALPHANUM>",
16     "position" : 1
17    },
18    {
19     "token" : "use",
20     "start_offset" : 8,
21     "end_offset" : 11,
22     "type" : "<ALPHANUM>",
23     "position" : 2
24    },
25    {
26     "token" : "elasticsearch",
27     "start_offset" : 12,
28     "end_offset" : 25,
29     "type" : "<ALPHANUM>",
30     "position" : 3
31    }
32  ],
33 }
```

默认分词器对于中文的分词

```
POST /_analyze
{
  "text": "中华人民共和国人民大会堂",
  "analyzer": "standard"
}
```

```

2 {
3   "tokens" : [
4     {
5       "token" : "中",
6       "start_offset" : 0,
7       "end_offset" : 1,
8       "type" : "<IDEOGRAPHIC>",
9       "position" : 0
10    },
11    {
12      "token" : "华",
13      "start_offset" : 1,
14      "end_offset" : 2,
15      "type" : "<IDEOGRAPHIC>",
16      "position" : 1
17    },
18    {
19      "token" : "人",
20      "start_offset" : 2,
21      "end_offset" : 3,
22      "type" : "<IDEOGRAPHIC>",
23      "position" : 2
24    },
25    {
26      "token" : "民",
27      "start_offset" : 3,
28      "end_offset" : 4,
29      "type" : "<IDEOGRAPHIC>",
30      "position" : 3
31    },
32    {
33      "token" : "共",

```

3.3 中文分词

中文分词是中文文本处理的一个基础步骤，也是中文人机自然语言交互的基础模块。

不同于英文的是，中文句子中没有词的界限，因此在进行中文自然语言处理时，通常需要先进行分词，分词效果将直接影响词性、句法树等模块的效果。

当然分词只是一个工具，场景不同，要求也不同。

部分分词工具如下：

- 中科院计算所NLPIR <http://ictclas.nlpir.org/nlpir/>
- ansj分词器 https://github.com/NLPchina/ansj_seg
- 哈工大的LTP <https://github.com/HIT-SCIR/ltp>
- 清华大学THULAC <https://github.com/thunlp/THULAC>
- 斯坦福分词器 <https://nlp.stanford.edu/software/segmenter.shtml>
- Hanlp分词器 <https://github.com/hankcs/HanLP>

- 结巴分词 <https://github.com/yanyiwu/cppjieba>
- KCWS分词器(字嵌入+Bi-LSTM+CRF) <https://github.com/koth/kcws>
- ZPar <https://github.com/frcchang/zpar/releases>
- IAnalyzer <https://github.com/wks/ik-analyzer>

3.4 IK分词器

IK分词器下载地址<https://github.com/medcl/elasticsearch-analysis-ik/releases>

将ik分词器上传到服务器上，然后解压，并改名字为ik

```
mkdir ~/ik
mv elasticsearch-analysis-ik-7.15.2.zip ~/ik
unzip elasticsearch-analysis-ik-7.15.2.zip
```

将ik目录拷贝到docker容器的plugins目录下

```
docker cp ./ik es:/usr/share/elasticsearch/plugins
```

特别说明，尼恩的一键启动版本，已经默认戴上了IK分词器

IAnalyzer是一个开源的，基于java语言开发的轻量级的中文分词工具包，IK分词器分为两种模式

1. ik_max_word：会将文本做最细粒度的拆分

比如会将“中华人民共和国人民大会堂”拆分为“中华人民共和国、中华人民、中华、华人、人民共和国、人民、共和国、大会堂、大会、会堂等词语。

```
POST /_analyze
{
  "text": "中华人民共和国人民大会堂",
  "analyzer": "ik_max_word"
}
```

```

{
  "tokens" : [
    {
      "token" : "中华人民共和国",
      "start_offset" : 0,
      "end_offset" : 7,
      "type" : "CN_WORD",
      "position" : 0
    },
    {
      "token" : "中华人民",
      "start_offset" : 0,
      "end_offset" : 4,
      "type" : "CN_WORD",
      "position" : 1
    },
    {
      "token" : "中华",
      "start_offset" : 0,
      "end_offset" : 2,
      "type" : "CN_WORD",
      "position" : 2
    },
    {
      "token" : "华人",
      "start_offset" : 1,
      "end_offset" : 3,
      "type" : "CN_WORD",
      "position" : 3
    },
    {
      "token" : "人民共和国"
    }
  ]
}

```

2. ik_smart: 会做最粗粒度的拆分

比如会将“中华人民共和国人民大会堂”拆分为中华人民共和国、人民大会堂。

```

POST /_analyze
{
  "text": "中华人民共和国人民大会堂",
  "analyzer": "ik_smart"
}

```



```

2  {
3    "tokens" : [
4      {
5        "token" : "中华人民共和国",
6        "start_offset" : 0,
7        "end_offset" : 7,
8        "type" : "CN_WORD",
9        "position" : 0
10     },
11     {
12       "token" : "人民大会堂",
13       "start_offset" : 7,
14       "end_offset" : 12,
15       "type" : "CN_WORD",
16       "position" : 1
17     }
18   ]
19 }

```

3.5 自定义扩展字典

IK分词器的两种模式的最佳实践是：索引时用ik_max_word，搜索时用ik_smart，索引时最大化的将文章内容分词，搜索时更精确的搜索到想要的结果。

举个例子：用户输入“华为手机”搜索，此时应该搜索出“华为手机”的商品，而不是“华为”和“手机”这两个词，这样会搜索出华为其它的商品，

此时使用ik_smart和ik_max_word都会将华为手机拆分为华为和手机两个词，那些只包括“华为”这个词的信息也被搜索出来了，我的目标是搜索只包含华为手机这个词的信息，这没有满足我的目标。

ik_smart默认情况下分词“华为手机”，依然会分成两个词“华为”和“手机”，这时需要使用自定义扩展字典

1. 进入es

```
docker exec -it es /bin/bash
```

2. 增加自定义字典文件

如果容器编辑乱码，可以在宿主机编辑，然后拷贝到容器中

```

#进入ik配置目录
cd plugins/ik/config/
vi new_word.dic

```

内容如下：

```

老铁
王者荣耀
洪荒之力
共有产权房
一带一路
java日知录
华为手机

```

3. 修改配置文件

```
vi IKAnalyzer.cfg.xml
```

内容如下:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE properties SYSTEM "http://java.sun.com/dtd/properties.dtd">
<properties>
    <comment>IK Analyzer 扩展配置</comment>
    <!--用户可以在这里配置自己的扩展字典 -->
    <entry key="ext_dict">new_word.dic</entry>
    <!--用户可以在这里配置自己的扩展停止词字典-->
    <entry key="ext_stopwords"></entry>
    <!--用户可以在这里配置远程扩展字典 -->
    <!-- <entry key="remote_ext_dict">words_location</entry> -->
    <!--用户可以在这里配置远程扩展停止词字典-->
    <!-- <entry key="remote_ext_stopwords">words_location</entry> -->
</properties>
```

4. 拷贝到宿主机

```
docker cp es:/usr/share/elasticsearch/plugins/elasticsearch-analysis-ik/config
~/ik
```

5. 重启

```
docker restart es
```

4 高级的DSL 查询

前面介绍了基础的DSL查询，接下来，介绍一下高级的DSL 查询

ES提供基于DSL(Domain Specific Language)的索引查询模式，

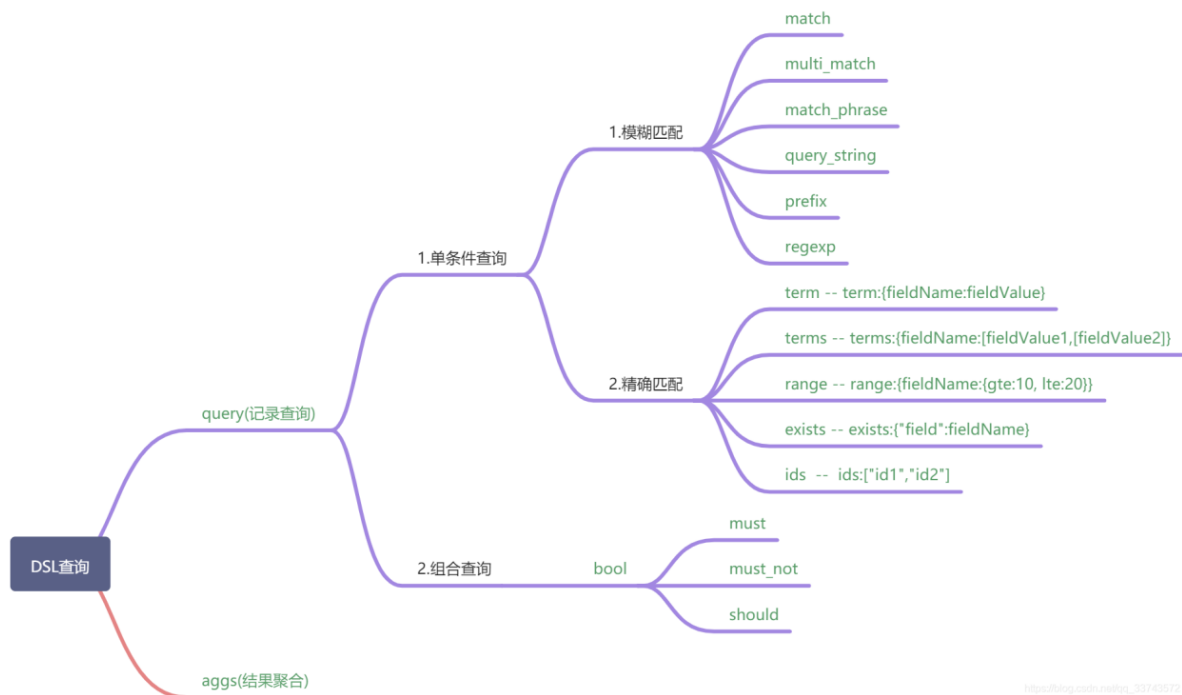
DSL查询基于JSON定义查询

Wikipedia对于DSL的定义"**为了解决某一类任务而专门设计的计算机语言**"

大师Martin Fowler对于DSL定义"**DSL 通过在表达能力上做的妥协换取在某一领域内的高效**"

我们在使用ES的时候，避免不了使用DSL语句去查询，就像使用关系型数据库的时候要学会SQL语法一样。

如果我们学习好了DSL语法的使用，那么在日后使用和使用Java Client调用时候也会变得非常简单。



Elasticsearch provides a full Query DSL (Domain Specific Language) based on JSON to define queries. Think of the Query DSL as an AST (Abstract Syntax Tree) of queries

4.1 管理索引

查看所有的索引

```
GET _cat/indices?v
```

帮助我们改进 Elastic Stack

要了解使用情况数据如何帮助我们管理和改善产品和服务，请参阅我们的[隐私声明](#)。要停止收集，请在此禁用使用情况数据。

关闭

控制台 Search Profiler Grok Debugger Painless 实验室 公测版

历史记录 设置 帮助

1	GET _cat/indices?v	200 - OK	59 ms
1	#! Elasticsearch built-in security features are not enabled. Without authentication, your cluster could be accessible to anyone. See https://www.elastic.co/guide/en/elasticsearch/reference/7.15/security-minimal-setup.html to enable security.		
2	health status index	uuid	pri rep docs.count docs.deleted store.size pri.store.size
3	green open .kibana-event-log-7.14.0-000001	RvGdM4eyQsyoghZuqX9TAQ	1 0 1 0 5.6kb 5.6kb
4	green open .kibana_7.14.0_001	oUexxKN0RBKZhJLM2jilwg	1 0 20 11 2.1mb 2.1mb
5	yellow open books	xZIECp1CRw6ppAdunRTP4A	1 1 0 0 208b 208b
6	green open .apm-custom-link	zBaU9Z9ySaK0SnqqPXFwNQ	1 0 0 0 208b 208b
7	green open .apm-agent-configuration	drdljXw0S36PPGX_oikyLQ	1 0 0 0 208b 208b
8	green open .kibana_task_manager_7.14.0_001	0hVFC-39QCekr7qN0sRZ_Q	1 0 14 2580 358.6kb 358.6kb
9	yellow open items	Qddhw-WtQEa1psdHy7Jvbg	1 1 0 0 208b 208b
10	yellow open users	4xS54RKeTDSLLeEKYhzaYg	1 1 0 0 208b 208b
11			

1. 删除某个索引

```
DELETE /skuinfo
```

2. 新增索引

```
PUT /user
```

3. 创建映射

```
PUT /user/_mapping
{
  "properties": {
    "name": {
      "type": "text",
      "analyzer": "ik_smart",
      "search_analyzer": "ik_smart"
    },
    "city": {
      "type": "text",
      "analyzer": "ik_smart",
      "search_analyzer": "ik_smart"
    },
    "age": {
      "type": "long"
    },
    "description": {
      "type": "text",
      "analyzer": "ik_smart",
      "search_analyzer": "ik_smart"
    }
  }
}
```

4. 新增文档数据

```
PUT /user/_doc/1
{
  "name": "李四",
  "age": 22,
  "city": "深圳",
  "description": "李四来自湖北武汉！"
}
```

再增加几条记录：

```
#新增文档数据 id=2
PUT /user/_doc/2
{
  "name": "王五",
  "age": 35,
  "city": "深圳",
  "description": "王五家住在深圳！"
}

#新增文档数据 id=3
PUT /user/_doc/3
{
  "name": "张三",
  "age": 19,
  "city": "深圳",
  "description": "在深圳打工，来自湖北武汉"
}

#新增文档数据 id=4
```

```

PUT /user/_doc/4
{
  "name": "张三丰",
  "age": 66,
  "city": "武汉",
  "description": "在武汉读书，家在武汉！"
}

#新增文档数据 id=5
PUT /user/_doc/5
{
  "name": "赵子龙",
  "age": 77,
  "city": "广州",
  "description": "赵子龙来自深圳宝安，但是在广州工作！",
  "address": "广东省茂名市"
}

#新增文档数据 id=6
PUT /user/_doc/6
{
  "name": "赵毅",
  "age": 55,
  "city": "广州",
  "description": "赵毅来自广州白云区，从事电子商务8年！"
}

#新增文档数据 id=7
PUT /user/_doc/7
{
  "name": "赵哈哈",
  "age": 57,
  "city": "武汉",
  "description": "武汉赵哈哈，在深圳打工已有半年了，月薪7500！"
}

```

5. 修改数据

(1) 操作1

更新数据可以使用之前的增加操作,这种操作会将整个数据替换掉，代码如下：

```

#更新数据,id=4
PUT /user/_doc/4
{
  "name": "张三丰",
  "description": "在武汉读书，家在武汉！在深圳工作！"
}

```

使用GET命令查看：

```

#根据ID查询
GET /user/_doc/4

```

(2) 操作2

我们先使用下面命令恢复数据：

```
#恢复文档数据 id=4
PUT /user/_doc/4
{
  "name": "张三丰",
  "age": 66,
  "city": "武汉",
  "description": "在武汉读书，家在武汉！"
}
```

使用POST更新某个列的数据

```
#使用POST更新某个域的数据
POST /user/_doc/4
{
  "doc": {
    "name": "张三丰",
    "description": "在武汉读书，家在武汉！在深圳工作！"
  }
}
```

6. 删除Document

```
#删除数据
DELETE user/userinfo/7
```

4.2 数据查询

1. 查询所有数据

```
#查询所有
GET /user/_search
```

2. 根据ID查询

```
#根据ID查询
GET /user/_doc/2
```

3. Sort排序

```
#搜索排序
GET /user/_search
{
  "query": {
    "match_all": {}
  },
  "sort": {
    "age": {
      "order": "desc"
    }
  }
}
```

4. 分页

```
#分页实现
GET /user/_search
{
  "query":{
    "match_all": {}
  },
  "sort":{
    "age":{
      "order":"desc"
    }
  },
  "from": 0,
  "size": 2
}
```

- from:从下N的记录开始查询
- size:每页显示条数

4.3 查询模式

1. term查询

term主要用于分词精确匹配，如字符串、数值、日期等（不适合情况：1.列中除英文字符外有其它值 2.字符串值中有冒号或中文 3.系统自带属性如_version）

```
#查询-term
GET _search
{
  "query":{
    "term":{
      "city":"武汉"
    }
  }
}
```

2. terms 查询

terms 跟 term 有点类似，但 terms 允许指定多个匹配条件。如果某个字段指定了多个值，那么文档需要一起去做匹配。

```
#查询-terms 允许多个Term
GET _search
{
  "query":{
    "terms":{
      "city":
        [
          "武汉",
          "广州"
        ]
    }
  }
}
```

3. match查询

```
GET _search
{
  "query": {
    "match": {
      "city": "广州武汉"
    }
  }
}
```

4. query_string查询

```
GET _search
{
  "query": {
    "query_string": {
      "default_field": "city",
      "query": "广州武汉"
    }
  }
}
```

5. range 查询

range过滤允许我们按照指定范围查找一批数据。例如我们查询年龄范围

```
#-range 范围过滤
#gt表示> gte表示=>
#lt表示< lte表示<=
GET _search
{
  "query": {
    "range": {
      "age": {
        "gte": 30,
        "lte": 57
      }
    }
  }
}
```

6. exists

exists 过滤可以用于查找拥有某个域的数据

#搜索 **exists**: 是指包含某个域的数据检索

```
GET _search
{
  "query": {
    "exists": {
      "field": "address"
    }
  }
}
```

7. bool 查询

bool 可以用来合并多个条件查询结果的布尔逻辑, 它包含以下操作符:

- **must**: 多个查询条件的完全匹配, 相当于 **and**。
- **must_not**: 多个查询条件的相反匹配, 相当于 **not**。
- **should**: 至少有一个查询条件匹配, 相当于 **or**。

这些参数可以分别继承一个过滤条件或者一个过滤条件的数组:

#过滤搜索 **bool**

#**must** : 多个查询条件的完全匹配, 相当于 **and**。

#**must_not** : 多个查询条件的相反匹配, 相当于 **not**。

#**should** : 至少有一个查询条件匹配, 相当于 **or**。

```
GET _search
{
  "query": {
    "bool": {
      "must": [
        {
          "term": {
            "city": {
              "value": "深圳"
            }
          }
        },
        {
          "range": {
            "age": {
              "gte": 20,
              "lte": 99
            }
          }
        }
      ]
    }
  }
}
```

8. match_all 查询

可以查询到所有文档, 是没有查询条件下的默认语句。

```
#查询所有 match_all
GET _search
{
  "query": {
    "match_all": {}
  }
}
```

9. match 查询

match查询是一个标准查询，不管你需要全文本查询还是精确查询基本上都要用到它。如果你使用match 查询一个全文本字段，它会在真正查询之前用分析器先分析match一下查询字符：

```
#字符串匹配
GET _search
{
  "query": {
    "match": {
      "description": "武汉广州"
    }
  }
}
```

10. prefix 查询

以什么字符开头的，可以更简单地用 prefix ,例如查询所有以张开始的用户描述

```
#前缀匹配 prefix
GET _search
{
  "query": {
    "prefix": {
      "name": {
        "value": "赵"
      }
    }
  }
}
```

10. multi_match 查询

multi_match查询允许你做match查询的基础上同时搜索多个字段，在多个字段中同时查一个

```
#多个域匹配搜索
GET _search
{
  "query": {
    "multi_match": {
      "query": "深圳",
      "fields": [
        "city",
        "description"
      ]
    }
  }
}
```

11. filter

因为过滤可以使用缓存，同时不计算分数，通常的规则是，使用查询（query）语句来进行全文搜索或者其它任何需要影响相关性得分的搜索。除此以外的情况都使用过滤（filters）

```
GET /user/_search
{
  "query": {
    "bool": {
      "filter": {
        "range": {
          "age": {
            "gte": 25,
            "lte": 80
          }
        }
      }
    }
  }
}
```

完整DSL案例代码如下：

```
GET _cat/health?v

GET _cat/nodes?v

GET _cat/indices?v

DELETE /user

PUT /user

PUT /user/_mapping
{
  "properties": {
    "name": {
      "type": "text",
      "analyzer": "ik_smart",
      "search_analyzer": "ik_smart"
    },
    "city": {
      "type": "text",
      "analyzer": "ik_smart",
      "search_analyzer": "ik_smart"
    },
    "age": {
      "type": "long"
    },
    "description": {
      "type": "text",
      "analyzer": "ik_smart",
      "search_analyzer": "ik_smart"
    }
  }
}
```

```
PUT /user/_doc/1
{
  "name": "李四",
  "age": 22,
  "city": "深圳",
  "description": "李四来自湖北武汉！"
}

#新增文档数据 id=2
PUT /user/_doc/2
{
  "name": "王五",
  "age": 35,
  "city": "深圳",
  "description": "王五家住在深圳！"
}

#新增文档数据 id=3
PUT /user/_doc/3
{
  "name": "张三",
  "age": 19,
  "city": "深圳",
  "description": "在深圳打工，来自湖北武汉"
}

#新增文档数据 id=4
PUT /user/_doc/4
{
  "name": "张三丰",
  "age": 66,
  "city": "武汉",
  "description": "在武汉读书，家在武汉！"
}

#新增文档数据 id=5
PUT /user/_doc/5
{
  "name": "赵子龙",
  "age": 77,
  "city": "广州",
  "description": "赵子龙来自深圳宝安，但是在广州工作！",
  "address": "广东省茂名市"
}

#新增文档数据 id=6
PUT /user/_doc/6
{
  "name": "赵毅",
  "age": 55,
  "city": "广州",
  "description": "赵毅来自广州白云区，从事电子商务8年！"
}

#新增文档数据 id=7
PUT /user/_doc/7
```

```
{
  "name": "赵哈哈",
  "age": 57,
  "city": "武汉",
  "description": "武汉赵哈哈，在深圳打工已有半年了，月薪7500！"
}
```

GET /user/_doc/4

#更新数据,id=4

PUT /user/_doc/4

```
{
  "name": "张三丰",
  "description": "在武汉读书，家在武汉！在深圳工作！"
}
```

PUT /user/_doc/4

```
{
  "name": "张三丰",
  "age": 66,
  "city": "武汉",
  "description": "在武汉读书，家在武汉！"
}
```

#使用POST更新某个域的数据

POST /user/_doc/4

```
{
  "doc": {
    "name": "张三丰",
    "description": "在武汉读书，家在武汉！在深圳工作！"
  }
}
```

GET /user/_search

GET /user/_doc/2

#搜索排序

GET /user/_search

```
{
  "query": {
    "match_all": {}
  },
  "sort": {
    "age": {
      "order": "desc"
    }
  }
}
```

#分页实现

GET /user/_search

```
{
  "query": {
    "match_all": {}
  },
}
```

```
"sort":{
  "age":{
    "order":"desc"
  }
},
"from": 0,
"size": 2
}
```

#查询-term

GET _search

```
{
  "query":{
    "term":{
      "city":"武汉"
    }
  }
}
```

#查询-terms 允许多个Term

GET _search

```
{
  "query":{
    "terms":{
      "city":
        [
          "武汉",
          "广州"
        ]
    }
  }
}
```

GET _search

```
{
  "query": {
    "match": {
      "city": "广州武汉"
    }
  }
}
```

GET _search

```
{
  "query": {
    "query_string": {
      "default_field": "city",
      "query": "广州武汉"
    }
  }
}
```

#-range 范围过滤

#gt表示> gte表示=>

#lt表示< lte表示<=

GET _search

```
{
  "query":{
```

```

    "range": {
      "age": {
        "gte": 30,
        "lte": 57
      }
    }
  }
}

```

#搜索 **exists**: 是指包含某个域的数据检索

GET **_search**

```

{
  "query": {
    "exists": {
      "field": "address"
    }
  }
}

```

GET **/user/_search**

#过滤搜索 **bool**

#**must** : 多个查询条件的完全匹配, 相当于 **and**。

#**must_not** : 多个查询条件的相反匹配, 相当于 **not**。

#**should** : 至少有一个查询条件匹配, 相当于 **or**。

GET **_search**

```

{
  "query": {
    "bool": {
      "must": [
        {
          "term": {
            "city": {
              "value": "深圳"
            }
          }
        },
        {
          "range": {
            "age": {
              "gte": 20,
              "lte": 99
            }
          }
        }
      ]
    }
  }
}

```

#查询所有 **match_all**

GET **_search**

```

{
  "query": {
    "match_all": {}
  }
}

```

```
#字符串匹配
GET _search
{
  "query": {
    "match": {
      "description": "武汉广州"
    }
  }
}
```

```
#前缀匹配 prefix
GET _search
{
  "query": {
    "prefix": {
      "name": {
        "value": "赵"
      }
    }
  }
}
```

```
#多个域匹配搜索
GET _search
{
  "query": {
    "multi_match": {
      "query": "深圳",
      "fields": [
        "city",
        "description"
      ]
    }
  }
}
```

```
GET /user/_search
{
  "query": {
    "bool": {
      "filter": {
        "range": {
          "age": {
            "gte": 25,
            "lte": 80
          }
        }
      }
    }
  }
}
```

4.4 使用别名

在mysql中 我们经常遇到产品修改需求 我们可能会在原有数据库表基础上 对字段 索引 类型进行修改比如 增加一个字段 添加一个字段的索引 又或者修改某个字段的类型，这一切都看起来这么自然 不过在ES这里却是行不通的

ES的mapping一旦设置了之后，可以改，但是改了没有用，因为ES默认是对所有字段进行索引 如果你修改了mapping 那么已经索引过的数据就必须全部重新索引一遍，

ES没有提供这个机制，只能利用别名手工刷数据，

1. 添加索引别名

```
PUT article1/_alias/article
{
  "acknowledged" : true
}
```

2. 创建新article2索引（增加了一个owner字段）

```
PUT article2
{
  "settings": {
    "number_of_shards": 3,
    "number_of_replicas": 1 ,
    "analysis" : {
      "analyzer" : {
        "ik" : {
          "tokenizer" : "ik_max_word"
        }
      }
    }
  },
  "mappings": {
    "properties": {
      "id": {
        "type": "keyword"
      },
      "title": {
        "type": "text",
        "analyzer": "ik_max_word"
      },
      "content": {
        "type": "text",
        "analyzer": "ik_max_word"
      },
      "viewCount": {
        "type": "integer"
      },
      "creatDate": {
        "type": "date",
        "format": "yyyy-MM-dd HH:mm:ss||yyyy-MM-dd||epoch_millis"
      },
      "tags": {
        "type": "keyword"
      },
      "owner": {
        "type": "keyword"
      }
    }
  }
}
```

```
}  
}
```

3. 重建索引 `reindex`

```
POST _reindex  
{  
  "source": {  
    "index": "article1"  
  },  
  "dest": {  
    "index": "article2"  
  }  
}
```

4. 修改别名映射

```
POST /_aliases  
{  
  "actions": [  
    {  
      "remove": {  
        "index": "article1",  
        "alias": "article"  
      }  
    },  
    {  
      "add": {  
        "index": "article2",  
        "alias": "article"  
      }  
    }  
  ]  
}
```

5. 使用别名搜索

```
GET /article/_search  
{  
  "query": {  
    "match_all": {}  
  }  
}
```

5 从0开始，ES工业级Java 应用开发

5.1 High Level Client基本用法

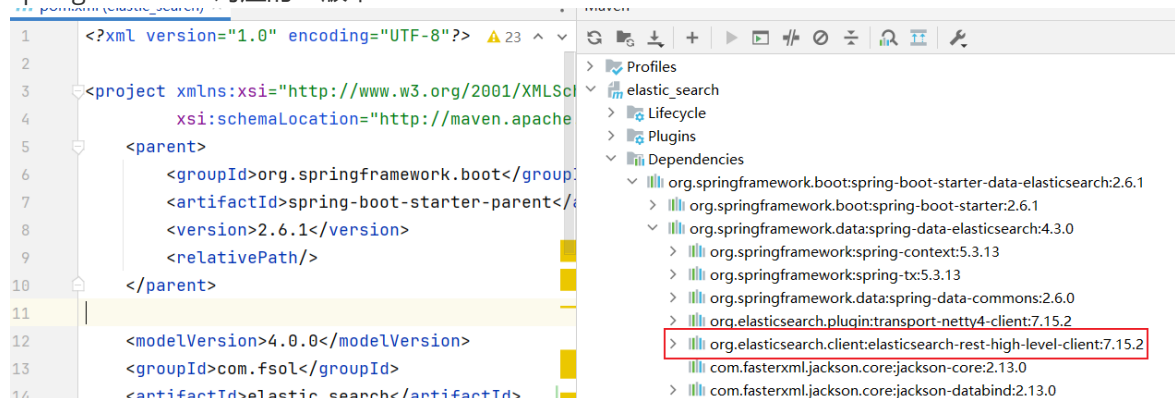
High Level Client客户端是构建于 Low Level Client之上的封装。

类似于Hibernate和JDBC的关系。

使用Spring Data Elasticsearch访问ElasticSearch,注意版本对应关系

Spring Data Release Train	Spring Data Elasticsearch	Elasticsearch	Spring Framework	Spring Boot
2022.0 (Turing)	5.0.x	8.5.0	6.0.x	3.0.x
2021.2 (Raj)	4.4.x	7.17.3	5.3.x	2.7.x
2021.1 (Q)	4.3.x	7.15.2	5.3.x	2.6.x
2021.0 (Pascal)	4.2.x ^[1]	7.12.0	5.3.x	2.5.x
2020.0 (Ockham) ^[1]	4.1.x ^[1]	7.9.3	5.3.2	2.4.x
Neumann ^[1]	4.0.x ^[1]	7.6.2	5.2.12	2.3.x

spring boot 2.6.1对应的ES版本



High Level Client客户端测试案例

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.NONE)
public class RestHighLevelClientTest
{

    //用来操作 Elasticsearch 服务器的一个客户端对象类
    @Autowired
    private RestHighLevelClient restHLCClient;

    @ParameterizedTest
    @ValueSource(ints = {1, 2, 3})
    void testWithSimpleValueSource(int argument) {
        System.out.println("Parameterized test with value: " + argument);
    }

    @Test
    @SneakyThrows
    void testWithSneakyThrows() {
        throw new Exception("This exception is sneaky thrown!");
    }

    @ParameterizedTest
    @CsvSource({"1, 1, 2", "2, 3, 5", "3, 5, 8"})
    void testAddition(int a, int b, int result) {
        int sum = a + b;
        Assertions.assertEquals(result, sum, "Sum of " + a + " and " + b + "
should be " + result);
    }

    //创建三个索引库
```

```

@ParameterizedTest //参数测试
@sneakyThrows
@ValueSource(strings = {"books", "items", "users"})
public void testCreateIndex(String indexName)
{
    //指定分词器创建索引库的json格式的数据，每一行用双引号包起来，然后里面的每个双引号前面
    用反斜杠\转义
    String json = "{" +
        "\"settings\": {" +
        "    \"analysis\": {" +
        "        \"analyzer\": {" +
        "            \"default\": {" +
        "                \"tokenizer\": \"ik_max_word\"" +
        "            }" +
        "        }" +
        "    }" +
        "}" +
        "\"}";

    CreateIndexRequest request = new CreateIndexRequest(indexName)
        //参数1: 指定创建索引库时要传入的参数 ; 参数2: 指定传入内容的类型
        .source(json, XContentType.JSON);
    //创建索引库后返回的响应类型--CreateIndexResponse
    CreateIndexResponse resp = restHLCClient.indices().create(request,
RequestOptions.DEFAULT);

    //获取Elasticsearch服务器的响应，就是响应索引库是否创建成功
    System.err.println(resp.isAcknowledged());
}

//删除索引库
@sneakyThrows
@ParameterizedTest
@ValueSource(strings = {"items", "users"})
public void testDeleteIndex(String indexName)
{
    //删除索引的请求数据
    DeleteIndexRequest request = new DeleteIndexRequest(indexName);
    //客户端调用操作索引的方法，然后再调用删除的方法
    AcknowledgedResponse resp = restHLCClient.indices().delete(request,
RequestOptions.DEFAULT);
    //查看删除后的响应
    System.err.println(resp.isAcknowledged());
}

//查询所有的索引库
@sneakyThrows
@Test //这个测试不需要参数，直接用这个@Test注解即可
public void testGetIndex()
{
    //参数 "*" : 表示匹配所有的索引库
    GetIndexRequest request = new GetIndexRequest("*");
    //用rest客户端的方法来查询
    GetIndexResponse resp = restHLCClient.indices().get(request,
RequestOptions.DEFAULT);

```

```

        //返回的索引库是一个String类型的数组
        String[] indices = resp.getIndices();
        //把数组转成字符串
        String s = Arrays.toString(indices);
        System.err.println(s);

    }

    //往索引库添加文档

    @ParameterizedTest
    @SneakyThrows
    //测试参数有多个值，用这个注解
    @CsvSource({
        "1,火影忍者,漩涡鸣人成长为第七代火影的故事,150",
        "2,家庭教师,废材纲成长为十代首领的热血事迹,200",
        "4,七龙珠Z,超级赛亚人贝吉塔来到地球后的热闹景象,400"
    })
    public void testSaveDocument(Integer id, String title, String description,
        Double price)
    {
        //表明向 books 索引库添加文档
        IndexRequest request = new IndexRequest("books")
            .id(id + "")
            .source(
                "title", title,
                "description", description,
                "price", price
            );

        IndexResponse resp = restHLCClient.index(request,
            RequestOptions.DEFAULT);
        System.err.println(resp);
    }

    //根据文档的id获取文档
    @SneakyThrows
    @ParameterizedTest
    @ValueSource(ints = {1, 3})
    public void testGetDocumentById(Integer id)
    {
        //表明从 books 索引库获取文档
        GetRequest request = new GetRequest("books")
            //表明根据指定的文档的id获取文档
            .id(id + "");

        GetResponse resp = restHLCClient.get(request, RequestOptions.DEFAULT);

        System.err.println(resp);
    }

    //根据条件查询文档（普通关键字查询和通配符查询）

    @SneakyThrows

```

```

@ParameterizedTest
@CsvSource({
    "description,热*",
    "description,成长"
})
public void testSearchDocument(String field, String term)
{
    // 构建查询条件的类
    SearchSourceBuilder builder = new SearchSourceBuilder();

    // 通过 SearchSourceBuilder 可以用面向对象的方式来构建查询的 JSON 字符串
    // SearchSourceBuilder 需要传入 QueryBuilders, 而 QueryBuilders 用于构建
    QueryBuilder
    if (term != null && term.contains("*"))
    {
        //根据字段和通配符关键字查询
        builder.query(QueryBuilders.wildcardQuery(field, term));
    } else
    {
        //根据字段和普通关键字查询
        builder.query(QueryBuilders.matchQuery(field, term));
    }

    //表明从 books 索引库查询文档
    SearchRequest request = new SearchRequest("books")
        // 此处的 builder 参数用于构建查询语法
        .source(builder);

    //客户端调用查询的方法 , 参数1: 查询条件语法 参数2: 默认的请求选项, 比如超时时间之
    类的
    SearchResponse resp = restHLCClient.search(request,
    RequestOptions.DEFAULT);

    System.err.println(resp);

}

//根据 id 删除文档

@ParameterizedTest
@SneakyThrows
@ValueSource(ints = {3,4})
public void testDeleteDocumentById(Integer id)
{
    //表明从 books 索引库删除文档
    DeleteRequest request = new DeleteRequest("books")
        //获取指定id的文档
        .id(id+"");
    //rest客户端调用删除文档的方法
    DeleteResponse resp = restHLCClient.delete(request,
    RequestOptions.DEFAULT);
    System.err.println(resp);
}

}

```

5.2 聚合查询

5.2.1 DSL聚合查询

1. 创建测试索引

```
PUT /jh_test
{
  "settings": {},
  "mappings": {
    "properties": {
      "name": {
        "type": "text",
        "fields": {
          "keyword": {
            "type": "keyword",
            "ignore_above": 256
          }
        }
      },
      "sex": {
        "type": "keyword"
      },
      "buyCount": {
        "type": "long"
      },
      "createMonth": {
        "type": "keyword"
      }
    }
  }
}
```

其中字段的含义为：name：姓名、buyCount：购买数量，sex：性别，createMonth：创建月
添加测试数据

```
POST /jh_test/_doc/1
{"name": "张三", "buyCount": 5, "sex": "男", "createMonth": "2021-01"}
POST /jh_test/_doc/2
{"name": "李四", "buyCount": 5, "sex": "男", "createMonth": "2021-01"}
POST /jh_test/_doc/3
{"name": "小卷", "buyCount": 18, "sex": "女", "createMonth": "2021-01"}
POST /jh_test/_doc/4
{"name": "小明", "buyCount": 6, "sex": "女", "createMonth": "2021-01"}
POST /jh_test/_doc/5
{"name": "张三", "buyCount": 3, "sex": "男", "createMonth": "2021-02"}
POST /jh_test/_doc/6
{"name": "王五", "buyCount": 8, "sex": "男", "createMonth": "2021-02"}
POST /jh_test/_doc/7
{"name": "赵四", "buyCount": 4, "sex": "男", "createMonth": "2021-02"}
```

```

POST /jh_test/_doc/8
{"name":"诸葛亮","buyCount":6,"sex":"男","createMonth":"2021-02"}
POST /jh_test/_doc/9
{"name":"黄忠","buyCount":9,"sex":"男","createMonth":"2021-03"}
POST /jh_test/_doc/10
{"name":"李白","buyCount":1,"sex":"男","createMonth":"2021-03"}
POST /jh_test/_doc/11
{"name":"赵四","buyCount":3,"sex":"男","createMonth":"2021-03"}
POST /jh_test/_doc/12
{"name":"张三","buyCount":2,"sex":"男","createMonth":"2021-03"}
POST /jh_test/_doc/13
{"name":"李四","buyCount":6,"sex":"男","createMonth":"2021-04"}
POST /jh_test/_doc/14
{"name":"王五","buyCount":9,"sex":"男","createMonth":"2021-04"}
POST /jh_test/_doc/15
{"name":"李四","buyCount":4,"sex":"男","createMonth":"2021-04"}
POST /jh_test/_doc/16
{"name":"王五","buyCount":2,"sex":"男","createMonth":"2021-04"}

```

聚合查询语法：

```

"aggs" : {
  "<aggregation_name>" : {                                     <!-- 聚合名称 -->
    "<aggregation_type>" : {                                   <!-- 聚合类型 -->
      <aggregation_body>                                       <!-- 聚合具体字段 -->
    }
    [, "meta" : { [<meta_data_body> ] } ]?                     <!-- 元信息 -->
    [, "aggs" : { [<sub_aggregation>]+ } ]?                   <!-- 子聚合 -->
  }
}

```

1. 查询 buyCount 的总和

```

GET /jh_test/_search
{
  "size":0,
  "aggs":{
    "buyCountSum":{
      "sum": {
        "field": "buyCount"
      }
    }
  }
}

```

2. 查询 2021-02 月 buyCount 的总和

```

{
  "size":0,
  "query": {
    "term": {

```



```

        "createMonth": "2021-02"
    }
},
"aggs":{
    "buyCountSum":{
        "sum": {
            "field": "buyCount"
        }
    }
}
}
}

```

3. 查询 2021-03 月 buyCount 的最大值:

```

{
    "size":0,
    "query": {
        "term": {
            "createMonth": "2021-03"
        }
    },
    "aggs":{
        "buyCountMax":{
            "max": {
                "field": "buyCount"
            }
        }
    }
}
}

```

4. 查询 2021-03 月 buyCount 的最小值:

```

{
    "size":0,
    "query": {
        "term": {
            "createMonth": "2021-03"
        }
    },
    "aggs":{
        "buyCountMin":{
            "min": {
                "field": "buyCount"
            }
        }
    }
}
}

```

5. 同时查询 2021-03 月 buyCount 的最大值和最小值:

```

{
    "size":0,

```

```

    "query": {
      "term": {
        "createMonth": "2021-03"
      }
    },
    "aggs": {
      "buyCountMax": {
        "max": {
          "field": "buyCount"
        }
      },
      "buyCountMin": {
        "min": {
          "field": "buyCount"
        }
      }
    }
  }
}

```

6. 查询所有 name 的去重后的数量

```

{
  "size": 0,
  "aggs": {
    "distinctName": {
      "cardinality": {
        "field": "name.keyword"
      }
    }
  }
}

```

7. 查询 2021-04 月 name 的去重后的数量

```

{
  "size": 0,
  "query": {
    "term": {
      "createMonth": "2021-04"
    }
  },
  "aggs": {
    "distinctName": {
      "cardinality": {
        "field": "name.keyword"
      }
    }
  }
}

```

8. 查询 BuyCount 的平均值

```
{
  "size": "0",
  "aggs": {
    "buyCountAvg": {
      "avg": {
        "field": "buyCount"
      }
    }
  }
}
```

9. 一次查询 总数，最大值，最小值，平均值，总和

```
{
  "size": 0,
  "aggs": {
    "statsAll": {
      "stats": {
        "field": "buyCount"
      }
    }
  }
}
```

10. 根据 createMonth 分组查询每个月的最大 buyCount

```
{
  "size": 0,
  "aggs": {
    "createMonthGroup": {
      "terms": {
        "field": "createMonth"
      },
      "aggs": {
        "buyCountMax": {
          "max": {
            "field": "buyCount"
          }
        }
      }
    }
  }
}
```

11. 查询每 createMonth 下，根据 sex 区分，统计 buyCount 的平均值

```
{
  "size": 0,
  "aggs": {
    "createMonthGroup": {
      "terms": {
```



```

    */
    @Test
    void aggs2() {
        QueryBuilder queryBuilder = QueryBuilders.termQuery("createMonth",
"2021-02");
        SumAggregationBuilder buyCountSum =
AggregationBuilders.sum("buyCountSum").field("buyCount");
        Query query = new NativeSearchQueryBuilder()
            .withQuery(queryBuilder)
            .withAggregations(buyCountSum)
//            .addAggregation(buyCountSum)
            .build();
        SearchHits<JhTestEntity> search =
elasticsearchRestTemplate.search(query, JhTestEntity.class);
        if (search.hasAggregations()) {
            Aggregations aggregations = (Aggregations)
search.getAggregations().aggregations();
            if (Objects.nonNull(aggregations)) {
                Sum sum = aggregations.get("buyCountSum");
                log.info("计算 buyCount 总数: {} ", sum.getValue());
            }
        }
    }

    /**
     * 查询 2021-03 月 buyCount 的最大值:
     */
    @Test
    void aggs3() {
        QueryBuilder queryBuilder = QueryBuilders.termQuery("createMonth",
"2021-03");
        MaxAggregationBuilder buyCountMax =
AggregationBuilders.max("buyCountMax").field("buyCount");
        Query query = new NativeSearchQueryBuilder()
            .withQuery(queryBuilder)
            .withAggregations(buyCountMax)
//            .addAggregation(buyCountMax)
            .build();
        SearchHits<JhTestEntity> search =
elasticsearchRestTemplate.search(query, JhTestEntity.class);
        if (search.hasAggregations()) {
            Aggregations aggregations = (Aggregations)
search.getAggregations().aggregations();
            if (Objects.nonNull(aggregations)) {
                Max max = aggregations.get("buyCountMax");
                log.info("计算 buyCount 最大值: {} ", max.getValue());
            }
        }
    }

    /**
     * 查询 2021-03 月 buyCount 的最小值:
     */
    @Test
    void aggs4() {
        QueryBuilder queryBuilder = QueryBuilders.termQuery("createMonth",
"2021-03");
    }

```

```

        MinAggregationBuilder buyCountMin =
AggregationBuilders.min("buyCountMin").field("buyCount");
        Query query = new NativeSearchQueryBuilder()
            .withQuery(queryBuilder)
            .withAggregations(buyCountMin)
//            .addAggregation(buyCountMin)
            .build();
        SearchHits<JhTestEntity> search =
elasticsearchRestTemplate.search(query, JhTestEntity.class);
        if (search.hasAggregations()) {
            Aggregations aggregations = (Aggregations)
search.getAggregations().aggregations();
            if (Objects.nonNull(aggregations)) {
                Min min = aggregations.get("buyCountMin");
                log.info("计算 buyCount 最小值: {} ", min.getValue());
            }
        }
    }

/**
 * 同时查询 2021-03 月 buyCount 的最大值和最小值:
 */
@Test
void aggs5() {
    QueryBuilder queryBuilder = QueryBuilders.termQuery("createMonth",
"2021-03");
    MaxAggregationBuilder buyCountMax =
AggregationBuilders.max("buyCountMax").field("buyCount");
    MinAggregationBuilder buyCountMin =
AggregationBuilders.min("buyCountMin").field("buyCount");
    Query query = new NativeSearchQueryBuilder()
        .withQuery(queryBuilder)
        .withAggregations(buyCountMax)
        .withAggregations(buyCountMin)
//            .addAggregation(buyCountMax)
//            .addAggregation(buyCountMin)
        .build();
    SearchHits<JhTestEntity> search =
elasticsearchRestTemplate.search(query, JhTestEntity.class);
    if (search.hasAggregations()) {
        Aggregations aggregations = (Aggregations)
search.getAggregations().aggregations();
        if (Objects.nonNull(aggregations)) {
            Max max = aggregations.get("buyCountMax");
            log.info("计算 buyCount 最大值: {} ", max.getValue());
            Min min = aggregations.get("buyCountMin");
            log.info("计算 buyCount 最小值: {} ", min.getValue());
        }
    }
}

/**
 * 查询所有 name 的去重后的数量
 */
@Test
void aggs6() {
    CardinalityAggregationBuilder distinctName =
AggregationBuilders.cardinality("distinctName").field("name.keyword");

```

```

        Query query = new NativeSearchQueryBuilder()
            .withAggregations(distinctName)
//            .addAggregation(distinctName)
            .build();
        SearchHits<JhTestEntity> search =
elasticsearchRestTemplate.search(query, JhTestEntity.class);
        if (search.hasAggregations()) {
            Aggregations aggregations = (Aggregations)
search.getAggregations().aggregations();
            if (Objects.nonNull(aggregations)) {
                Cardinality cardinality = aggregations.get("distinctName");
                log.info("计算 name 的去重后的数量: {} ", cardinality.getValue());
            }
        }
    }

/**
 * 查询 2021-04 月 name 的去重后的数量
 */
@Test
void aggs7() {
    QueryBuilder queryBuilder = QueryBuilders.termQuery("createMonth",
"2021-04");
    CardinalityAggregationBuilder distinctName =
AggregationBuilders.cardinality("distinctName").field("name.keyword");
    Query query = new NativeSearchQueryBuilder()
        .withQuery(queryBuilder)
        .withAggregations(distinctName)
//        .addAggregation(distinctName)
        .build();
    SearchHits<JhTestEntity> search =
elasticsearchRestTemplate.search(query, JhTestEntity.class);
    if (search.hasAggregations()) {
        Aggregations aggregations = (Aggregations)
search.getAggregations().aggregations();
        if (Objects.nonNull(aggregations)) {
            Cardinality cardinality = aggregations.get("distinctName");
            log.info("计算 name 的去重后的数量: {} ", cardinality.getValue());
        }
    }
}

/**
 * 查询 BuyCount 的平均值
 */
@Test
void aggs8() {
    AvgAggregationBuilder buyCountAvg =
AggregationBuilders.avg("buyCountAvg").field("buyCount");
    Query query = new NativeSearchQueryBuilder()
        .withAggregations(buyCountAvg)
//        .addAggregation(buyCountAvg)
        .build();
    SearchHits<JhTestEntity> search =
elasticsearchRestTemplate.search(query, JhTestEntity.class);
    if (search.hasAggregations()) {

```

```

        Aggregations aggregations = (Aggregations)
search.getAggregations().aggregations();
        if (Objects.nonNull(aggregations)) {
            Avg avg = aggregations.get("buyCountAvg");
            log.info("计算 buyCount 的平均值: {} ", avg.getValue());
        }
    }
}

/**
 * 一次查询 总数, 最大值, 最小值, 平均值, 总和
 */
@Test
void aggs9() {
    StatsAggregationBuilder stats =
AggregationBuilders.stats("stats").field("buyCount");
    Query query = new NativeSearchQueryBuilder()
        .withAggregations(stats)
//        .addAggregation(stats)
        .build();
    SearchHits<JhTestEntity> search =
elasticsearchRestTemplate.search(query, JhTestEntity.class);
    if (search.hasAggregations()) {
        Aggregations aggregations = (Aggregations)
search.getAggregations().aggregations();
        if (Objects.nonNull(aggregations)) {
            Stats s = aggregations.get("stats");
            log.info("计算 buyCount 的 count: {} ", s.getCount());
            log.info("计算 buyCount 的 min: {} ", s.getMin());
            log.info("计算 buyCount 的 max: {} ", s.getMax());
            log.info("计算 buyCount 的 avg: {} ", s.getAvg());
            log.info("计算 buyCount 的 sum: {} ", s.getSum());
        }
    }
}

/**
 * 根据 createMonth 分组查询每个月的最大 buyCount
 */
@Test
void aggs10() {
    TermsAggregationBuilder createMonthGroup =
AggregationBuilders.terms("createMonthGroup").field("createMonth")

        .subAggregation(AggregationBuilders.max("buyCountMax").field("buyCount"));

    Query query = new NativeSearchQueryBuilder()
        .withAggregations(createMonthGroup)
//        .addAggregation(createMonthGroup)
        .build();
    SearchHits<JhTestEntity> search =
elasticsearchRestTemplate.search(query, JhTestEntity.class);
    if (search.hasAggregations()) {
        Aggregations aggregations = (Aggregations)
search.getAggregations().aggregations();
        if (Objects.nonNull(aggregations)) {
            Terms terms = aggregations.get("createMonthGroup");
            terms.getBuckets().forEach(bucket -> {

```



```

        String createMonth = bucket.getKeyAsString();
        Aggregations subAggs = bucket.getAggregations();
        if (Objects.nonNull(subAggs)) {
            Max max = subAggs.get("buyCountMax");
            log.info("计算 {} 月的最大值为: {} ", createMonth,
max.getValue());
        }
    });
}

/**
 * 查询每 createMonth 下, 根据 sex 区分, 统计buyCount 的平均值
 */
@Test
void aggs11() {
    TermsAggregationBuilder createMonthGroup =
AggregationBuilders.terms("createMonthGroup").field("createMonth")

.subAggregation(AggregationBuilders.terms("sexGroup").field("sex")

.subAggregation(AggregationBuilders.avg("buyCountAvg").field("buyCount")));

    Query query = new NativeSearchQueryBuilder()
        .withAggregations(createMonthGroup)
//        .addAggregation(createMonthGroup)
        .build();

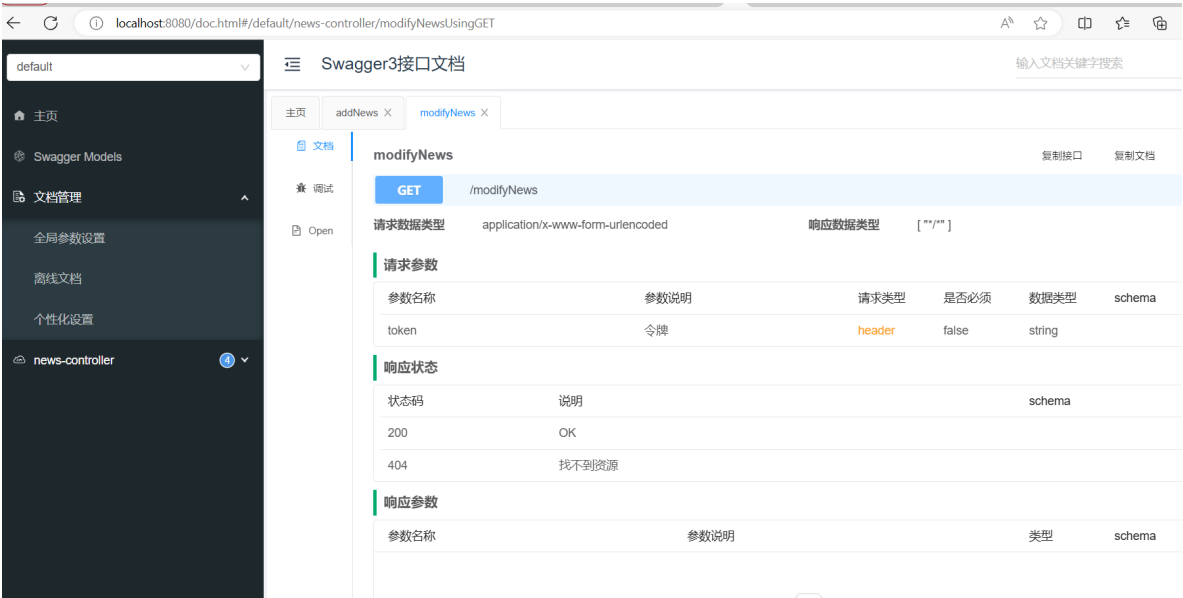
    SearchHits<JhTestEntity> search =
elasticsearchRestTemplate.search(query, JhTestEntity.class);
    if (search.hasAggregations()) {
        Aggregations aggregations = (Aggregations)
search.getAggregations().aggregations();
        if (Objects.nonNull(aggregations)) {
            Terms terms = aggregations.get("createMonthGroup");
            terms.getBuckets().forEach(bucket -> {
                String createMonth = bucket.getKeyAsString();
                Aggregations sexAggs = bucket.getAggregations();
                if (Objects.nonNull(sexAggs)) {
                    Terms sexTerms = sexAggs.get("sexGroup");
                    sexTerms.getBuckets().forEach(sexBucket -> {
                        String sex = sexBucket.getKeyAsString();
                        Aggregations avgAggs = sexBucket.getAggregations();
                        if (Objects.nonNull(avgAggs)) {
                            Avg avg = avgAggs.get("buyCountAvg");
                            log.info("计算 {} 月, {} 性 的平均值为: {} ",
createMonth, sex, avg.getValue());
                        }
                    });
                }
            });
        }
    }
}

```

5.3 ES ES工业级Java 应用开发

一般不会直接使用RestHighLevelClient对于ES操作，因为语法过于繁琐，重复，基于分离变和不变的原则可以进行工业级封装。

<http://localhost:8080/doc.html>



工业级封装后的Service:

```
/**
 * ES服务类
 */
@Component
public class ESRestServiceImpl implements ESRestService {

    public static final String NUMBER_OF_SHARDS = "index.number_of_shards",
    NUMBER_OF_REPLICAS = "index.number_of_replicas";

    @Autowired
    RestHighLevelClient highLevelClient;

    /**
     * createIndex
     *
     * @param clazz
     * @throws IOException
     */
    public boolean createIndex(Class<? extends BaseEntity> clazz) throws
    IOException {
        return createIndex(clazz.getSimpleName().toLowerCase(), clazz);
    }
}
```

```

    }

    /**
     * createIndex
     *
     * @param indexName
     * @param clazz
     * @return
     * @throws IOException
     */
    public boolean createIndex(String indexName, Class<? extends BaseEntity>
clazz) throws IOException {
        return createIndex(indexName, clazz, 1, 1);
    }

    /**
     * createIndex
     *
     * @param indexName
     * @param numberOfShards
     * @param numberOfReplicas
     * @return
     * @throws IOException
     */
    public boolean createIndex(String indexName, Class<? extends BaseEntity>
clazz, int numberOfShards, int numberOfReplicas) throws IOException {
        CreateIndexRequest request = new CreateIndexRequest(indexName);
        request.settings(Settings.builder().put(NUMBER_OF_SHARDS,
numberOfShards).put(NUMBER_OF_REPLICAS, numberOfReplicas));
        // 字段映射
        Map<String, Object> mapping = new HashMap<>();
        mapping.put("properties", getMappingProperties(clazz));
        request.mapping(mapping);
        CreateIndexResponse createIndexResponse =
highLevelClient.indices().create(request, RequestOptions.DEFAULT);
        return createIndexResponse.isAcknowledged() &&
createIndexResponse.isShardsAcknowledged();
    }

    /**
     * getMappingProperties
     *
     * @param clazz
     * @return
     */
    private Map<String, Object> getMappingProperties(Class<?> clazz) {
        Field[] fields = clazz.getDeclaredFields();
        Map<String, Object> mappingMap = new HashMap<>();
        for (int i = 0; i < fields.length; i++) {
            Field field = fields[i];
            field.setAccessible(true);

            Map<String, Object> typeMap = new HashMap<>();
            if (field.getGenericType().toString().equals("class
java.lang.String")) {
                typeMap.put("type", "text");
                mappingMap.put(field.getName(), typeMap);
                continue;
            }

```

```

    }

    if (field.getGenericType().toString().equals("int") ||
field.getGenericType().toString().equals("class java.lang.Integer")) {
        typeMap.put("type", "integer");
        mappingMap.put(field.getName(), typeMap);
        continue;
    }

    if (field.getGenericType().toString().equals("long") ||
field.getGenericType().toString().equals("class java.lang.Long")) {
        typeMap.put("type", "long");
        mappingMap.put(field.getName(), typeMap);
        continue;
    }

    if (field.getGenericType().toString().equals("double") ||
field.getGenericType().toString().equals("class java.lang.Double")) {
        typeMap.put("type", "double");
        mappingMap.put(field.getName(), typeMap);
        continue;
    }

    if (field.getGenericType().toString().equals("boolean") ||
field.getGenericType().toString().equals("class java.lang.Boolean")) {
        typeMap.put("type", "boolean");
        mappingMap.put(field.getName(), typeMap);
        continue;
    }

    if (field.getGenericType().toString().equals("class
java.util.Date")) {
        typeMap.put("type", "date");
        mappingMap.put(field.getName(), typeMap);
        continue;
    }

    if (field.getGenericType().toString().equals("class
java.lang.Object")) {
        typeMap.put("type", "object");
        mappingMap.put(field.getName(), typeMap);
    }
}
return mappingMap;
}

/**
 * isExistsIndex
 *
 * @param indexName
 * @return
 * @throws IOException
 */
public boolean isExistsIndex(String indexName) throws IOException {
    GetIndexRequest request = new GetIndexRequest(indexName);
    return highLevelClient.indices().exists(request,
RequestOptions.DEFAULT);
}

```

```

    }

    /**
     * getIndexSetting
     *
     * @param indexName
     * @return
     * @throws IOException
     */
    public Map<String, String> getIndexSetting(String indexName) throws
IOException {
        GetSettingsRequest request = new
GetSettingsRequest().indices(indexName);
        GetSettingsResponse getSettingsResponse =
highLevelClient.indices().getSettings(request, RequestOptions.DEFAULT);
        System.out.println(JSON.toJSONString(getSettingsResponse));
        return new HashMap<>();
    }

    /**
     * deleteIndex
     *
     * @param indexName
     * @return
     * @throws IOException
     */
    public boolean deleteIndex(String indexName) throws IOException {
        try {
            DeleteIndexRequest request = new DeleteIndexRequest(indexName);
            return highLevelClient.indices().delete(request,
RequestOptions.DEFAULT).isAcknowledged();
        } catch (ElasticsearchException exception) {
            if (exception.status() == RestStatus.NOT_FOUND) {
                return true;
            }
        }
        return false;
    }

    /**
     * createDoc
     *
     * @param object
     * @return
     * @throws IOException
     */
    public <T extends BaseEntity> boolean createDoc(T object) throws IOException
{
        return this.createDoc(object.getClass().getSimpleName().toLowerCase(),
object);
    }

    /**
     * createDoc
     *
     * @param indexName
     * @param object
     * @return

```

```

        * @throws IOException
        */
        public <T extends BaseEntity> boolean createDoc(String indexName, T object)
        throws IOException {
            createIndexIfNotExist(indexName, object);
            IndexResponse response =
            highLevelClient.index(createIndexRequest(indexName, object),
            RequestOptions.DEFAULT);
            return response.getResult() == DocWriteResponse.Result.CREATED;
        }

        /**
         * createDoc
         *
         * @param indexName
         * @param dataMap
         * @return
         * @throws IOException
         */
        public boolean createDoc(String indexName, Map<String, Object> dataMap)
        throws IOException {
            if (!existsIndex(indexName)) {
                return false;
            }
            IndexResponse response =
            highLevelClient.index(createIndexRequest(indexName, dataMap),
            RequestOptions.DEFAULT);
            return response.getResult() == DocWriteResponse.Result.CREATED;
        }

        /**
         * createIndexIfNotExist
         *
         * @param indexName
         * @param t
         * @param <T>
         * @throws IOException
         */
        private <T extends BaseEntity> void createIndexIfNotExist(String indexName, T
        t) throws IOException {
            //如果索引不存在就创建索引，其实这样不好，最好是抛出异常，必须主动创建索引
            if (!existsIndex(indexName)) {
                this.createIndex(indexName, t.getClass());
            }
        }

        /**
         * @param object
         * @param <T>
         * @return
         * @throws NoSuchFieldException
         * @throws IllegalAccessException
         */
        private <T> String getIdFromInstance(T object) throws NoSuchFieldException,
        IllegalAccessException {
            Field field = object.getClass().getDeclaredField(PK_ID);
            field.setAccessible(true);
            Object ido = field.get(object);

```

```

        if (ido == null) {
            return null;
        }
        String ids = ido.toString().trim();
        if (StringUtils.hasText(ids)) {
            return ids;
        }
        return null;
    }

    /**
     * createBatchDoc
     *
     * @param indexName
     * @param dataList
     * @return
     * @throws IOException
     */
    public <T extends BaseEntity> BulkResponse createBatchDoc(String indexName,
        List<T> dataList) throws IOException, NoSuchFieldException,
        IllegalAccessException {

        if (dataList == null || dataList.isEmpty()) {
            return null;
        }

        createIndexIfNotExist(indexName, dataList.get(0));

        BulkRequest bulkRequest = new BulkRequest();
        for (T t : dataList) {
            bulkRequest.add(createIndexRequest(indexName, t));
        }
        return highLevelClient.bulk(bulkRequest, RequestOptions.DEFAULT);
    }

    /**
     * createIndexRequest
     *
     * @param indexName
     * @param object
     * @return
     */
    private IndexRequest createIndexRequest(String indexName, Object object) {
        Map<String, Object> dataMap = null;
        if (BaseEntity.class.isAssignableFrom(object.getClass())) {
            dataMap = JSON.parseObject(JSON.toJSONString(object), Map.class);
        } else if (object instanceof Map) {
            dataMap = (Map<String, Object>) object;
        } else {
            throw new RuntimeException("Neither Map nor subclass of
BaseEntity");
        }

        if (dataMap.containsKey(PK_ID) && dataMap.getOrDefault(PK_ID, null) !=
null) {
            return new IndexRequest(indexName)
                .id(dataMap.remove(PK_ID).toString())
                .source(dataMap, XContentType.JSON)

```

```

        .setRefreshPolicy(IMMEDIATE);
    } else {
        dataMap.remove(PK_ID);
        return new IndexRequest(indexName)
            .id(UUID.randomUUID().toString())
            .source(dataMap, XContentType.JSON)
            .setRefreshPolicy(IMMEDIATE);
    }
}

/**
 * getDocAsString
 *
 * @param indexName
 * @param id
 * @return
 * @throws IOException
 */
public String getDocAsString(String indexName, String id) throws IOException
{
    return JSON.toJSONString(getDocAsMap(indexName, id));
}

/**
 * getDocAsMap
 *
 * @param indexName
 * @param id
 * @return
 * @throws IOException
 */
public Map<String, Object> getDocAsMap(String indexName, String id) throws
IOException {
    GetRequest getRequest = new GetRequest(indexName, id);
    GetResponse response = highLevelClient.get(getRequest,
RequestOptions.DEFAULT);
    if (response != null && response.exists()) {
        Map<String, Object> map = response.getSourceAsMap();
        map.put(PK_ID, response.getId());
        return map;
    }
    return null;
}

/**
 * deleteDoc
 *
 * @param indexName
 * @param id
 * @return
 * @throws IOException
 */
public boolean deleteDoc(String indexName, String id) throws IOException {
    DeleteRequest request = new DeleteRequest(indexName, id);
    DeleteResponse deleteResponse = highLevelClient.delete(request,
RequestOptions.DEFAULT);
    return deleteResponse.getResult() == DocWriteResponse.Result.DELETED ||
deleteResponse.getResult() == DocWriteResponse.Result.NOT_FOUND;
}

```



```

    }

    /**
     * updateDoc
     *
     * @param indexName
     * @param id
     * @param dataMap
     * @return
     * @throws IOException
     */
    public boolean updateDoc(String indexName, String id, Map<String, Object>
dataMap) throws IOException {
        if (StringUtils.hasText(id)) {
            UpdateRequest request = new UpdateRequest(indexName,
id).doc(dataMap);
            UpdateResponse updateResponse = highLevelClient.update(request,
RequestOptions.DEFAULT);
            return updateResponse.getResult() ==
DocWriteResponse.Result.UPDATED;
        }
        throw new RuntimeException("primary key of the document is missing");
    }

    /**
     * updateDoc
     *
     * @param indexName
     * @param t
     * @param <T>
     * @return
     * @throws IOException
     */
    public <T extends BaseEntity> boolean updateDoc(String indexName, String id,
T t) throws IOException {
        Map<String, Object> dataMap = JSON.parseObject(JSON.toJSONString(t),
Map.class);
        return updateDoc(indexName, id, dataMap);
    }

    /**
     * updateDocsByQuery
     *
     * @param indexName
     * @param queryList
     * @return
     * @throws IOException
     */
    public BulkByScrollResponse updateDocsByQuery(String indexName,
List<ESQuery> queryList, Script script) throws IOException {
        return updateBatchByQuery(indexName, queryList, script);
    }

    /**
     * updateDocsByQuery
     *
     * @param indexName
     * @param esQuery

```

```

    * @return
    * @throws IOException
    */
    public BulkByScrollResponse updateDocsByQuery(String indexName, ESQuery
esQuery, Script script) throws IOException {
        return updateBatchByQuery(indexName, esQuery, script);
    }

    /**
     * updateBatchByQuery
     *
     * @param indexName
     * @param esQueryObject
     * @return
     * @throws IOException
     */
    private BulkByScrollResponse updateBatchByQuery(String indexName, Object
esQueryObject, Script script) throws IOException {

        if (script == null) {
            return null;
        }

        UpdateByQueryRequest updateByQueryRequest = new
UpdateByQueryRequest(indexName);
        SearchSourceBuilder searchSourceBuilder = new SearchSourceBuilder();

        if (esQueryObject != null) {
            if (esQueryObject instanceof ESQuery) {
                querySetting(searchSourceBuilder, (ESQuery) esQueryObject);
            } else {
                querySetting(searchSourceBuilder, (List<ESQuery>)
esQueryObject);
            }
        }

        updateByQueryRequest.setQuery(searchSourceBuilder.query());
        updateByQueryRequest.setScript(script);
        return highLevelClient.updateByQuery(updateByQueryRequest,
RequestOptions.DEFAULT);
    }

    /**
     * searchAllAsMap
     *
     * @param indexName
     * @return
     * @throws IOException
     */
    public List<Map<String, Object>> searchAllAsMap(String indexName) throws
IOException {
        return searchAllAsMap(indexName, MAX_COUNT_PER_PAGE);
    }

    /**
     * searchAllAsMap
     *
     * @param indexName

```

```

    * @return
    * @throws IOException
    */
    public List<Map<String, Object>> searchAllAsMap(String indexName, int count)
    throws IOException {
        SearchHits searchHits = searchAll(indexName, new ESPage<>(1,
count)).getHits();
        List<Map<String, Object>> dataList = new ArrayList<>();
        for (SearchHit hit : searchHits) {
            Map<String, Object> map = hit.getSourceAsMap();
            map.put(PK_ID, hit.getId());
            dataList.add(map);
        }
        return dataList;
    }

    /**
     * searchAllAsString
     *
     * @param indexName
     * @return
     * @throws IOException
     */
    public List<String> searchAllAsString(String indexName) throws IOException {
        return searchAllAsString(indexName, MAX_COUNT_PER_PAGE);
    }

    /**
     * searchAllAsString
     *
     * @param indexName
     * @return
     * @throws IOException
     */
    public List<String> searchAllAsString(String indexName, int count) throws
IOException {
        SearchHits searchHits = searchAll(indexName, new ESPage<>(1,
count)).getHits();
        List<String> dataList = new ArrayList<>();
        for (SearchHit hit : searchHits) {
            JSONObject map = JSON.parseObject(hit.getSourceAsString());
            map.put(PK_ID, hit.getId());
            dataList.add(map.toJSONString());
        }
        return dataList;
    }

    /**
     * searchAllAsString
     *
     * @param indexName
     * @return
     * @throws IOException
     */
    private SearchResponse searchAll(String indexName, ESPage<?> ESPage) throws
IOException {
        SearchRequest searchRequest = new SearchRequest(indexName);
        SearchSourceBuilder searchSourceBuilder = new SearchSourceBuilder();

```

```

        searchSourceBuilder.query(QueryBuilders.matchAllQuery());
        pageSetting(searchSourceBuilder, ESPage);
        searchRequest.source(searchSourceBuilder);
        return highLevelClient.search(searchRequest, RequestOptions.DEFAULT);
    }

    /**
     * orderSetting
     *
     * @param searchSourceBuilder
     * @param sortList
     */
    private void sortSetting(SearchSourceBuilder searchSourceBuilder,
        List<ESSort> sortList) {
        if (sortList != null && !sortList.isEmpty()) {
            List<SortBuilder<?>> sortBuilderList = new ArrayList<>();
            for (ESSort esSort : sortList) {
                if (StringUtils.hasText(esSort.getFieldName()) &&
                    esSort.getSortOrder() != null) {
                    sortBuilderList.add(new
                        FieldSortBuilder(esSort.getFieldName()).order(esSort.getSortOrder()));
                }
            }
            searchSourceBuilder.sort(sortBuilderList);
        } else {
            searchSourceBuilder.sort(new
                ScoreSortBuilder().order(SortOrder.DESC));
        }
    }

    /**
     * querySetting
     *
     * @param searchSourceBuilder
     * @param queryList
     */
    private void querySetting(SearchSourceBuilder searchSourceBuilder,
        List<ESQuery> queryList) {
        if (queryList != null && !queryList.isEmpty()) {
            BoolQueryBuilder boolQueryBuilder = QueryBuilders.boolQuery();
            searchSourceBuilder.query(buildBoolQueryBuilder(boolQueryBuilder,
                queryList));
        }
    }

    /**
     * querySetting
     *
     * @param searchSourceBuilder
     * @param esQuery
     */
    private void querySetting(SearchSourceBuilder searchSourceBuilder, ESQuery
        esQuery) {
        if (esQuery != null) {
            BoolQueryBuilder boolQueryBuilder = QueryBuilders.boolQuery();
            searchSourceBuilder.query(buildBoolQueryBuilder(boolQueryBuilder,
                esQuery));
        }
    }

```

```

    }

    /**
     * buildBoolQueryBuilder
     *
     * @param boolQueryBuilder
     * @param queryList
     * @return
     */
    private BoolQueryBuilder buildBoolQueryBuilder(BoolQueryBuilder
boolQueryBuilder, List<ESQuery> queryList) {
        if (queryList != null && !queryList.isEmpty()) {
            for (ESQuery esQuery : queryList) {
                buildBoolQueryBuilder(boolQueryBuilder, esQuery);
            }
        }
        return boolQueryBuilder;
    }

    /**
     * buildBoolQueryBuilder
     *
     * @param boolQueryBuilder
     * @param esQuery
     * @return
     */
    private BoolQueryBuilder buildBoolQueryBuilder(BoolQueryBuilder
boolQueryBuilder, ESQuery esQuery) {
        if (StringUtils.hasText(esQuery.getFieldName())) {
            if (ESQuery.QueryType.MUST == esQuery.getQueryType()) {

                boolQueryBuilder.must(QueryBuilders.matchQuery(esQuery.getFieldName(),
esQuery.getQueryParam()));
            } else if (ESQuery.QueryType.MUST_NOT == esQuery.getQueryType()) {

                boolQueryBuilder.mustNot(QueryBuilders.matchQuery(esQuery.getFieldName(),
esQuery.getQueryParam()));
            } else {

                boolQueryBuilder.should(QueryBuilders.matchQuery(esQuery.getFieldName(),
esQuery.getQueryParam()));
            }
        }
        return boolQueryBuilder;
    }

    /**
     * pageSetting
     *
     * @param searchSourceBuilder
     * @param ESPage
     */
    private void pageSetting(SearchSourceBuilder searchSourceBuilder, ESPage<?>
ESPage) {
        if (ESPage == null) {
            ESPage = new ESPage<>();
        }
    }

```

```

        searchSourceBuilder.from((ESPage.getCurrPage() - 1) *
ESPage.getPageSize());
        searchSourceBuilder.size(ESPage.getPageSize());
    }

    /**
     * searchPage
     *
     * @param indexName
     * @param queryList
     * @param sortList
     * @param ESPage
     * @return
     * @throws IOException
     */
    public SearchResponse searchPage(String indexName, List<ESQuery> queryList,
List<ESSort> sortList, ESPage<?> ESPage) throws IOException {
        return searchPage(indexName, queryList, sortList, null, null, ESPage);
    }

    /**
     * searchPage
     *
     * @param indexName
     * @param queryList
     * @param sortList
     * @param includeFields
     * @param excludeFields
     * @param ESPage
     * @return
     * @throws IOException
     */
    public SearchResponse searchPage(String indexName, List<ESQuery> queryList,
List<ESSort> sortList, String[] includeFields, String[] excludeFields, ESPage<?>
ESPage) throws IOException {
        SearchRequest searchRequest = new SearchRequest(indexName);
        SearchSourceBuilder searchSourceBuilder = new SearchSourceBuilder();
        pageSetting(searchSourceBuilder, ESPage);
        querySetting(searchSourceBuilder, queryList);
        sortSetting(searchSourceBuilder, sortList);
        searchSourceBuilder.fetchSource(true);
        searchSourceBuilder.fetchSource(includeFields, excludeFields);
        searchRequest.source(searchSourceBuilder);
        return highLevelClient.search(searchRequest, RequestOptions.DEFAULT);
    }

    /**
     * searchPageAsString
     *
     * @param indexName
     * @param queryList
     * @param sortList
     * @return
     * @throws IOException
     */
    public ESPage<String> searchPageAsString(String indexName, List<ESQuery>
queryList, List<ESSort> sortList, ESPage<String> ESPage) throws IOException {

```

```

        return searchPageAsString(indexName, queryList, sortList, null, null,
        ESPage);
    }

    /**
     * searchPageAsString
     *
     * @param indexName
     * @param queryList
     * @param sortList
     * @param includeFields
     * @param excludeFields
     * @param ESPage
     * @return
     * @throws IOException
     */
    public ESPage<String> searchPageAsString(String indexName, List<ESQuery>
    queryList, List<ESSort> sortList, String[] includeFields, String[]
    excludeFields, ESPage<String> ESPage) throws IOException {
        if (ESPage == null) {
            ESPage = new ESPage<>();
        }
        SearchHits searchHits = searchPage(indexName, queryList, sortList,
    includeFields, excludeFields, ESPage).getHits();
        List<String> dataList = new ArrayList<>();
        for (SearchHit hit : searchHits) {
            dataList.add(hit.getSourceAsString());
        }
        return ESPage.rebuild(new
    Long(searchHits.getTotalHits().value).intValue(), dataList);
    }

    /**
     * searchPageAsMap
     *
     * @param indexName
     * @param queryList
     * @param sortList
     * @param ESPage
     * @return
     * @throws IOException
     */
    public ESPage<Map<String, Object>> searchPageAsMap(String indexName,
    List<ESQuery> queryList, List<ESSort> sortList, ESPage<Map<String, Object>>
    ESPage) throws IOException {
        return searchPageAsMap(indexName, queryList, sortList, null, null,
    ESPage);
    }

    /**
     * searchPageAsMap
     *
     * @param indexName
     * @param queryList
     * @param sortList
     * @param includeFields
     * @param excludeFields
     * @param ESPage

```

```

    * @return
    * @throws IOException
    */
    public ESPage<Map<String, Object>> searchPageAsMap(String indexName,
List<ESQuery> queryList, List<ESSort> sortList, String[] includeFields, String[]
excludeFields, ESPage<Map<String, Object>> ESPage) throws IOException {
        if (ESPage == null) {
            ESPage = new ESPage<>();
        }
        SearchHits searchHits = searchPage(indexName, queryList, sortList,
includeFields, excludeFields, ESPage).getHits();
        List<Map<String, Object>> dataList = new ArrayList<>();
        for (SearchHit hit : searchHits) {
            dataList.add(hit.getSourceAsMap());
        }
        return ESPage.rebuild(new
Long(searchHits.getTotalHits().value).intValue(), dataList);
    }
}

```

##

6 ES集群架构

6.1 ES集群的5大角色

1. Master Node : 主节点

主节点，该节点不和应用创建连接，每个节点都保存了集群状态。master节点控制整个集群的元数据。只有Master Node节点可以修改节点状态信息，及元数据(metadata)的处理。元数据(metadata)，比如：

- 索引的分片信息、主副本信息
- 分片的节点分配信息，路由分配
- index、type、Mapping
- Setting 配置等等。

从资源占用的角度来说：master节点不占用磁盘IO和CPU，内存使用量一般，没有data节点高类似于kafa中的 controller，负责集群元数据的管理和维护

2. Master eligible nodes : 合格节点

有资格成为Master节点但暂时并不是Master的节点被称为 eligible 节点，该节点可以参加选主流程，成为Master节点。

该节点只是与集群保持心跳，判断Master是否存活，如果Master故障则参加新一轮的Master选举。

从资源占用的角度来说：eligible节点比Master节点更节省资源，因为它还未成为 Master 节点，只是有资格成功Master节点。

3. Data Node : 数据节点

职责：数据节点，用于建立文档索引，管理shard。类似于rocket 中的 broker，负责数据的管理和维护，数据节点职责：

- 该节点用于建立文档索引，接收应用创建连接、接收索引请求
- 查询，接收用户的搜索请求

data节点的分片执行查询语句获得查询结果后将结果反馈给Coordinating节点，在查询的过程中非常消耗硬件资源，如果在分片配置及优化没做好的情况下，进行一次查询非常缓慢(硬件配置也要跟上数据量)。

数据节点：保存包含索引文档的分片数据，执行CRUD、搜索、聚合相关的操作。属于：内存、CPU、IO密集型，对硬件资源要求高。

从资源占用的角度来说：data节点会占用大量的CPU、IO和内存

4. Coordinating Node：协调节点(/路由节点/client节点)

协调节点，该节点专用与接收应用的查询连接、接受搜索请求，但其本身不负责存储数据

协调节点，接受客户端搜索请求后将请求转发到与查询条件相关的多个data节点的分片上，然后多个data节点的分片执行查询语句或者查询结果再返回给协调节点，协调节点把各个data节点的返回结果进行整合、排序等一系列操作后再将最终结果返回给用户请求。

data节点的分片执行查询语句获得查询结果后将结果反馈给Coordinating节点，在查询的过程中非常消耗硬件资源，如果在分片配置及优化没做好的情况下，进行一次查询非常缓慢(硬件配置也要跟上数据量)。

搜索请求在两个阶段中执行（query 和 fetch），这两个阶段由接收客户端请求的节点 - 协调节点协调。

- 在请求query 阶段，协调节点将请求转发到保存数据的数据节点。每个数据节点在本地执行请求并将其结果返回给协调节点。
- 在收集fetch阶段，协调节点将每个数据节点的结果汇集为单个全局结果集。

从资源占用的角度来说：协调节点，可当负责均衡节点，该节点不占用io、cpu和内存

总结：Coordinating 大致的职责：请求分发、结果的合并

5. Ingest Node：ingest节点

ingest 节点可以看作是数据前置处理转换的节点，支持 pipeline管道 设置，可以使用 ingest 对数据进行过滤、转换等操作，类似于 logstash 中 filter 的作用，功能相当强大。

Ingest节点处理时机——在数据被索引之前，通过预定义好的处理管道对数据进行预处理。默认情况下，所有节点都启用Ingest，因此任何节点都可以处理Ingest任务。

当然，我们也可以创建专用的Ingest节点。

6. 部落 (tribe)

接着说一下ES里面的部落：tribe，

可以在查询过程中链接两个集群的数据，查询的数据将会汇总到tribe节点，有tribe节点对数据进行整合再发送给client；

tribe还可以写数据，但是这里有一个限制，就是写的索引只能是一个集群所有；如果写入两个集群同名索引，那么只能成功写入一个，至于写入哪一个可以通过配置偏好实现。

可以通过配置指明tribe只能读，不能写。

6.2 ES集群节点角色配置

1. node.master
2. node.data
3. node.ingest

配置实例/usr/share/elasticsearch/config/elasticsearch.yml

```
bootstrap.memory_lock: true
```

```
cluster.name: "es-cluster"
node.name: master1
node.master: true
node.data: true
network.host: 0.0.0.0
http.port: 9200
transport.tcp.port: 9300
cluster.initial_master_nodes: ["master1"]
discovery.zen.ping.unicast.hosts: master1, master2, master3
#官方推荐 master-eligible nodes / 2 + 1 向下取整的个数
discovery.zen.minimum_master_nodes: 2
path.logs: /usr/share/elasticsearch/logs
http.cors.enabled: true
http.cors.allow-origin: "*"
xpack.security.audit.enabled: true
```

默认情况下这三个属性的值都是true，实际上，一个节点在默认情况下会同时扮演：Master Node，Data Node 和 Ingest Node。

节点类型	配置参数	默认值
Master Eligible	node.master	true
Data	node.data	true
Coordinating only	无	设置上面2 个参数全为 false，节点为专用协调节点
Ingest	node.ingest	true

6.3 ES节点配置组合

1. 组合1

```
node.master: true
node.data: true
node.ingest: true
```

这种组合表示这个节点既有成为主节点的资格，又可以存储数据，还可以作为预处理节点这个时候如果某个节点被选举成为了真正的主节点，那么他还要存储数据，这样对于这个节点的压力就比较大了。

elasticsearch 默认是：每个节点都是这样的配置，在测试环境下这样做没问题。实际工作中建议不要这

样设置，这样相当于 主节点 和 数据节点 的角色混合到一块了。

2. 组合2

```
node.master: false
node.data: true
node.ingest: false
```

这种组合表示这个节点没有成为主节点的资格，也就不参与选举，只会存储数据。这个节点我们称为 data(数据)节点。在集群中需要单独设置几个这样的节点负责存储数据。后期提供存储和查询服务

3. 组合3

```
node.master: true
node.data: false
node.ingest: false
```

这种组合表示这个节点不会存储数据，有成为主节点的资格，可以参与选举，有可能成为真正的主节点。这个节点我们称为master节点

4. 组合4

```
node.master: false
node.data: false
node.ingest: true
```

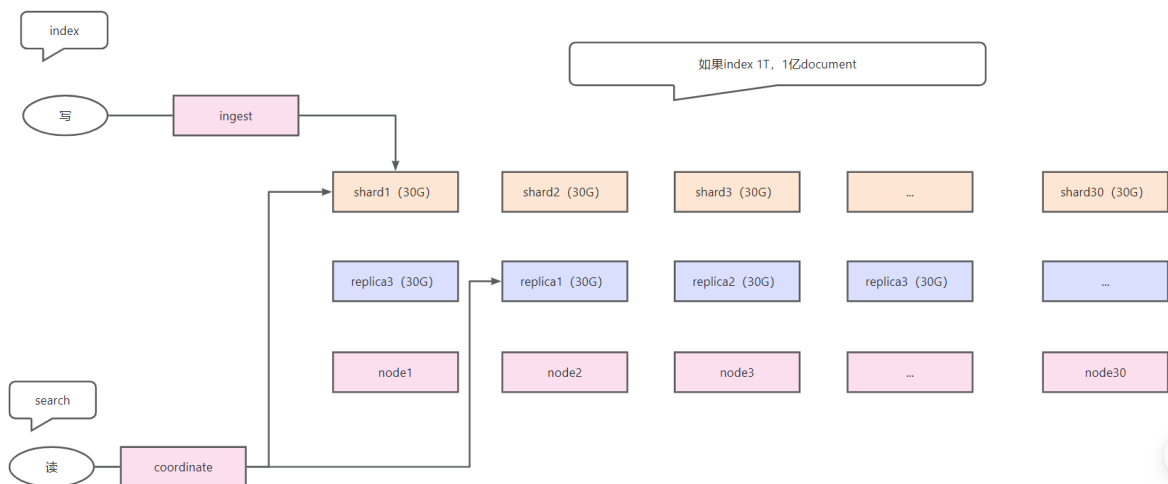
这种组合表示这个节点即不会成为主节点，也不会存储数据，这个节点的意义是作为一个 client(客户端)节点，主要是针对海量请求的时候可以进行负载均衡。

在新版 ElasticSearch5.x 之后该节点称之为: coordinate 节点, 其中还增加了一个叫: ingest 节点, 用

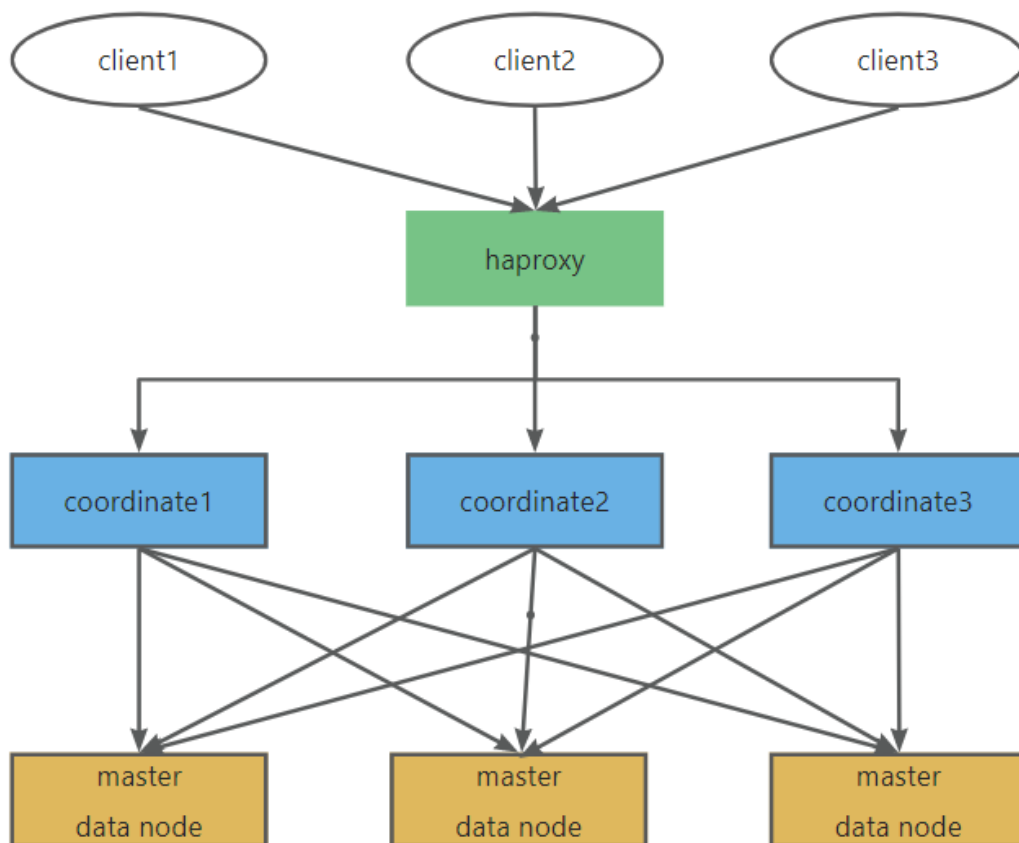
于预处理数据（索引和搜索阶段都可以用到）。

当然，作为一般应用是不需要这个预处理节点做什么额外的预处理过程，那么这个节点和我们称之为 client 节点之间可以看做是等同的，我们在代码中配置访问节点就都可以配置这些 ingest 节点即可。 \

6.4 高可用ES的部署架构



小型的ES集群 (<10) 的节点架构



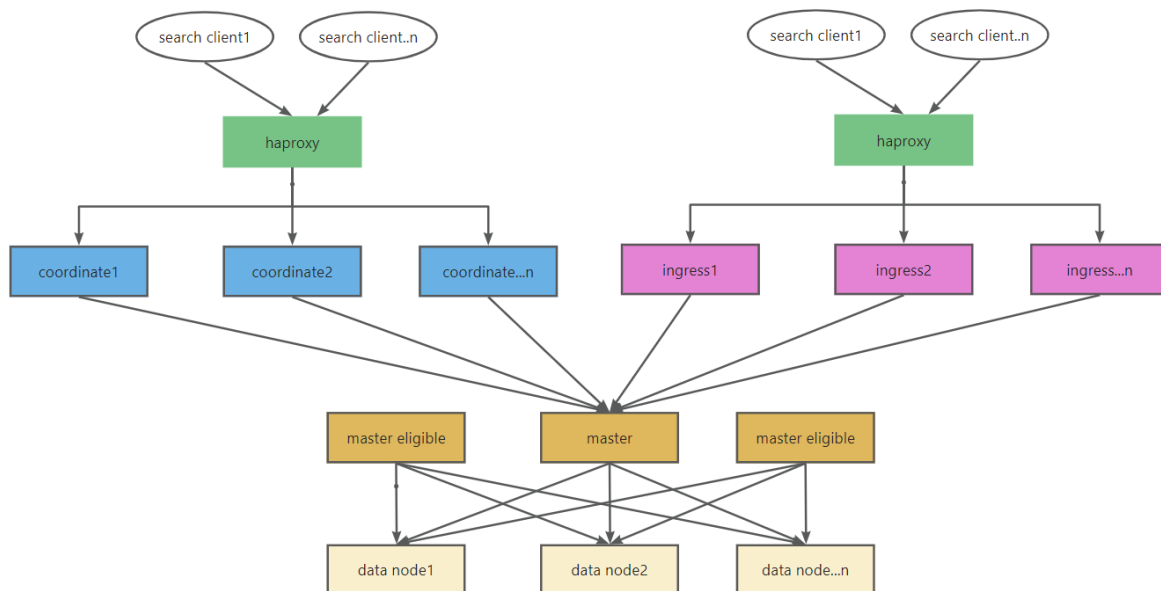
小型的ES集群，就是3/5/7这种少于10个节点的集群。对于3个节点、5个节点甚至更多节点角色的配置，3个主节点+数据节点，2个协调节点即可。

小型的ES集群的节点角色规划：

- 对于Ingest转换节点，如果我们没有格式转换、类型转换等需求，直接设置为false。
- 3-5个节点属于轻量级集群，要保证主节点个数满足 $((\text{节点数}/2)+1)$ 。
- 轻量级集群，节点的多重属性如：Master&Data设置为同一个节点可以理解的。
- 如果进一步优化，5节点可以将Master和Data再分离。

中型的ES集群（10-50）的节点架构

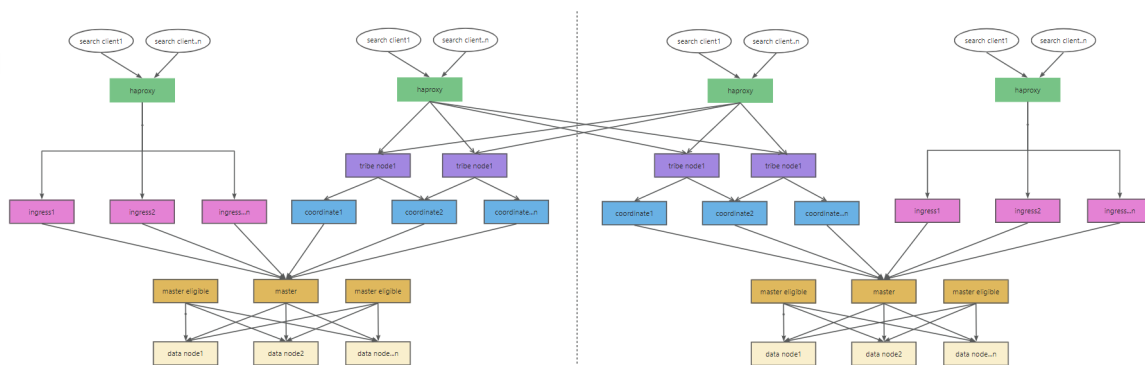
- 三台服务器做master节点（可选）
- N（比如20）台服务器作为data节点（存储资源要大）
- N（比如5）台做coordinate/ingest节点（用于搜索结果合并，可以提高ES查询效率）



超大型的ES集群的节点架构 (150个节点+)

可以按照100个节点为单位，分成多个集群，通过 tribenode连接，单个ES数据库最好的高可用集群部署架构为：每个集群，三台服务器做master节点

- N (比如50) 台服务器作为data节点 (存储资源要大)
- N (比如5) 台做coordinate节点 (用于搜索结果合并，可以提高ES查询效率)
- N (比如2) 台做ingest节点 (用于数据转换，可以提高ES索引效率)



6.5 小型ES集群的安装

6.5.1 Image镜像

有外网环境，拉取镜像代码如下：

```
#下载elasticsearch,带中文分词器的版本
docker pull andylsr/elasticsearch-with-ik-icu:7.14.0
#下载kibana
docker pull docker.elastic.co/kibana/kibana:7.14.0
```

无外网环境，可以先从有公网的环境拉取镜像，然后导出镜像

```
docker save andylsr/elasticsearch-with-ik-icu:7.14.0 -o /root/elasticsearchwith-ik-icu.tar
docker save docker.elastic.co/kibana/kibana:7.14.0 -o /root/kibana.tar
```

然后上传导出的de镜像到dao目标虚拟机，然后导入镜像到docker

```
docker load -i /vagrant/3G-middleware/elasticsearch-with-ik-icu.tar
docker load -i /vagrant/3G-middleware/kibana.tar
docker load -i /vagrant/3G-middleware/haproxy.tar
```

6.5.2 创建目录结构

```
mkdir -p /home/docker-compose/elasticsearch7/{coordinate1,coordinate2}-{logs,data}
mkdir -p /home/docker-compose/elasticsearch7/{master1,master2,master3}-{logs,data}
chmod -R 777 /home/docker-compose/elasticsearch7
```

6.5.3 安装ES集群

把docker-compose和ES相应配置文件拷贝到 /home/docker-compose 目录下，执行如下命令安装ES集群

```
docker-compose --compatibility up -d # 兼容模式后台启动
```

- --compatibility: 表示已兼容模式启动容器
- -d: 表示后台启动

6.6 使用kibana访问集群

1. 查看集群健康情况

```
GET _cat/health?v
```

结果:

epoch	timestamp	cluster	status	node.total	node.data	shards	pri	relo	init
unassign	pending_tasks	max_task_wait_time	active_shards_percent						
1670418184	13:03:04	es-cluster	green	5	3	16	8	0	0
0	0		-		100.0%				

status选项的值

- green : 所有primary shard和replica shard都已成功分配, 集群是100%可用的
- yellow : 所有primary shard都已成功分配, 但至少有一个replica shard缺失. 此时集群所有功能都正常使用, 数据不会丢失, 搜索结果依然完整, 但集群的可用性减弱. —— 需要及时处理的警告
- red : 至少有一个primary shard(以及它的全部副本分片)缺失 —— 部分数据不能使用, 搜索只能返回部分数据, 而分配到这个分配上的写入请求会返回一个异常. 此时虽然可以运行部分功能, 但为了索引数据的完整性, 需要尽快修复集群

2. 查看集群中的节点个数

```
GET _cat/nodes?v
```

ip	heap.percent	ram.percent	cpu	load_1m	load_5m	load_15m	node.role	
master name								
172.19.0.2	36	95	12	0.20	0.43	0.73	cdfhilmrstw	-
master2								
172.19.0.7	75	95	12	0.20	0.43	0.73	cdfhilmrstw	*
master3								
172.19.0.5	29	95	12	0.20	0.43	0.73	cdfhilmrstw	-
master1								
172.19.0.6	64	96	12	0.20	0.43	0.73	1r	-
coordinate2								
172.19.0.3	23	96	12	0.20	0.43	0.73	1r	-
coordinate1								

- 第一列 (ip) : es节点ip
- 第二列 (heap.percent) : 堆内存占比
- 第三列 (ram.percent) : 内存使用占比
- 第四列 (cpu) : cpu使用率
- 第五列 (load_1m) : 1分钟内平均load情况
- 第六列 (load_5m) : 5分钟内平均load情况
- 第七列 (load_15m) : 15分钟内平均load情况
- 第八列 (node.role) : 节点权限
- 第九列 (master) : 是否master节点, *为master节点
- 第十列 (name) : 节点名称

(1) heap.percent

表示ES使用的JVM内存情况, 该值应该低于75, 如果长时间大于75, 表示JVM内存配置不够, 如果JVM已经配置到30G, 则表示该Datanode节点上的压力较大, 需要考虑增加Datanode节点来分摊压力

(2) ram.percent

表示机器上内存的使用情况, 实际对应linux上的 used+cache内存使用情况, 如果该值接近100%, 则表示机器上cache内存不够用, 主要是由于ES检索中, lucene会消耗大量cache内存, 如果cache不够, 会导致lucene无法将部分文件加载到cache中, 会频繁从磁盘中进行读取, 导致查询延时加大

3. 查看集群中的索引

```
GET _cat/indices?v
```

health	status	index	uuid	pri	rep
docs.count	docs.deleted	store.size	pri.store.size		
green	open	.kibana-event-log-7.14.0-000001	1uw627v6T8KPBv2xr1P1PQ	1	1
9	0	97.8kb	48.9kb		
green	open	.kibana_7.14.0_001	1G2WGixOT2Wvux7R_5TuLA	1	1
86	14	4.9mb	2.7mb		
green	open	.apm-custom-link	dMukcWA2QVOFzsOhwiPong	1	1
0	0	416b	208b		
green	open	.apm-agent-configuration	ReYFH-LaQbmWjGTwif0nHA	1	1
0	0	416b	208b		
green	open	.kibana_task_manager_7.14.0_001	H-3hQBgzR02AfYSWLS6LQw	1	1
14	1275	622.9kb	337.1kb		
green	open	user	HdrFzj7TQ_ejXe_9xdx1LQ	1	1
6	2	37.4kb	17.4kb		
green	open	.tasks	DudUNrFYTq24R7ZY9IFK5A	1	1
16	0	80.9kb	43.4kb		

7 数据类型和映射

7.1 映射的创建

和传统数据库不同，传统的数据库我们尝试向表中插入数据的前提是这个表已经存在数据结构的定义，且插入数据的字段要在表结构中被定义。

而ES的映射的创建支持主动和被动创建。

1. 被动创建（动态映射）

此时字段和映射类型不需要事先定义，只需要存在文档的索引，当向此索引添加数据的时候当遇到不存在的映射字段，ES会根据数据内容自动添加映射字段定义。

2. 主动创建（显示映射）

动态映射只能保证最基础的数据结构的映射，所以很多时候我们需要对字段除了数据结构定义更多的限制的时候，动态映射创建的内容很可能不符合我们的需求，所以可以使用 PUT {index}/mapping 来更新指定索引的映射内容。

7.2 动态映射Dynamic Mapping

写入文档的时候，索引不存在，会自动创建索引，无需手动创建，ES会根据内容推断字段的类型，推断会不准确，可能造成某些功能无法使用，例如 范围查询。

```
POST /log2/_doc/1
{
  "uid" : 1,
  "ip" : "192.1.1.1",
  "transTime" : "2018-01-01",
  "content" : "中华人民共和国人民大会堂"
}
```

查看一个索引当前的mapping

GET /log2/_mapping

结果如下

```
{
  "log2": {
    "mappings": {
      "properties": {
        "content": {
          "type": "text",
          "fields": {
            "keyword": {
              "type": "keyword",
              "ignore_above": 256
            }
          }
        },
        "ip": {
          "type": "text",
          "fields": {
            "keyword": {
              "type": "keyword",
              "ignore_above": 256
            }
          }
        },
        "transTime": {
          "type": "date"
        },
        "uid": {
          "type": "long"
        }
      }
    }
  }
}
```

动态映射规则

JSON中数据类型	Elasticsearch 数据类型
null	不添加任何字段
true或者false	boolean类型
浮点数据	float类型
integer数据	long类型
object	object类型
array	取决于数组中的第一个非空值的类型。
string	匹配日期格式，设置为date；匹配数字，设置为float或者long，功能默认关闭；设置为text，并增加keyword子字段。

7.3 显示的设置mapping

显示的设置mapping可以更灵活控制ES。

映射创建时，除了对字段的定义，Mapping创建的时候提供了一些对于查询策略和自身定义的参数配置。

下面只是简单介绍下映射支持的字段参数内容。

参数	说明
analyzer	定义此字段索引时使用的分词方式
normalizer	normalizer功能类似于analyzer，但是其可以使查询条件输出唯一的查询条件（可以认为其只是实现了条件小写等不会产生多个查询条件的相关操作）
boost	定义当前字段的查询权重
coerce	此字段控制是否尝试修复部分错误的数据格式，（比如对一个整数字段插入字符串比如"5"，此时此字符串可以被解析为数字），默认为true
copy_to	类似于别名，不同之处参数可以将此字段内容复制到指定字段中，多个字段可以复制到同一个字段中
doc_values	倒排索引虽然可以快速查询文档中内容，但是在进行排序或聚合操作的时候，倒排索引并不能获得文档内容，所以需要存储一份文档数据到doc_values，而此参数控制字段是否需要存储在doc_values中的开关。
dynamic	是否开启动态映射，目前支持三个参数：true/false 开启和关闭，strict 当出现未定义的字段，抛出异常并拒绝添加文档
enabled	此参数控制字段是否可以被索引，当被设置为false的时候表示此字段仅用来存储而无需索引，此时ES不会分析此字段内的数据，所以即使插入的非法的数据内容ES依旧允许执行
fielddata	类似doc_values都是单独存储额外的文档数据，这样通过倒排索引获取文档内容，从而实现在排序和聚合上的功能。不同的是doc_values不支持text格式，text格式数据需要使用fielddata。此参数默认是禁止的，这是因为在第一次对字段进行排序或聚合的时候它会把这个列数据都加载到内存中，这样会带来大量的内存消耗。
eager_global_ordinals	是否使用全局序号来进行聚合。主要在聚合分析构建hash的时候，使用序号来替代doc的值，这样在文档收集阶段根据需要收集到各个桶中，在计算结果时将序号转换为具体doc内容。但是此操作在每次查询时需要重建doc序号关系
format	日期类型字段用来解析的日期格式
ignore_above	当插入字段长度超过此字段设置的值后，此内容将不被索引或存储。对于数组结构字段会作用到每一个元素
ignore_malformed	当向一个字段插入错误的数据类似时，会抛出异常并拒绝文档。但设置此参数后，对字段插入错误的数据时会忽略异常，此文档错误的数据将不被索引，但是其他字段则正常。
index_options	控制将哪些信息添加到反向索引中
index_phrases	主要将两个单词的组合索引到单独字段中，这样在进行精确的短语查询的时候会更有效。支持true和false参数。默认为false。此参数会使索引变大
index_prefixes	允许对字段的前缀进行索引，此参数用来提高查询的速度

参数	说明
index	控制字段是否可以被索引，被设置为false的字段无法被索引到
fields	此参数可以为同一个字段设置不同的索引方式，但是在_source字段中只会保存一份，并不会实际增加存储。但是会增加索引大小
norms	norms里面存储的是各种各样的归一化因子，此内容会影响到文档的得分，在不需要对字段进行打分的时候可以禁用此参数，需要注意的是对于keyword字段默认为false
null_value	一般来说空值是无法被索引的，但是此参数允许使用指定的值替换空值，以对其进行索引
position_increment_gap	增加近似值匹配
properties	定义类型映射、对象字段和嵌套字段等数据
search_analyzer	定义此字段查询时使用的分词方式
similarity	此参数可以配置用来计算字段相似性的算法
store	默认情况下字段内容会被索引但是并不会存储字段中的值，想获取字段中的值则需要在source中获取对应字段的数据，当查询仅仅是尝试获取指定字段的内容的时候，可以设置此参数为true，那么系统可以直接获取此字段的内容，不再尝试获取source中的数据。
term_vector	术语向量的定义，存储一些术语向量，以便可以为特定文档检索它们
index	控制字段是否可以被索引，被设置为false的字段无法被索引到
fields	此参数可以为同一个字段设置不同的索引方式，但是在_source字段中只会保存一份，并不会实际增加存储。但是会增加索引大小
norms	norms里面存储的是各种各样的归一化因子，此内容会影响到文档的得分，在不需要对字段进行打分的时候可以禁用此参数，需要注意的是对于keyword字段默认为false
null_value	一般来说空值是无法被索引的，但是此参数允许使用指定的值替换空值，以对其进行索引
position_increment_gap	增加近似值匹配
properties	定义类型映射、对象字段和嵌套字段等数据
search_analyzer	定义此字段查询时使用的分词方式
similarity	此参数可以配置用来计算字段相似性的算法
store	默认情况下字段内容会被索引但是并不会存储字段中的值，想获取字段中的值则需要在source中获取对应字段的数据，当查询仅仅是尝试获取指定字段的内容的时候，可以设置此参数为true，那么系统可以直接获取此字段的内容，不再尝试获取source中的数据。

1. index

表示字段是否索引。

2. index_options

index_options 控制倒排索引记录的内容，一共有4种配置可选。

index_options	含义
docs	只记录文档id (doc id)
freqs	记录 doc id 和 term frequencies
positions	doc id 、 term frequencies 、 term position

| offsets | doc id 、 term frequencies 、 term position 、 character offsets |

文本类型 text 默认的配置是 positions ，其他默认是docs。

需要注意的是，虽然index_options提供了offsets这种内容较多的配置级别，但是记录的内容越多，占用的空间也会越多，在实际操作中还是要根据实际情况进行配置。

例如创建mapping,字段名为user_name，字符串类型。user_name不需要索引,info字段的倒排索引类型为positions。

```
PUT mapping_test3
{
  "mappings": {
    "properties": {
      "user_name": {
        "index": false,
        "type": "text"
      },
      "info": {
        "index_options": "positions",
        "type": "text"
      },
      "doc": {
        "type": "text",
        "index_options": "docs"
      },
      "freq": {
        "type": "text",
        "index_options": "freqs"
      },
      "offset": {
        "type": "text",
        "index_options": "offsets"
      }
    }
  }
}
```

3. ANALYZER

分词器。es有内置的分词器，也可以使用第三方的分词工具。如IK。

```
{
  "mappings": {
    "my_type": {
      "properties": {
        "content": {
```

```

        "type": "text",
        "analyzer": "ik_max_word", //写入是的分词器
        "search_analyzer": "ik_max_word" //搜索时的分词器
    }
}
}
}
}

```

4. COPY_TO

允许将一个或者多个字段的值复制到某一个字段中。用来满足一些搜索需要，类似于数据库 title like "%a%" or title2 like "%a%"

copy_to的字段不会出现在_source里面

```

{
  "mappings": {
    "my_type": {
      "properties": {
        "first_name": {
          "type": "text",
          "copy_to": "full_name"
        },
        "last_name": {
          "type": "text",
          "copy_to": "full_name"
        },
        "full_name": {
          "type": "text"
        }
      }
    }
  }
}

PUT my_index/my_type/1
{
  "first_name": "John",
  "last_name": "Smith"
}
//full_name = ["John","Smith"]

```

5. DOC_VALUES

为了加快排序、聚合操作，在建立倒排索引的时候，额外增加一个列式存储映射，是一个空间换时间的做法。

默认是开启的，对于确定不需要聚合或者排序的字段可以关闭。

注意：text类型没有doc_values。

```

{
  "mappings": {
    "my_type": {
      "properties": {
        "status_code": {

```



```
{
  "mappings": {
    "my_type": {
      "properties": {
        "city": {
          "type": "text",
          "fields": {
            "raw": {
              "type": "keyword"
            }
          }
        }
      }
    }
  }
}
```

11. STORE

我们知道，source字段存储了原始数据(默认)。当然可以通过设置其属性值来选择不存储。此外，还可以通过store选择是否额外存储某个字段。store属性默认为no,表示不存储。当设置为yes时，会在source之

外独立存储。此时，搜索时，会绕过_source，单独进行一次IO得到该字段的值。

store会严重影响搜索效率，尽管如此，在以下两种情况下，还是可以选择使用：

- 字段很长，每次检索_source代价很大。
- 需要单独对某些字段进行索引重建。

7.4 ES中常见数据类型

1. 字符串

字符串在之前的版本主要指的是 string 类型。但是在5.X版本已经不支持 string 类型。其被 text 和 keyword 类型替代

2. text

text字段需要被全文搜索的内容，它可以保存非常长的内容。查询的时候一般使用分词器进行行行分词然后进行全文搜索。text类型的字段不用于排序，很少用于聚合。

(text类型的数据被用来索引长文本，例如电子邮件主体部分或者一款产品的介绍，这些文本会被分析，在建立索引文档之前会被分词器进行分词，转化为词组。经过分词机制之后es允许检索到该文本切分而成的词语，但是text类型的数据不能用来过滤、排序和聚合等操作。

3. keyword

此字段不能使用分词器进行查询，只能搜索该字段的完整的值。所以其主要保存一些可以索引的结构化内容。此字段可以进行排序、聚合等操作。

keyword类型的数据可以满足电子邮箱地址、主机名、状态码、邮政编码和标签等数据的要求，不进行分词，常常被用来过滤、排序和聚合。

综上，可以发现text类型在存储数据的时候会默认进行分词，并生成索引。而keyword存储数据的时候，不会分词建立索引，显然，这样划分数据更加节省内存。)

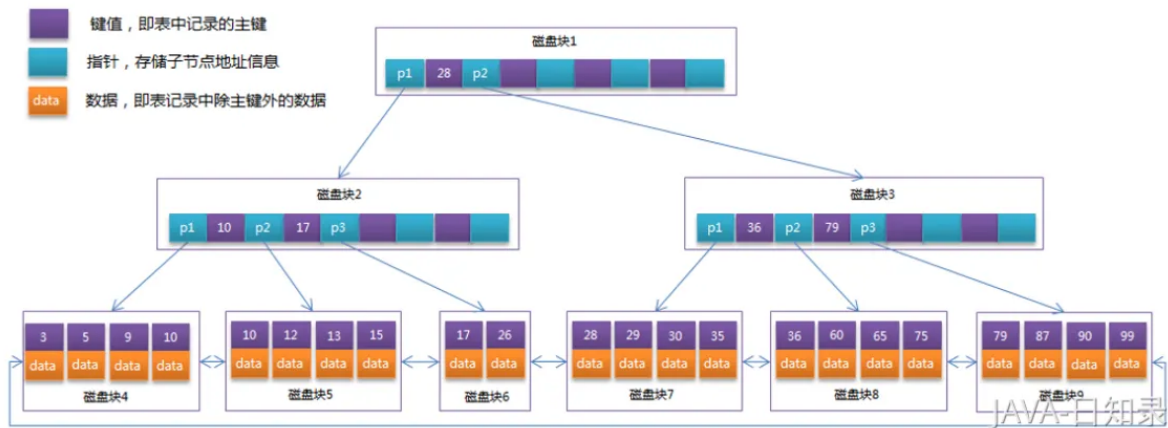
8 底层知识：正排索引和倒排索引底层原理

8.1 什么是正排索引

正排索引是按照文档编号或文档ID等有序的方式将每个文档存储在索引中，通过文档编号或ID进行检索。

这种方式类似于数据库表的行，可以很方便地根据文档ID检索到具体的文档，但是不适合处理大规模文档库的情况。

比如mysql的b+锁索引结构



比如书籍目录，可以根据页码找文档内容，就是正排索引

目录 查看章节

编程规约	1
(一) 命名风格	1
(二) 常量定义	4
(三) 代码格式	5
(四) OOP 规约	7
(五) 日期时间	11
(六) 集合处理	12
(七) 并发处理	17

指定页码

8.2 什么是倒排索引

倒排索引是按照单词或关键字将文档进行索引，并记录包含该词汇的文档列表。这种方式类似于数据库表的列，可以将具有相同属性的文档按照关键词进行分类，从而实现更加高效和精确的文本搜索。

倒排索引可以理解为Map< item, list< id>>, 能够由查询词快速（时间复杂度O(1)）找到包含这个查询词的文件的的数据结构。

正排索引

文档 ID	文档内容
1	elasticsearch 是最流行的搜索引擎
2	php 是世界上最好的语言
3	搜索引擎是如何诞生的

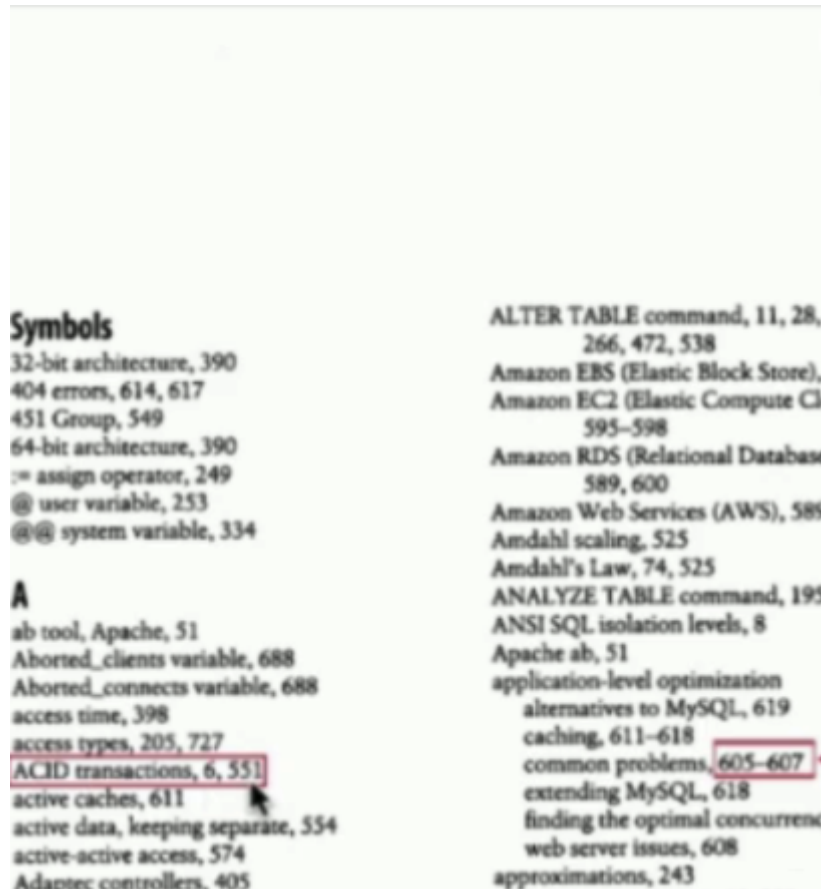


倒排索引

单词	文档 ID 列表
elasticsearch	1
流行	1
搜索引擎	1、3
php	2
世界	2
最好	2
语言	2
如何	3
诞生	3

https://blog.csdn.net/weixin_38718516

比如书籍索引页根据关键词，找页码就是倒排索引



8.3 ES如何做到快速索引

**假设有这么几条数据

```
| ID | Name | Age | Sex |
| -- | :-----: | ----: | ----: |
| 1 | Kate | 24 | Female
| 2 | John | 24 | Male
| 3 | Bill | 29 | Male
```

ID是Elasticsearch自建的文档id，那么Elasticsearch建立的索引如下：

Name：

```
| Term | Posting List |
| -- | :----: |
| Kate | 1 |
| John | 2 |
| Bill | 3 |
```

Age:

```
| Term | Posting List |
| -- | :----: |
| 24 | [1,2] |
| 29 | 3 |
```

8.3.1 Posting List

Elasticsearch分别为每个field都建立了一个倒排索引，

- Kate, John, 24, Female这些叫term，
- 而[1,2]就是Posting List。

Posting list就是一个int的数组，存储了所有符合某个term的文档id。

根据id查找的话，通过posting list这种索引方式似乎可以很快进行查找，比如要找age=24的同学，在id数组中查找即可。二分查找，但是，如果想通过name来查找呢？

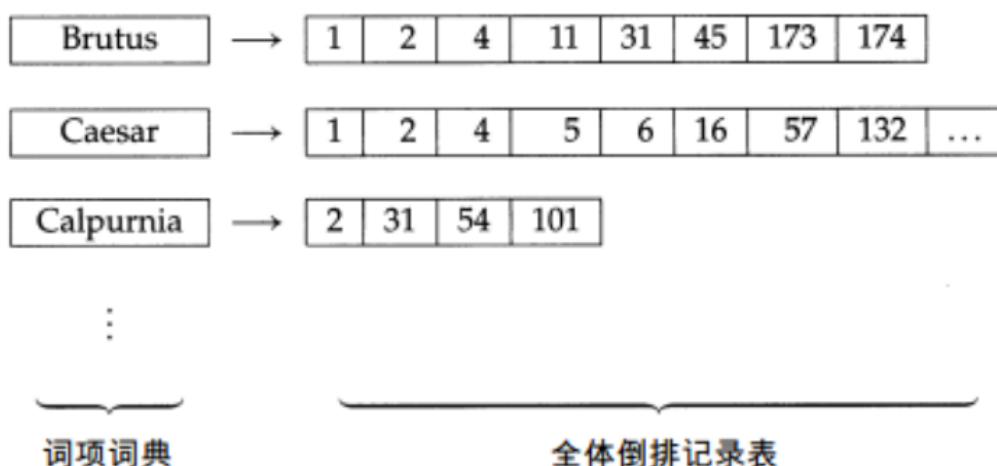
8.3.2 Term Dictionary

Elasticsearch为了能快速找到某个term，将所有的term排个序，二分法查找term，logN的查找效，就像通过字典查找一样。

这样我们可以用二分查找的方式，比全遍历更快地找出目标的term。

这个就是 term dictionary。

所以：反向索引分成两部分，如下图(来自《信息检索导论》)：



左边是词项词典(Term Dictionary)，右边是倒排记录表(Posting)。

在Lucene中，词典和倒排是分开存储的，词典存储在.tii和.tis文件中。

而倒排又分为两部分存储，第一部分是文档号和词频信息，存储在.frq中；另一部分是词的位置信息，存储在.prx文件中。

有了term dictionary之后，可以用logN次磁盘查找得到目标。

问题是：现在再看起来，似乎和传统数据库通过B+Tree的方式类似啊，为什么说比B+Tree的查询快？

8.3.3 Term Index

B-Tree通过**减少磁盘寻道次数**来提高查询性能，Elasticsearch也是采用同样的思路

但是磁盘的随机读操作仍然是非常昂贵的（一次random access大概需要10ms的时间）。

所以尽量少的读磁盘，有必要把一些数据缓存到内存里。**Elasticsearch直接通过内存查找term，不读磁盘**

但是整个term dictionary本身又太大了，无法完整地放到内存里。

于是就有了term index。term index有点像一本字典的大的章节表，或者说，像一本书的目录。

比如：

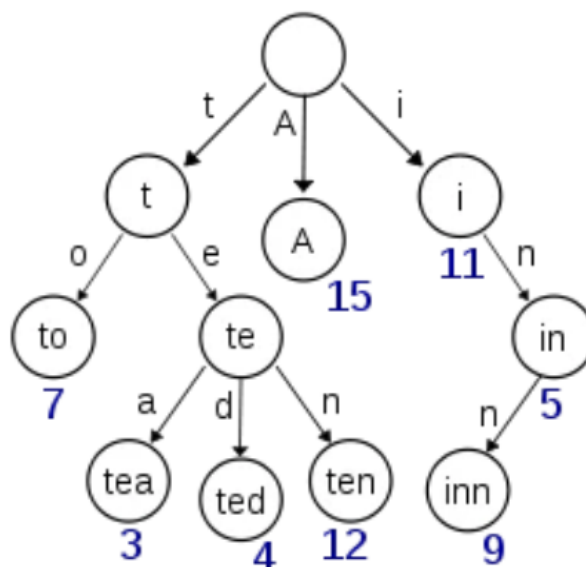
```
A开头的term ..... Xxx页
C开头的term ..... Xxx页
E开头的term ..... Xxx页
```

如果所有的term都是英文字符的话，可能这个term index就真的是26个英文字符表构成的了。

但是实际的情况是，term未必都是英文字符，term可以是任意的byte数组。

而且26个英文字符也未必是每一个字符都有均等的term，比如x字符开头的term可能一个都没有，而s开头的term又特别多。

实际的term index，的内部结构，类似一棵 trie 树：



例子是一个包含“A”，“to”，“tea”，“ted”，“ten”，“i”，“in”，和“inn”的 trie 树。

Term Dictionary与Term Index存储，Term Dictionary文件的后缀名为tim，Term Index文件的后缀名是tip。

Lucene为词典做了一层前缀索引(Term Index)，这个索引在Lucene4.0以后采用的数据结构是FST (Finite State Transducer)，一种前缀树的变种，可以称之为前缀索引。

这种数据结构占用空间很小，Lucene打开索引的时候将其全量装载到内存中，加快磁盘上词典查询速度的同时减少随机磁盘访问次数。

8.3.4 Trie树（前缀树，字典树）

Trie，又经常叫前缀树，字典树等等。

- 一个节点的所有子孙都有相同的前缀，也就是这个节点对应的字符串，而根节点对应空字符串。

- 一般情况下，不是所有的节点都有对应的值，只有叶子节点和部分内部节点所对应的键才有相关的值。

trie中的键通常是字符串，但也可以是其它的结构。它有很多变种，如后缀树，Radix Tree/Trie，PATRICIA tree，以及bitwise版本的crit-bit tree。

当然很多名字的意义其实有交叉。

在计算机科学中，trie，又称前缀树或字典树，是一种有序树，用于保存关联数组，其中的键通常是字符串。

与二叉查找树不同，键不是直接保存在节点中，而是由节点在树中的位置决定。

trie的算法可以很容易地修改为处理其它结构的有序序列，比如一串数字或者形状的排列。

这棵树不会包含所有的 term，它包含的是 term 的一些前缀。

通过 term index 可以快速定位到 term dictionary 的某个 offset，然后从这个位置再往后顺序查找。

再加上一些压缩技术（搜索 Lucene Finite State Transducers），term index 的尺寸可以只有所有term 的尺寸的**几十分之一**，使得用内存缓存整个 term index 变成可能。整体上来说就是这样的效果：



8.3.5 为什么ES检索比Mysql快

现在我们可以回答“为什么 Elasticsearch/Lucene 检索可以比 Mysql 快”了。

Mysql 只有 term dictionary 这一层，是以 b+tree 排序的方式存储在磁盘上的。

检索一个 term 需要若干次（1-3次）的 random access 的磁盘操作。

而ES/ Lucene 在 term dictionary 的基础上添加了 term index 来加速检索，term index 以类似前缀树的形式缓存在内存中。

从 term index 查到对应的 term dictionary 的 block 位置之后，再去磁盘上找 term，大大减少了磁盘的 random access 次数，将3次 变成了1次。

附录：DSL常用

一、前言

工作中几乎每天都需要使用到ES查询数据，需要根据自己的查询需求构造DSL查询语句来实现，附录里边，是常用的DSL语句以及所遇到问题的解决方案，DSL语句灵活多变，可进行多种组合，平时多看看，希望对你有帮助。

二、DSL常用查看索引语句

2.1 查看所有索引的信息 (indices?v)

```
GET _cat/indices
# 以表格形式展示，推荐*
GET _cat/indices?v
```

2.2 查看特定索引的信息 (_stats)

```
index_name/_stats
```

2.3 查看索引的映射 (_mapping)

```
index_name/_mapping
```

2.4 查看索引的设置 (_settings)

```
GET index_name/_settings
```

2.5 查看索引的别名 (aliases)

```
GET _cat/aliases
```

2.6 查看索引的文档数量 (_count)

```
GET index_name/_count
```

2.7 查看索引的存储大小 (_stats/store)

```
GET index_name/_stats/store
```

2.8 查看索引的字段名 (_search)

```
GET index_name/_search?size=0
```

2.9 查看索引的分片信息（_cat/shards）

```
GET index_name/_cat/shards
```

2.10 查看索引的健康状态（_cluster/health）

```
GET _cluster/health/index_name
```

2.11 查看索引的索引速率（_cat/thread_pool）

```
GET _cat/thread_pool/index_name
```

2.12 查看索引的搜索速率（_cat/thread_pool/search）

```
GET _cat/thread_pool/search
```

2.13 查看索引的索引速率（_cat/thread_pool/index）

```
GET _cat/thread_pool/index
```

2.14 查看索引的删除速率（_cat/thread_pool/delete）

```
GET _cat/thread_pool/delete
```

2.15 查看索引的刷新速率（_cat/thread_pool/refresh）

```
GET _cat/thread_pool/refresh
```

2.16 查看索引的合并速率（_cat/thread_pool/merge）

```
GET _cat/thread_pool/merge
```

2.17 查看索引的查询速率 (_cat/thread_pool/get)

```
GET _cat/thread_pool/get
```

2.18 查看索引的更新速率 (_cat/thread_pool/update)

```
GET _cat/thread_pool/update
```

三、DSL常用查询语句

3.1 查询所有 (match_all)

查询所有字段，默认显示10条数据。

```
GET index_name/?search
{
  "query": {
    "match_all": {
    }
  }
}
```

3.2 关键词全文查询 (match)

```
GET index_name/?search
{
  "query": {
    "match": {
      "user": "YiShuoIT"
    }
  }
}
```

3.3 精确匹配查询 (term)

```
GET index_name/?search
{
  "query": {
    "term": {
      "user": "YiShuoIT"
    }
  }
}
```


3.4 多值精确匹配查询 (terms)

```
GET index_name/?search
{
  "query": {
    "terms": {
      "user": ["YiShuoIT", "YiShuo"]
    }
  }
}
```

3.5 通配符查询 (wildcard)

```
GET index_name/?search
{
  "query": {
    "wildcard": {
      "user": "*YiShuo*"
    }
  }
}
```

3.6 多值通配符查询 (多wildcard)

```
GET index_name/?search
{
  "query": {
    "bool": {
      "should": [
        {
          "wildcard": {
            "command": "curl*password*"
          }
        },
        {
          "wildcard": {
            "user": "YiShuo*"
          }
        }
      ]
    }
  }
}
```

3.7 字段前缀查询 (prefix)

```
GET index_name/?search
{
  "query": {
    "prefix": { "user": "Yi" }
  }
}
```

3.8 多值字段前缀查询 (多prefix)

```
GET index_name/?search
{
  "query": {
    "bool": {
      "should": [
        {
          "prefix": {
            "user": "Yi"
          }
        },
        {
          "prefix": {
            "command": "curl"
          }
        }
      ]
    }
  }
}
```

3.9 模糊匹配查询 (fuzzy)

`fuzziness` 参数用于指定模糊匹配的容忍度。它可以是一个数字（表示编辑距离）或一个字符串（表示模糊度）。例如，“2”表示编辑距离为 2，“auto”表示根据术语的长度自动计算模糊度。

```
GET index_name/?search
{
  "query": {
    "fuzzy": {
      "user": {
        "value": "yi",
        "fuzziness": "2"
      }
    }
  }
}
```

3.10 正则表达式查询 (regexp)

```
GET index_name/?search
{
  "query": {
    "regexp": {
      "user": ".*YiShuo.*"
    }
  }
}
```

3.11 根据当天时间查询 (range + filter + now)

```
GET index_name/?search
{
  "query": {
    "bool": {
      "filter": {
        "range": {
          "timestamp": {
            "gte": "now/d",
            "lte": "now"
          }
        }
      }
    }
  }
}
```

3.12 根据自定义时间段查询 (range + filter + datetime)

```
GET index_name/?search
{
  "query": {
    "bool": {
      "filter": {
        "range": {
          "timestamp": {
            "gte": "2023-12-01 00:00:00",
            "lt": "2023-12-02 23:59:59",
            "time_zone": "+08:00",
            "format": "yyyy-MM-dd HH:mm:ss"
          }
        }
      }
    }
  }
}
```

3.13 满足条件查询 (must)

```

GET index_name/?search
{
  "query": {
    "bool": {
      "filter": {
        "range": {
          "timestamp": {
            "gte": "2023-12-01 00:00:00",
            "lt": "2023-12-02 23:59:59",
            "time_zone": "+08:00",
            "format": "yyyy-MM-dd HH:mm:ss"
          }
        }
      },
      "must": [
        {
          "wildcard": {
            "command": "*"
          }
        },
        {
          "wildcard": {
            "ip": "192.*"
          }
        }
      ]
    }
  },
  "size": 1
}

```

3.14 排除条件查询 (must_not)

```

GET index_name/?search
{
  "query": {
    "bool": {
      "filter": {
        "range": {
          "timestamp": {
            "gte": "2023-12-01 00:00:00",
            "lt": "2023-12-02 23:59:59",
            "time_zone": "+08:00",
            "format": "yyyy-MM-dd HH:mm:ss"
          }
        }
      },
      "must_not": [
        {
          "wildcard": {
            "ip": "127.*"
          }
        },
        {
          "wildcard": {

```

```
        "ip": "localhost*"
      }
    }
  ]
},
"size": 1
}
```

3.15 包含key查询 (must + exists)

```
GET index_name/?search
{
  "query": {
    "bool": {
      "must": [
        {
          "wildcard": {
            "user": "Yi*"
          }
        },
        {
          "exists": {
            "field": "address"
          }
        }
      ]
    }
  },
  "size": 1
}
```

3.16 不包含key查询 (must_not + exists)

```
GET index_name/?search
{
  "query": {
    "bool": {
      "must": [
        {
          "wildcard": {
            "user": "Yi*"
          }
        }
      ],
      "not_must": [
        {
          "exists": {
            "field": "address"
          }
        }
      ]
    }
  }
}
```

```
    }  
  },  
  "size": 1  
}
```

3.17 排序查询 (sort)

3.17.1 降序

```
GET index_name/?search  
{  
  "query": {  
    "wildcard": {  
      "ip": "192.168.*"  
    }  
  },  
  "sort": [  
    {  
      "timestamp": {  
        "order": "desc"  
      }  
    }  
  ]  
}
```

3.17.2 升序

```
GET index_name/?search  
{  
  "query": {  
    "wildcard": {  
      "ip": "192.168.*"  
    }  
  },  
  "sort": [  
    {  
      "timestamp": {  
        "order": "asc"  
      }  
    }  
  ]  
}
```

3.18 指定返回字段查询 (_source)

```
GET index_name/?search  
{  
  "query": {  
    "wildcard": {  
      "ip": "192.168.*"  
    }  
  }  
}
```

```

    }
  },
  "sort": [
    {
      "timestamp": {
        "order": "desc"
      }
    }
  ],
  "_source": [
    "user",
    "ip",
    "phone",
    "address"
  ]
}

```

3.19 指定条数查询 (size)

最多单次查询10000条，超过10000条需要分页查询。

```

GET index_name/?search
{
  "query": {
    "wildcard": {
      "ip": "192.168.*"
    }
  },
  "sort": [
    {
      "timestamp": {
        "order": "desc"
      }
    }
  ],
  "_source": [
    "user",
    "ip",
    "phone",
    "address"
  ],
  "size": 10000
}

```

3.20 分页查询 (from)

```

GET index_name/?search
{
  "query": {
    "wildcard": {
      "ip": "192.168.*"
    }
  }
}

```

```

},
"sort": [
  {
    "timestamp": {
      "order": "desc"
    }
  }
],
"_source": [
  "user",
  "ip",
  "phone",
  "address"
],
"from": 1,
"size": 100
}

```

3.21 关联多个index查询

GET index_name_one,index_name_two/?search

```

{
  "query": {
    "bool": {
      "must": [
        {
          "wildcard": {
            "user": "Yi*"
          }
        }
      ],
      "must_not": [
        {
          "wildcard": {
            "ip": "127.*"
          }
        },
        {
          "wildcard": {
            "ip": "localhost*"
          }
        }
      ]
    }
  },
  "size": 1
}

```

GET index_name_*/?search

```

{
  "query": {
    "bool": {
      "must": [
        {
          "wildcard": {

```



```

        "user": "Yi*"
      }
    },
    "must_not": [
      {
        "wildcard": {
          "ip": "127.*"
        }
      },
      {
        "wildcard": {
          "ip": "localhost*"
        }
      }
    ]
  },
  "size": 1
}

```

3.22 布尔查询 (bool)

bool语句是常用的用于构建复杂查询逻辑的语句。bool语句可以通过组合多个条件子句来实现逻辑运算，包括must、must_not、should和filter。

3.22.1 bool+must查询

```

GET index_name/?search
{
  "query": {
    "bool": {
      "must": [
        {
          "wildcard": {
            "user": "Yi*"
          }
        }
      ],
      "must_not": [
        {
          "wildcard": {
            "ip": "127.*"
          }
        },
        {
          "wildcard": {
            "ip": "localhost*"
          }
        }
      ]
    }
  },
  "size": 1
}

```

3.22.2 bool+must_not查询

```
GET index_name/?search
{
  "query": {
    "bool": {
      "must": [
        {
          "wildcard": {
            "user": "Yi*"
          }
        }
      ],
      "must_not": [
        {
          "wildcard": {
            "ip": "127.*"
          }
        },
        {
          "wildcard": {
            "ip": "localhost*"
          }
        }
      ]
    }
  },
  "size": 1
}
```

3.22.3 bool+should查询

```
GET index_name/?search
{
  "track_total_hits": true,
  "query": {
    "bool": {
      "must": [
        {
          "bool": {
            "should": [
              {
                "wildcard": {
                  "host": "www.baidu.com"
                }
              },
              {
                "wildcard": {
                  "host": "www.qq.com"
                }
              }
            ]
          }
        }
      ]
    }
  }
}
```

```

    }
  },
  {
    "bool": {
      "should": [
        {
          "wildcard": {
            "user": "*龙*"
          }
        },
        {
          "wildcard": {
            "user": "*虎*"
          }
        }
      ]
    }
  }
],
"sort": [
  {
    "timestamp": {
      "order": "desc"
    }
  }
],
"size": 1
}

```

3.22.4 bool+filter查询

```

GET index_name/?search
{
  "track_total_hits": true,
  "query": {
    "bool": {
      "filter": {
        "range": {
          "timestamp": {
            "gte": "2023-11-27 00:00:00",
            "lt": "2023-12-03 23:59:59",
            "time_zone": "+08:00",
            "format": "yyyy-MM-dd HH:mm:ss"
          }
        }
      }
    }
  },
  "sort": [
    {
      "timestamp": {
        "order": "desc"
      }
    }
  ]
}

```

```

    }
  ],
  "size": 0
}

```

3.22.5 bool多种组合 (bool+must+must_not+should+filter)

```

GET index_name/?search
{
  "track_total_hits": true,
  "query": {
    "bool": {
      "filter": {
        "range": {
          "timestamp": {
            "gte": "2023-11-27 00:00:00",
            "lt": "2023-12-03 23:59:59",
            "time_zone": "+08:00",
            "format": "yyyy-MM-dd HH:mm:ss"
          }
        }
      },
      "must": [
        {
          "bool": {
            "should": [
              {
                "wildcard": {
                  "host": "www.baidu.com"
                }
              },
              {
                "wildcard": {
                  "host": "www.qq.com"
                }
              }
            ]
          }
        }
      ],
      "should": [
        {
          "wildcard": {
            "user": "*龙*"
          }
        },
        {
          "wildcard": {
            "user": "*虎*"
          }
        }
      ]
    }
  }
}

```

```

    ],
    "must_not": [
      {
        "wildcard": {
          "address": {
            "value": "*广东*"
          }
        }
      }
    ]
  },
  "sort": [
    {
      "timestamp": {
        "order": "desc"
      }
    }
  ],
  "size": 0
}

```

3.23 match、term、terms、should、must_not、bool、filter区别

match查询是全文搜索查询，根据相关性匹配查询字符串，进行分词处理。

term查询是精确匹配查询，直接与文档中的词项进行比较，不进行分词处理。

terms查询是多值匹配查询，匹配多个值中的任意一个。

should查询表示满足任意一个条件即可匹配成功，用于构建OR逻辑关系。

must_not查询表示条件不能满足才能匹配成功，用于排除特定条件。

bool查询是复合查询，通过组合多个查询子句构建复杂的查询逻辑，包括**must**、**should**和**must_not**。

filter查询用于过滤文档，不计算相关性分数，仅根据条件进行精确匹配，提高查询性能。

1234567

四、DSL常用统计语句

4.1 aggs terms

```

GET index_name/?search
{
  "track_total_hits": true,
  "query": {
    "match_all": {}
  },
  "aggs": {
    "NAME": {
      "terms": {
        "field": "user",
        "size": 10
      }
    }
  }
}

```

4.2 aggs cardinality

```
GET index_name/?search
{
  "track_total_hits": true,
  "query": {
    "match_all": {}
  },
  "aggs": {
    "NAME": {
      "cardinality": {
        "field": "user"
      }
    }
  }
}
```

4.3 aggs value_count

```
GET index_name/?search
{
  "track_total_hits": true,
  "query": {
    "match_all": {}
  },
  "aggs": {
    "NAME": {
      "value_count": {
        "field": "user"
      }
    }
  }
}
```

4.4 aggs terms+script

```
GET index_name/?search
{
  "track_total_hits": true,
  "query": {
    "match_all": {}
  },
  "aggs": {
    "NAME": {
      "terms": {
        "script": "doc['user'] + '####' + doc['ip']",
        "size": 1000
      }
    }
  }
}
```

```
}  
}
```

4.5 aggs多重组合

```
GET index_name/?search  
{  
  "track_total_hits": true,  
  "query": {  
    "match_all": {}  
  },  
  "aggs": {  
    "NAME": {  
      "terms": {  
        "field": "user",  
        "size": 10  
      },  
      "aggs": {  
        "NAME": {  
          "terms": {  
            "script": "doc['ip'] + '####' + doc['address']",  
            "size": 1000  
          }  
        }  
      }  
    }  
  }  
}
```

4.6 aggs语句参数解析

这些聚合语句可以根据具体的需求进行组合和嵌套，以实现更复杂的统计和分析操作。通过使用这些聚合语句，可以从查询结果中提取有用的统计信息，进行数据分析、可视化和业务洞察。

terms聚合：按字段进行分组，并统计每个分组的文档数量。
date_histogram聚合：按时间间隔对日期字段进行分组，并统计每个时间间隔内的文档数量。
range聚合：将字段的值划分为不同的范围，并统计每个范围内的文档数量。
histogram聚合：将数值字段的值划分为不同的区间，并统计每个区间内的文档数量。
avg聚合：计算数值字段的平均值。
sum聚合：计算数值字段的总和。
min聚合：找到数值字段的最小值。
max聚合：找到数值字段的最大值。
cardinality聚合：计算字段的基数（不重复值的数量）。
top_hits聚合：返回每个分组中的顶部文档。
extended_stats聚合：计算数值字段的统计信息，包括平均值、标准差、最小值、最大值等。
percentiles聚合：计算数值字段的百分位数。
geo_distance聚合：按地理距离对地理坐标字段进行分组，并统计每个距离范围内的文档数量。
filter聚合：根据指定的过滤条件对文档进行聚合。
nested聚合：在嵌套的文档结构中进行聚合操作。
value_count聚合：计算某个字段的值的数量。
stats聚合：计算数值字段的统计信息，包括平均值、总和、最小值、最大值和文档数量。
scripted_metric聚合：使用自定义脚本计算聚合结果。

五、所遇到问题的解决方案

5.1 解决统计显示所有数量而不是10000问题

添加 "track_total_hits": true

```
GET index_name/?search
{
  "track_total_hits": true,
  "query": {
    "match_all": {
    }
  }
}
```

5.2 解决must与should同时使用must不生效问题

将should整个包装成must一个条件就能解决

```
GET index_name/?search
{
  "query": {
    "bool": {
      "must": [
        {
          "bool": {
            "should": [
              {
                "wildcard": {
                  "user": "*Yi*"
                }
              },
              {
                "wildcard": {
                  "user": "*龙*"
                }
              },
              {
                "wildcard": {
                  "user": "*虎*"
                }
              }
            ]
          }
        }
      ]
    }
  },
  "size": 10000
}
```


说在最后：有问题找老架构取经

百亿级数据存储架构，一定是一个超级牛掰的简历亮点项目，黄金项目，稍微晚点把全量的架构方案和视频进行发布。

这个项目写入简历，面试的时候如果大家能对答如流，如数家珍，基本上面试官会被你震惊到、吸引到。

最终，让面试官爱到“不能自己、口水直流”。offer，也就来了。

在面试之前，建议大家系统化的刷一波 5000页《[尼恩Java面试宝典](#)》V174，在刷题过程中，如果有啥问题，大家可以来找 40岁老架构师尼恩交流。

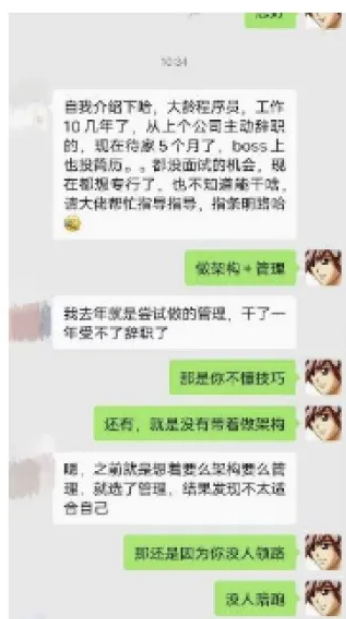
另外，如果没有面试机会，可以找尼恩来帮扶、领路。

- 大龄男的最佳出路是 架构+ 管理
- 大龄女的最佳出路是 DPM，



架构师尼恩6

大龄程序员，找不到出路了



54分钟前

架构师尼恩6：大龄男最佳出路： 架构+管理

架构师尼恩6：大龄女最佳出路： DPM

女程序员如何成为DPM，请参见：

[DPM（双栖）陪跑，助力小白一步登天，升格 产品经理+研发经理](#)

领跑模式，尼恩已经指导了大量的就业困难的小伙伴上岸。

前段时间，领跑一个40岁+就业困难小伙伴拿到了一个**年薪100W**的offer，小伙伴实现了 **逆天改命**。

技术自由圈 技术圣经系列

技术自由圈 7个 学习圣经 PDF

- 1 《SpringCloud Alibaba 学习圣经》
v1 版，发布日期：2023年03月15日 377页 PDF
- 2 《Docker 学习圣经》
v2 版，发布日期：2023年03月12日 145页 PDF
- 3 《Kubernetes 学习圣经》
v1 版，发布日期：2023年04月11日 530页 PDF
- 4 《响应式圣经》
v2 版，发布日期：2023年02月14日 114页 PDF
- 5 《缓存之王 Caffeine 红宝书》
v2 版，发布日期：2022年11月30日 175页 PDF
- 6 《队列之王 Disruptor 红宝书》
v1 版，发布日期：2022年10月05日 75页 PDF
- 7 《Prometheus+grafna 红宝书》
v1 版，发布日期：2022年10月08日 100页 PDF

技术自由圈 3个 高并发圣经 PDF

- 8 《Java高并发核心编程 卷1 加强版：NIO、Netty、Redis、ZooKeeper》
加强版，发布日期：2023年01月01日 609页 PDF
- 9 《Java高并发核心编程 卷2 加强版：多线程、锁、JMM、JUC、高并发设计模式》
加强版，发布日期：2022年11月14日 448页 PDF
- 10 《Java高并发核心编程 卷3 加强版：亿级用户Web应用架构与实战》
加强版，发布日期：2022年11月14日 481页 PDF

技术自由圈 面试圣经 40个 面试题 PDF

- 11 《尼恩 Java 面试宝典》40个专题
v60 版，发布日期：2023年03月11日 4000页 PDF

领取方式：技术自由圈 公众号



硬核推荐：尼恩Java硬核架构班

详情：<https://www.cnblogs.com/crazymakercircle/p/9904544.html>



尼恩java 硬核架构班



已经发布

- ★ 《高性能RPC的基础实操之：从0到1开始IM撸一个IM》
- ★ 《分布式高性能RPC的基础实操之：千万级用户分布式IM实操- 含简历指导》
- ★ 《亿级用户超高并发秒杀实操- 含简历指导》
亮点：助力小伙伴搞定70W年薪，N个涨薪50%，**2023夏招面试涨薪神器**
- ★ 《横扫全网，工业级elasticsearch底层原理与高并发、高可用架构实操》
亮点：40岁老架构师细致解读，处处透着分布式、高性能中间件的原理和精髓
- ★ 《第1部曲：超级底层：葵花宝典（高性能秘籍）__架构师视角解读OS操作系统》
亮点：大制作解读OS操作系统，并揭秘mmap、pagecache、zerocopy等底层的底层原理
2023夏招面试涨薪大神器
- ★ 《Rocketmq视频第2部曲：横扫全网工业级 rocketmq 高可用（HA）底层原理和实操》
亮点：起底式、绞杀式解读 rocketmq如何保障消息的可靠性？
- ★ 《Rocketmq视频第3部曲：超级内功篇、横扫全网 rocketmq 源码学习以及3高架构模式解读》
亮点：大制作解读 Rocketmq源码以及3高架构模式，助力大家内力猛增
- ★ 《Rocketmq视频第4部曲：10Wqps消息推送中台架构、设计、编码、测试实操》
亮点：Netty实操、分库分表实操、Rocketmq工业级使用实操
- ★ 《架构师内功篇：横扫全网 netty 高性能、高并发架构 底层原理、源码学习》
- ★ 《架构师实操篇：redis cluster 工业级高可用实操》
- ★ 《架构师实操篇：100W级别QPS日志平台实操》
- ★ 《彻底穿透：skywalking 源码(代表链路跟踪)+Java agent+bytebuddy 探针》
- ★ 《超高并发场景100Wqps三级缓存组件原理和实操》
- ★ 《全链路异步超底层原理和实操：手写hystrix熔断+webflux+Lettuce+Dubbo》

规划中



左手大数据 (写入简历, 让简历 蓬荜生辉、金光闪闪)

HBASE + Flink + ElasticSearch 原理、架构、真刀实操



右手云原生 (写入简历, 让简历 蓬荜生辉、金光闪闪)

K8S + Devops + ServiceMesh 原理、架构、真刀实操

架构师实操篇: 基于netty 手写 rpc 框架- 参考 dubbo、seata rpc框架

架构师实操篇: go语言学习, 以及基于 go 手写 rpc 框架

架构师实操篇: 千万级任务调度平台 架构与实操- 基于尼恩17年的亿级搜索项目

架构师实操篇: 工业级 亿级文档搜索 平台 架构与实操- 基于尼恩17年的亿级搜索项目

特色

会员制

提供技术方向指导,
职业生涯指导, 少躺坑, 少弯路

简历指导

这个很重要,
对于挪窝涨薪来说

实操性

以上项目, 都是老架构师
在生产上实操过的项目

非水货

40岁老架构师, 不是水货架构师
《Java高并发三部曲》为证

架构班（社群 VIP）的起源：

最初的视频，主要是给读者加餐。很多的读者，需要一些高质量的实操、理论视频，所以，我就围绕书，和底层，做了几个实操、理论视频，然后效果还不错，后面就做成迭代模式了。

架构班（社群 VIP）的功能：

提供高质量实操项目整刀真枪的架构指导、快速提升大家的：

- 开发水平
- 设计水平
- 架构水平

弥补业务中 CRUD 开发短板，帮助大家尽早脱离具备 3 高能力，掌握：

- 高性能
- 高并发
- 高可用

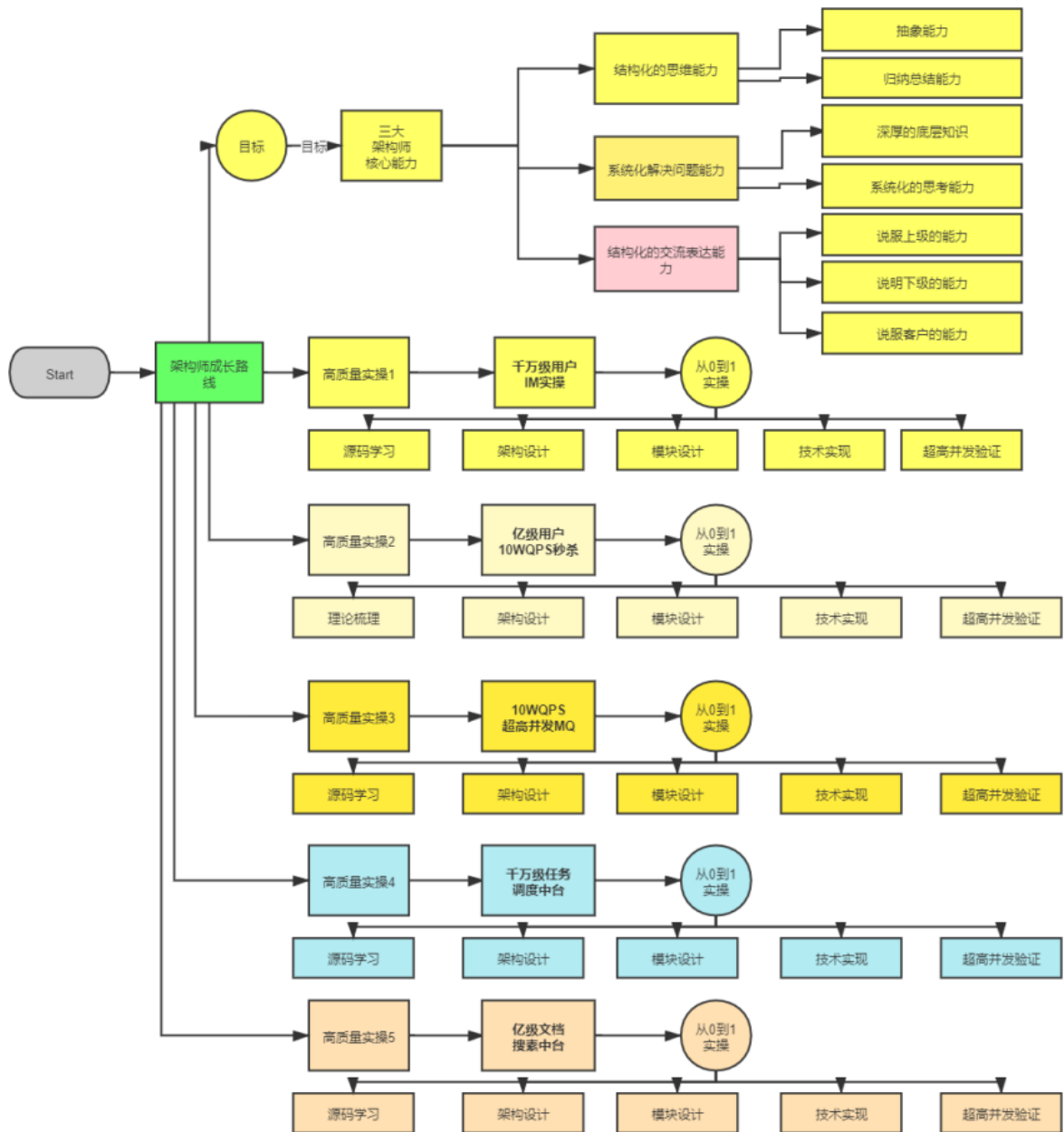
作为一个高质量的架构师成长、人脉社群，把所有的卷王聚焦起来，一起卷：

- 卷高并发实操
- 卷底层原理
- 卷架构理论、架构哲学
- 最终成为顶级架构师，实现人生理想，走向人生巅峰

架构班（社群 VIP）的目的：

- 高质量的实操，大大提升简历的含金量，吸引力，增强面试的召唤率
- 为大家提供九阳真经、葵花宝典，快速提升水平
- 进大厂、拿高薪
- 一路陪伴，提供助学视频和指导，辅导大家成为架构师
- 自学为主，和其他卷王一起，卷高并发实操，卷底层原理、卷大厂面试题，争取狠卷 3 月成高手，狠卷 3 年成为顶级架构师

N 个超高并发实操项目：简历压轴、个顶个精彩



【样章】第 17 章：横扫全网 Rocketmq 视频第 2 部曲：工业级 rocketmq 高可用（HA）底层原理和实操

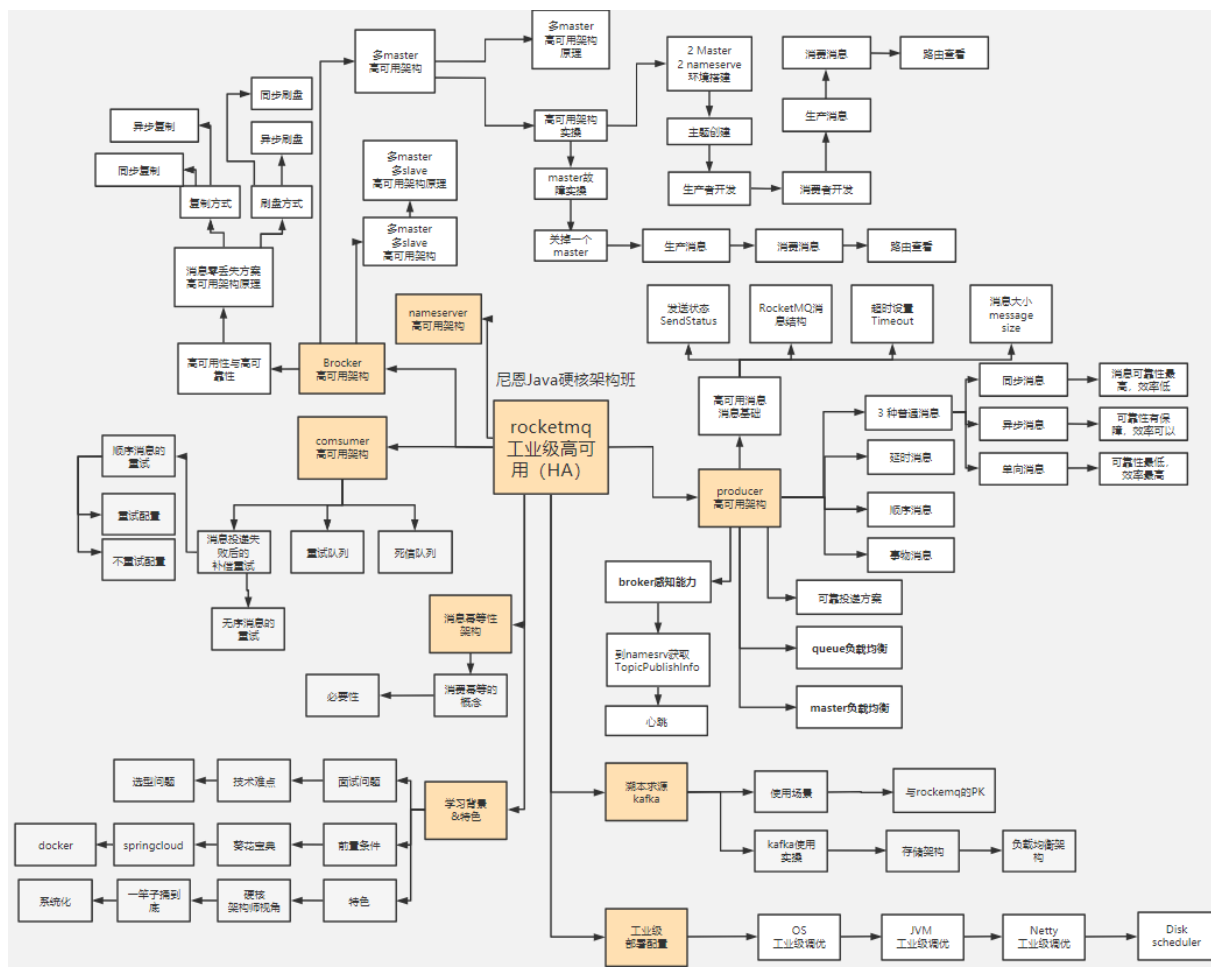
工业级 rocketmq 高可用底层原理，包含：消息消费、同步消息、异步消息、单向消息等不同消息的底层原理和源码实现；消息队列非常底层的主从复制、高可用、同步刷盘、异步刷盘等底层原理。

工业级 rocketmq 高可用底层原理和搭建实操，包含：高可用集群的搭建。

解决以下难题：

- 1、技术难题：RocketMQ 如何最大限度的保证消息不丢失的呢？RocketMQ 消息如何做到高可靠投递？
- 2、技术难题：基于消息的分布式事务，核心原理不理解
- 3、选型难题：kafka or rocketmq，该娶谁？

下图链接：<https://www.processon.com/view/6178e8ae0e3e7416bde9da19>



成功案例：2 年翻 3 倍，35 岁卷王成功转型为架构师

详情：<http://topcoder.cloud/forum.php?mod=forumdisplay&fid=43&page=1>

最新 最后发表 热门 精华

成功案例：[1057号卷王] 3年小伙拿到外企offer，薪酬涨了200%

卷王1号 超级版主 前天 17:41

成功案例：[645号卷王] 4年经验卷王逆袭，被毕业后，反涨24W

卷王1号 超级版主 2022-9-21

成功案例：[878号卷王] 小伙8年经验，年薪60W

卷王1号 超级版主 2022-8-13

年薪70W案例：通过尼恩的指导，小伙伴年薪从40W涨到70W

卷王1号 超级版主 2022-2-11

成功案例：[493号卷王] 5年小伙拿满意offer，就业寒冬季逆涨30%

卷王1号 超级版主 前天 17:43

成功案例：[250号卷王] 就业极寒时代，收offer 涨25%

卷王1号 超级版主 前天 17:38

成功案例：[612号卷王] 就业极寒时代，从外包到自研

卷王1号 超级版主 前天 17:15

成功案例：[913号卷王] 热烈祝贺6年经验卷王，年薪40W

卷王1号 超级版主 2022-9-21

成功案例：[959号卷王] 4年经验卷王，喜获百度、Boss直聘等N个优质offer，最高涨100%

卷王1号 超级版主 2022-9-21

成功案例：[529号卷王] 5年经验卷王喜收2大offer，最高涨5K

卷王1号 超级版主 2022-9-21

成功案例：[811号卷王] 热烈祝贺7年经验卷王，薪酬涨30%

卷王1号 超级版主 2022-9-21

成功案例：[287号卷王] 不惧大寒潮，卷王逆市收4 offer，涨30%，可喜可贺

卷王1号 超级版主 2022-5-30

成功案例：[1002号卷王] 5月份“被毕业”，改简历后，斩获顶级央企Offer，涨薪7000+

卷王1号 超级版主 2022-7-5

成功案例: [7号卷王] 热烈祝贺小伙伴涨薪120%

1 卷王1号 超级版主 2022-8-13

成功案例: [134号卷王] 大三小伙卷1年, 斩获顶级央企Offer, 成功逆袭

1 卷王1号 超级版主 2022-7-6

成功案例: [1008号卷王] 5年经验卷王收42W offer, 月涨8000, 可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例: [453号卷王] 非全日制 6年卷王喜提3 offer, 年薪30W, 可喜可贺

1 卷王1号 超级版主 2022-5-21

成功案例: [924号卷王] 6年卷王喜提4 offer, 最高涨薪9000, 可喜可贺

1 卷王1号 超级版主 2022-5-21

成功案例: [15号卷王] 4年卷王入职 微软, 涨薪50%, 可喜可贺

1 卷王1号 超级版主 2022-5-12

成功案例: [527号卷王] 4年卷王喜提2 offer, 涨薪50%, 可喜可贺

1 卷王1号 超级版主 2022-5-13

成功案例: [788号卷王] 3年卷王喜提优质Offer, 涨薪60%

1 卷王1号 超级版主 2022-5-11

成功案例: 热烈祝贺: 非全日制卷王, 喜提2个心仪offer, 面3家过2家

1 卷王1号 超级版主 2022-4-21

成功案例: [693号卷王] 二线城市6年卷王喜提4大优质Offer, 含央企offer, 最高薪酬35W

1 卷王1号 超级版主 2022-4-16

成功案例: [85号卷王] 双非2本小伙, 春招大捷, 喜提9个offer, 最高薪酬近30万

1 卷王1号 超级版主 2022-4-14

成功案例: [741号卷王] 卷王逆袭! 6年小伙从很少面试机会到搞定35K*14薪Offer

1 卷王1号 超级版主 2022-4-12

成功案例: [642号卷王] 热烈祝贺, 6年卷王喜提优质国企offer

1 卷王1号 超级版主 2022-4-7

成功案例: [796号卷王] 热烈祝贺, 36岁卷王喜提52万优质offer

1 卷王1号 超级版主 2022-3-25

❑ 成功案例: [15号卷王] 小伙卷1年, 涨薪9K+, 喜收ebay等多个优质offer

① 卷王1号 超级版主 2022-3-24

❑ 成功案例: [821号卷王] 小伙狠卷3个月, 喜提10多个offer

① 卷王1号 超级版主 2022-3-21

❑ 成功案例: [736号卷王] 3年半经验收22k offer, 但是小伙志存高远, 冲击25k+

① 卷王1号 超级版主 2022-3-20

❑ 成功案例: 热烈祝贺1群小卷王offer拿到手软, 甚至拒了阿里offer

① 卷王1号 超级版主 2022-3-16

❑ 简历案例: 简历一改, 腾讯的邀请就来了! 热烈祝贺, 小伙收到一大堆面试邀请

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 祝贺我圈两大超级卷王, 一个过了阿里HR面, 一个过了阿里2面

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 小伙伴php转Java, 卷1.5年Java, 涨薪50%, 喜收多个优质offer

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 4年小伙狠卷半年, 拿到 移动、京东 两大顶级offer

 尼恩 超级版主 2022-3-5

❑ 成功案例: [267号卷王] 助力3年经验卷王, 拿到蜂巢的17k x 14薪的offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [143号卷王] 二本院校00后卷神, 毕业没到一年跳到字节, 年薪45W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [494号卷王] 尼恩分布式事务助力卷王拿到 中信银行offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [76号卷王] 2线城市卷王, 狠卷1.5年, 喜收22K offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [429号卷王] 小伙伴在社群卷5个月, 涨8k+

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [154号卷王] 双非学校毕业卷王, 连拿 京东到家&滴滴 两个大厂Offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [232号卷王] 涨薪10K, 继续卷向食物链顶端

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: 狠卷1年技术, 喜收 腾讯、阿里、微软三大Offer, 最高年薪56W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [449号卷王] 应届毕业卷王喜收 滴滴offer, 年薪33W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [551号卷王] 小伙伴学完后, 成功进入大厂, 并且推荐自己的朋友加VIP学习

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [214号卷王] 助力2年经验卷王, 成功拿到17K月薪

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [92号卷王] 课程实操助力社群小伙伴喜收 喜马拉雅Offer

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: 社群卷王小伙伴成功过了滴滴三面 获滴滴Offer

① 卷王1号 超级版主 2022-2-10

❑ [612号卷王]滴滴小伙伴, 蹲点考察半年, 觉得靠谱后加入 疯狂创客圈

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [732号卷王] 尼恩助力3年经验卷王收获 京东offer, 年薪35W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [558号卷王] 2年经验卷王, 喜收 网易和阿里子公司两个优质offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [569号卷王] 双非应届生卷王, 喜收字节跳动实习offer

① 卷王1号 超级版主 2022-2-25

❑ 成功案例: [420号卷王] 狠卷1年, 卷王涨薪80%, 涨薪12000元!

① 卷王1号 超级版主 2022-2-25

❑ 成功案例: [76号卷王] 通过尼恩1年半的指导, 专科学历小伙伴从0.8K涨到22K

① 卷王1号 超级版主 2022-2-10

简历优化后的成功涨薪案例（VIP含免费简历优化）

6年专科，2年翻4倍

2年从8K涨到 35K

2021年从8K涨到22K

2022年涨到35K

秘诀：

简历指导+ 狠狠卷





4年经验卷王逆袭 被毕业后，反涨24W

7月改简历

8月30日晒offer



小伙5月份"被毕业"，改简历后 斩获顶级央企Offer 涨薪7000+

5月29日改简历

7月5日晒offer



卷王逆袭成功案例 武汉6年喜收4个优质offer 最高的年薪35W

2月9日改简历

4月15日晒offer

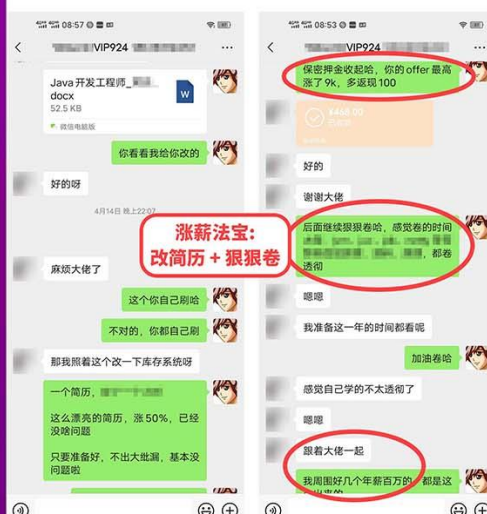


卷王逆袭成功案例

6年小伙喜提4个Offer 最高涨9k，年薪35W

4月14日改简历

5月17日晒offer



卷王逆袭成功案例

5年经验小伙收2个offer 最高涨薪8k，年薪42W

5月9日改简历

5月30日晒offer

秘诀:
简历指导+ 狠卷3高

以此为样
大家狠狠卷
打造最卷IT社群

卷王逆袭成功案例

非全日制 6年经验卷王 喜提3个Offer，年包30W

5月9日改简历

5月18日晒offer

面试法宝:
改简历+ 狠狠卷

卷王逆袭成功案例

寒五冻六之际卷王大逆袭 收3大offer，涨30%

5月17日改简历

5月27日晒offer

秘诀:
简历指导+ 狠卷3高

卷王逆袭成功案例

4年卷王入职微软，涨50%

3月7日改简历

5月12日晒offer

涨薪法宝:
改简历+ 狠狠卷

4年小伙喜收百度、Boss直聘等N个顶级Offer 最高涨幅100%

6月27日改简历 9月19日晒offer

**秘诀：
改简历 + 狠狠卷**

有个offer选择问题

boss直聘和小满之间，boss那边给的offer，工资比小满高，但是小满那边给的offer，福利比boss高，不知道该怎么选，如果你应该怎么选呀。

还有没有其他offer的，还有一个小满的。

总体上看boss和小满之间选一个

了

boss比小满年包多7w以上。

我这张幅都接近百分之百了

太牛啦

按照你那个思路结合上次指导的内容改了几个点，发出去确实大上了很多，加上一些指标的说明确实会让人眼前一亮的感觉。

明天难时候有没了我和我看下哈，一是看两个点我实在不知道该怎么改了，二是我在推荐表达这方面确实不行，还得让你帮忙把把关。

卷王逆袭成功案例 4年卷王入收2个offer，涨50%

3月23日改简历 5月12日晒offer

offer决策图

n+1 电商erp，部门成熟，人数多，加班少。

n+2 做业务中台 内部系统，新部门，人数少，加班多。

又搞到个offer

能搞定两个offer，不尴尬

现在很多小伙伴面试机会都没有

涨了多少呀

地点在哪里

涨50%的样子

忘记你工作几年啦，大概工作几年啦

**涨薪法宝：
改简历 + 狠狠卷**

这个不用加粗

都是一样，加粗了反而没有效果

下边请继续，有不懂的问我

小伙大三暑期很焦虑 跟着尼恩卷一年 校招斩获顶级央企Offer

去年5月19日加入VIP群 今年7月5日晒offer

**秘诀：
狠狠卷书+视频**

邀请你加入群聊

尼恩老师

我校招去华润电力控股有限公司了

跟着你卷了一年 大学顺便拿了几个国奖

不错不错，这是央企

放空中心

这太牛啦

跟着你卷了一年 大学顺便拿了几个国奖

其实拿到手的也就一个a类国一

尼恩老师 大三的暑期实习找不到 现在准备秋招应该没关系吧

看身边 总是很焦虑 自己算法这一块卡住

网盘里边有算法视频

去刷一刷吧

秋招来得及

谢谢大佬 不过经过几个月练习 看您写的书比之前轻松多了

趁着还是学生这段时间 慢慢把知识吃透

嗯哪，我的书，比较深

期待大佬的下一本书，已经迫不及待去学习了

小伙高中学历 薪酬涨120%

5月6日改简历 7月22日晒offer

你这块估计要送你668

模块的开发工作，就这个三个哈

哈哈，不用了老师，您真的讲了就行 光今天晚上辅导就值几千了

老弟很感谢您

还有很多其他的模块

面试官要问

就说是其他人做的

嗯哪，好

**秘诀：
改简历 + 狠狠卷**

之前你的工资是多少来的

翻了一翻

我得送你多少奖金来的

不知道，原价给老弟就行了

咱们得说这话呀

老弟拿着您的那个高开发改造亮点，所向披靡。

从从到尾给面试官讲明白

后面继续狠狠卷哈

卷王逆袭成功案例

非全日制卷王 面试3家
收2个offer 涨薪30%

4月13日改简历

4月21日晒offer

5年卷王喜收2大Offer

最高涨5K

5月19日改简历

9月13日晒offer

面试法宝:
改简历 + 面试题

卷王逆袭成功案例

3年经验卷王，涨60%

4月16日改简历

5月11日晒offer

卷王逆袭成功案例

双非二本小伙春招大翻身
喜提9大offer

2月22日改简历

4月13日晒offer

面试法宝:
改简历 + IM实操

涨薪法宝:
改简历 + 狠狠卷

9大offer 最高年薪30万

序号	公司	部门	岗位	薪资结构	总包
1	某公司	研发部	java后端开发	18.5k+14.5k+5k+200k/每月+500k/期权	>30w
2	某公司	研发部	java后端开发	16k+14k	22.4w
3	某公司	研发部	java后端开发	15k+15k+餐补+100k/每月	>22.5w
4	某公司	研发部	java后端开发	11k+13k	14.2w
5	某公司	研发部	java后端开发	14k	>16.8w
6	某公司	研发部	java后端开发	9k	9k
7	某公司	研发部	java后端开发	9k+餐补+社保+公积金	10k+餐补+房补
8	某公司	研发部	java后端开发	10k+餐补+房补	17w
9	某公司	研发部	java后端开发	10k+餐补+房补	17w

修改简历找尼恩（资深简历优化专家）

- 如果面试表达不好，尼恩会提供 简历优化指导
- 如果项目没有亮点，尼恩会提供 项目亮点指导
- 如果面试表达不好，尼恩会提供 面试表达指导

作为 40 岁老架构师，尼恩长期承担技术面试官的角色：

- 从业以来，“阅历”无数，对简历有着点石成金、改头换面、脱胎换骨的指导能力。
- 尼恩指导过刚刚就业的小白，也指导过 P8 级的老专家，都指导他们上岸。

如何联系尼恩。尼恩微信，请参考下面的地址：

语雀：<https://www.yuque.com/crazymakercircle/gkkw8s/khigna>

码云：<https://gitee.com/crazymaker/SimpleCrayIM/blob/master/疯狂创客圈总目录.md>